

Nativx Assistant

Masterclass complet — cod & arhitectură (RO, for dummies)

- 1 · Manual complet cap-coadă (21 capitole)
- 2 · Deep Dive pe tot proiectul (11 valuri, ~93 fișiere)
- 3 · Prompt de predare (motorul cu 9 secțiuni)

Generat din docs/MASTERCLASS-*.md — conținut fidel fișierelor.

Nativx Assistant — Manual complet „cap-coadă” (RO, for dummies)

Scop: după ce citești acest document, poți explica **fiecare** componentă, urmări **orice** mesaj prin cod, depana în producție și modifica proiectul **fără să-l strici**.

Regula de aur peste tot: **codul e sursa de adevăr**. Unde documentația (CLAUDE.md, docstring-uri) bate cu codul, câștigă codul — și îți spun de fiecare dată unde bate.

Fiecare afirmație e ancorată cu `fișier:linie`. Complementar cu diagramele din `ARCHITECTURE-WORKFLOWS.md` (10 diagrame Mermaid verificate pe cod).

Cuprins

1. Ce este proiectul (imaginea mare)
 2. Cele 7 procese și cum comunică
 3. Harta fișierelor pe subsisteme
 4. TurnContext — obiectul care curge prin tot
 5. INTRARE: webhook-ul (plafon de corp, semnătură, dedupe L1)
 6. COADA + WORKER: consumer, debounce, lock, reaper
 7. handle_turn — inima unui tur, pas cu pas
 8. PIPELINE-ul de 11 stagii, stagiul cu stagiul
 9. Bugetul de context (mesaj lung → păstrăm doar o parte)
 10. Cusătura LLM (retry, tokeni maximi, tool loop)
 11. Căutarea de produse (hibrid lexical + semantic)
 12. Grounding-ul anti-halucinație (validator + compose)
 13. IEȘIRE: Sender TX + dispatcher + canale
 14. Memoria conversației (state, istoric, rezumat, profil)
 15. Baza de date (pool-uri, RLS, tabele)
 16. Guvernanța costului (3 niveluri)
 17. Tratarea erorilor (niciodată tăcere)
 18. Proactiv (mesaje inițiate de bot)
 19. Referință de configurare (toate butoanele)
 20. Ghid de depanare pe subsisteme
 21. Exerciții + validare
-

1. Ce este proiectul

În trei propoziții: un vânzător-robot pe WhatsApp (și web + Telegram) pentru magazine online din România. Clientul scrie „ce ser cu vitamina C aveți sub 100 lei?”, iar botul caută în catalogul magazinului și răspunde ca un vânzător priceput — cu prețuri și linkuri **reale**, nu inventate. E **multi-tenant**: aceeași aplicație servește mai multe magazine, fiecare izolat de celelalte prin `business_id`.

Obsesia centrală: botul nu are voie să inventeze prețuri, produse sau linkuri. Modelul AI produce doar cuvinte + ID-uri de produs; **codul** pune faptele reale din baza de date. Asta se numește *grounding* și e apărat de două mecanisme (validatorul + compose), pe care le vei înțelege complet la [capitolul 12](#).

Trei principii pe care le vei vedea peste tot: 1. **Pipeline liniar** — mesajul trece printr-o „bandă rulantă” cu 11 stații, în ordine fixă, fără sărituri înapoi. 2. **LLM doar în 2 puncte** — triaj (model ieftin „nano”) + agent (model „mini”). Tot restul e cod determinist. 3. **Niciodată tăcere** — mereu iese ceva spre client. Degradare: mini → retry → nano → template → om notificat.

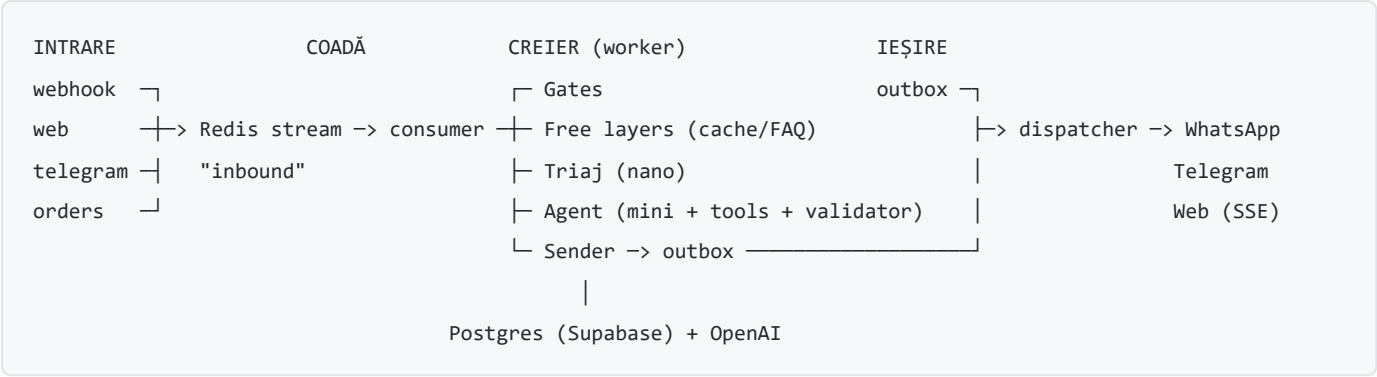
Stack: Python 3.12 asyncio, FastAPI (webhook), Redis Streams (coadă), Postgres 16 pe Supabase (DB), OpenAI (LLM), Meta Cloud API (WhatsApp), Telegram Bot API, widget web (SSE).

2. Cele 7 procese

Nu e „o aplicație”. Sunt **7 procese** care rulează în paralel (din `docker-compose.yml`). Analogie: un restaurant cu bucătărie, ospătari, oficiu de livrare și un manager de mentenanță — fiecare face un lucru.

Proces	Comandă	Rol	Fișier entry point
webhook	<code>uvicorn src.webhook.app:app</code>	Poarta HTTP: primește de la Meta/web/shop	<code>src/webhook/app.py</code>
worker	<code>python -m src.worker.consumer</code>	Creierul: coadă → pipeline → răspuns	<code>src/worker/consumer.py</code>
dispatcher	<code>python -m src.worker.dispatcher</code>	Singura ieșire: outbox → canale	<code>src/worker/dispatcher.py</code>
scheduler	<code>python -m src.jobs.scheduler</code>	Mentenanță (rollup, embed, lifecycle, curățenie)	<code>src/jobs/scheduler.py</code>
proactive	<code>python -m src.proactive.scheduler</code>	Mesaje inițiate de bot	<code>src/proactive/scheduler.py</code>
telegram-poller	<code>python -m src.channels.telegram.poller</code>	Ascultă Telegram	<code>src/channels/telegram/poller.py</code>
redis	—	Coadă + lock-uri + dedupe + contoare cost	(serviciu)

Cum comunică: nimeni nu cheamă pe altul direct. Totul curge prin **coada Redis** (stream-ul `inbound`) și prin **tabelul outbox**. De ce? Ca `webhook` -ul să răspundă în <50ms (Meta face retry agresiv), iar procesarea grea (LLM, DB) să se facă separat în `worker`. E ca la restaurant: ospătarul ia comanda și pleacă (bonul pe cui), nu stă lângă bucătar până e gata mâncarea.



3. Harta fișierelor pe subsisteme

```
src/
├─ config.py          ← TOATE setările/bugetele/plafoanele (cap. 19)
├─ models.py          ← TurnContext + toate dataclass-urile (cap. 4)
├─ redis_bus.py       ← client Redis: XADD inbound, dedupe L1, lock-uri
├─ meta_client.py     ← MetaClient (trimite pe WhatsApp)
├─
├─ webhook/          ← INTRAREA (margine subțire, fără DB)
│   └─ app.py         ← FastAPI: GET verify + POST inbound + POST orders
│   └─ signature.py   ← verifică X-Hub-Signature-256
│   └─ meta.py        ← parsează payload Meta → envelope neutru
│   └─ body_limit.py  ← plafon de mărime pe corp (anti-OOM)
│   └─ orders.py      ← webhook comenzi → atribuire
│   └─
├─ worker/           ← CREIERUL
│   └─ consumer.py    ← bucla Redis: citește, rezolvă tenant, rutează
│   └─ processor.py   ← handle_turn: inima turului + Sender TX
│   └─ runner.py      ← motorul pipeline-ului (stagiile în ordine)
│   └─ context.py     ← bugetul de context pt prompturi (cap. 9)
│   └─ compose.py     ← compunerea rich + grounding (cap. 12)
│   └─ dispatcher.py  ← IEȘIREA: outbox → canale
│   └─ debounce.py    ← coalesce mesaje rapide (3s)
│   └─ callback.py    ← navigare carusel
│   └─ badges.py      ← badge-uri de card din semnale reale
│   └─ summarizer.py  ← rezumat rolling (nano)
│   └─ profile.py     ← extragere profil + lead_score (nano)
│   └─ reply_split.py ← spargere text lung în 2
│   └─ limits.py      ← cost guard + rate limit (contoare Redis)
│   └─ order_gate.py  ← poarta de comandă (login web)
│   └─ stages/        ← CELE 11 STAGII
│       └─ gates.py   ← 1: filtre (bot activ? risc? moderare? poză?)
│       └─ language.py ← 2: detect RO/HU/EN
│       └─ clarify.py  ← 3: reia un slot în așteptare
│       └─ greeting.py ← 4: salut → welcome determinist
│       └─ alias.py    ← 5: match exact pe fraze aprobate
│       └─ cache.py    ← 6: cache semantic
│       └─ faq.py      ← 7: răspuns din FAQ
│       └─ triage.py   ← 8: nano clasifică ruta
│       └─ handoff.py  ← 9: escaladare la om
│       └─ agent.py    ← 10: mini + tool loop + validator (1411 linii!)
│       └─
├─ agent/
│   └─ llm.py         ← SINGURUL loc care vorbește cu OpenAI (cap. 10)
│   └─ prompt_builder.py ← system prompt GENERAT din DB
│   └─ tool_definitions.py ← schemele tool-urilor (OpenAI)
│   └─ usage.py       ← contorizare tokeni/cost
│   └─ pricing.py     ← tarife LLM
```

```

|
├─ tools/                ← tool-urile agentului (cod determinist)
|   ├─ catalog_tools.py  ← search_products, get_product_details, compare (cap. 11)
|   ├─ commerce_tools.py ← cart_add, checkout_link, back_in_stock
|   ├─ orders_tools.py   ← check_order, reorder
|   ├─ faq_tools.py       ← faq_lookup
|   ├─ handoff_tools.py   ← request_human
|   └─ taxonomy.py        ← mapare concerns
|
├─ db/
|   ├─ connection.py      ← pool-uri asyncpg + RLS + izolare (cap. 15)
|   └─ queries/           ← SQL per domeniu (contacts, conversations, catalog, ...)
|
├─ channels/             ← marginile de canal
|   ├─ base.py            ← ChannelSender Protocol + Capability matrix
|   ├─ media.py           ← download media (Vision)
|   ├─ telegram/          ← client + poller
|   └─ web/               ← sender SSE + render
|
├─ web/                  ← gateway widget web
|   ├─ app.py             ← /web/bootstrap, /messages, /stream, /chat
|   ├─ session.py         ← sesiune HMAC
|   └─ identity.py        ← login passthrough (JWT)
|
├─ domain/               ← DomainPack (config per-vertical)
├─ proactive/            ← motor + inițiatori + template-uri (cap. 18)
├─ jobs/                 ← rollup, embed, lifecycle, cleanup
├─ lang/                  ← detectare limbă
└─ gdpr/                 ← ștergere/export

```

4. TurnContext — obiectul care curge prin tot

 `src/models.py:477`

Într-un pipeline liniar ai nevoie de **un singur obiect** care poartă tot: cine e clientul, ce a scris, ce s-a discutat, ce a decis fiecare stadiu. Țasta e `TurnContext`. Analogie: o **tavă de spital** care circulă pe bandă — fiecare stație pune ceva pe ea, nimeni nu ia de pe ea ce a pus altă stație.

Regula ABSOLUTĂ

Fiecare câmp are EXACT un stadiu care-l scrie. (proprietarul e notat în docstring, `models.py`)

```

route: RouteDecision | None = None      # owner: Triaaj      (models.py:494)
retrieval: RetrievalResult | None = None # owner: Retrieval/tools (models.py:495)
reply: Reply | None = None              # owner: orice stagiou → early exit (models.py:496)
halt: bool = False                     # owner: Gates (tăcere intenționată) (models.py:497)
state_patch: dict = ...                 # owner UNIC: Agent      (models.py:505)
usage: TurnUsage | None = None          # owner: runner-ul      (models.py:509)

```

DE CE atât de strict? Dacă doi bucătari scriu pe același bon, nu mai știi cine a greșit. Dacă doar Triaajul scrie `route`, când `route` e greșit știi exact unde să te uiți. Debugging **liniar**, nu ghicit.

Metodele-helper (le folosești mereu)

- `emit(type_, **props)` (models.py:511) — un stagiou adaugă un event de analytics fără să știe cum e scris. Injectează automat `turn_id` (`setdefault`, ca să nu suprascrie unul explicit). Asta e „black box recorder”-ul: toate event-urile unui tur au același `turn_id` → poți reconstrui traiectoria.
- `set_reply(text, ...)` (models.py:527) — setează `ctx.reply` → **oprește banda** (runner-ul iese).
- `halt_silent(reason)` (models.py:520) — **singura** tăcere permisă (a preluat un om). Emite `gate_halt`.
- `set_clarify(text, field, resume_route, ...)` (models.py:590) — pune o întrebare ȘI memorează slotul. Subtilitate: dacă re-întrebi **același** `field`, `attempts++` (anti-bucă → escaladare după N).
- `set_rich_reply` / `set_comparison_reply` / `set_offer` — variante pentru carduri / tabel / buton.

ConversationState — hidratare DEFENSIVĂ

 `src/models.py:191` (`from_jsonb`)

State-ul vine din DB ca JSON. Codul îl transformă în obiect, dar **niciodată nu are încredere** în el:

```

for p in raw.get("displayed_products") or []:
    pid = p.get("product_id") or p.get("id")
    if pid is None or p.get("name") is None or p.get("price") is None:
        continue # produs incomplet → SĂRIT, nu crapă
    products.append(ProductRef(product_id=pid, name=p["name"], price=float(p["price"])))

```

DE CE paranoia? `from_jsonb` rulează la **FIECARE tur pe hot path**. Un cursor ne-numeric (state vechi / editat manual) NU trebuie să crape turul — altfel conversația s-ar bloca permanent (încalcă „niciodată tăcere”).

Principiul 8 — state = ref-uri, nu obiecte: stochezi `{product_id, name, price}`, nu tot produsul. Bugetul e 8KB (impus în DB cu CHECK). Când ai nevoie de detalii, re-hidratezi din DB. Câmpurile din state: `displayed_products`, `active_search` (pool + cursor de paginare), `pending_question`, `asked_intents`, `constraints`, `search_constraints`, `cart`, `state_version`.

5. INTRARE: webhook-ul

 `src/webhook/app.py`

Marginea de intrare e **subțire și fără DB**. Face doar: verifică, deduplică rapid, pune pe coadă, răspunde 200. De ce subțire? Meta face retry agresiv la timeout → trebuie ACK în <50ms.

Middleware: plasa anti-OOM (`app.py:25`)

```
@app.middleware("http")
async def _request_size_guard(request, call_next):
    cl = request.headers.get("content-length")
    if cl is not None and int(cl) > get_settings().webhook_max_body_bytes: # 256KB global
        return PlainTextResponse("payload too large", status_code=413)
```

DE CE? VPS-ul e mic (1vCPU, 4GB, 0 swap). Un POST uriaș ar putea OOM-ui procesul înainte să-l parzezi. Cap global 256KB (webhook), rafinat per-endpoint (web = 16KB). Țasta e **primul** filtru din Diagrama 3 (`CAP`).

GET /webhook — handshake Meta (`app.py:62`)

Meta trimite `hub.mode=subscribe` + `hub.verify_token` + `hub.challenge`. Dacă token-ul corespunde, întorci challenge-ul ca **text brut** (Meta compară exact). Altfel 403. Se face o singură dată, la configurare.

POST /webhook — mesajele inbound (`app.py:80`)

Pașii, toți rapizi (fără LLM, fără DB):

```
raw = await enforce_body_cap(request, ...) # 1. plasă de mărime
signature = request.headers.get("X-Hub-Signature-256")
if not verify_meta_signature(app_secret, raw, signature):
    return 403 # 2. semnătura pe corpul BRUT
payload = json.loads(raw) # 3. JSON valid? altfel 400
for event in parse_webhook(payload):
    if await seen_before(redis, event.channel_account_id, event.provider_msg_id):
        continue # 4. dedupe L1 (Redis) – retry Meta
    await enqueue_inbound(redis, event.to_dict()) # 5. XADD pe stream
for status in parse_statuses(payload):
    await enqueue_inbound(redis, status.to_dict()) # statusurile NU se deduplică
return 200 # 6. ACK rapid
```

Detalii pe care le vei fi întrebat: - **Semnătura pe corpul BRUT** (`signature.py`): dacă ai reserializa JSON-ul, ai schimba bytes → semnătura n-ar mai potrivi. De aceea verifici `raw`, nu `payload`. - **Dedupe L1** (`redis_bus.py:53`, `seen_before`) = `SET NX EX` pe (`channel_account_id`, `provider_msg_id`). E **primul strat** de dedupe. Al doilea (durabil, în DB) e în worker — vezi cap. 7. De ce două? Redis se poate goli (FLUSHALL/restart); DB-ul prinde ce scapă. - **Statusurile nu se deduplică**: `delivered` și `read` au același `wamid` → dacă le-ai deduplica, ai pierde `read`. - **Redis jos** → **503** (`app.py:119`): NU pierdem tăcut. 503 → Meta reîncearcă, iar la retry dedupe-ul prinde ce-a intrat deja.

POST /webhook/orders/{business_id} (`app.py:127`)

Webhook de comenzi de la platforma de shop. Margine subțire, ca Meta: verifică HMAC pe corpul brut, pune un envelope `kind="order"` pe stream. Comenzile au `business_id` din path (autentificat de secret), deci NU trec prin

`resolve_channel`.

Ce se întâmplă cu un mesaj TROLL / semnătură falsă? 403 imediat, fără să atingem coada sau DB-ul.

6. COADA + WORKER

 `src/worker/consumer.py`

Webhook-ul a pus mesajul pe stream și a plecat. Workerul îl scoate și-l procesează. E o buclă infinită.

run_consumer — bucla (`consumer.py:275`)

```
while True:
    await consume_once(...)          # citește + procesează
    if now - last_reap >= 30s:
        await reap_pending(...)      # recuperează mesaje de la workeri morți
```

consume_once — un ciclu (`consumer.py:187`)

```
resp = await redis.xreadgroup(CONSUMER_GROUP, consumer_name, {STREAM_INBOUND: ">"}, count=10, block=2000)
```

XREADGROUP explicat: un teanc de bonuri (stream) + mai mulți ospătari (workeri) în același „grup”. Redis dă fiecare bon **unui singur** ospătar. `>` = „bonuri pe care nu le-a văzut nimeni din grup”. Asta permite să rulezi 3 workeri fără dublă procesare. Fiecare are nume unic (`worker-<hostname>` , `consumer.py:324`).

Pentru fiecare mesaj (`consumer.py:210-230`):

```
if event.get("kind") == "message":
    if typing_enabled:
        asyncio.create_task(_safe_typing(registry, event)) # "typing..." INSTANT, fire-and-forget
        await debouncer.add(event, msg_id) # NU ACK aici – Debouncer-ul ACK-uește DUPĂ flush
    else:
        await process_event(pool, redis, event) # status/callback/order → imediat
        await redis.xack(...)
```

⚠ Gaura #1 (NX-140), memoreaz-o: dacă `process_event` aruncă o excepție (`consumer.py:228-230`), mesajul e **ACK-uit** și doar logat (`log.exception("eroare la procesarea mesajului")`). Meta a primit deja 200 → nu re-trimite → **clientul nu primește nimic**. E singura încălcare structurală a principiului 6. DE CE totuși ACK? Ca un mesaj „otrăvit” (care crapă mereu) să nu blocheze coada pentru toți. Fix corect: re-queue cu contor + fallback în outbox la epuizare. **La depanare „botul n-a răspuns”: ăsta e primul grep.**

_safe_typing — de ce ocolește outbox-ul (`consumer.py:59`)

„typing...” se trimite **direct** prin ChannelSender, nu prin outbox. Un typing livrat cu 3s întârziere (după ce răspunsul a plecat) e inutil. Singura excepție documentată de la „un singur punct de ieșire” (săgeata punctată în Diagrama 1).

Gardat pe capabilitatea `TYPING`, nu pe canal.

Debouncer — coalesce (`debounce.py`)

Dacă clientul scrie „vreau un ser” apoi „...cu vitamina C” apoi „...sub 100 lei” în 2 secunde, nu vrei 3 tururi.

Debouncer-ul așteaptă 3s de liniște, adună mesajele într-unul singur (N→1), apoi procesează. ACK-ul se face **după flush reușit** (durabilitate — NX-87): dacă workerul moare înainte de flush, mesajele rămân ne-ACK-uite și reaper-ul le recuperează.

process_event — rezolvarea tenantului (`consumer.py:102`)

Aici „un envelope neutru” devine „un tur pentru businessul X”. Trei ramuri:

1. **order** (`consumer.py:109`): `business_id` din envelope → `process_order` direct (fără resolve).
2. **resolve channel** (`consumer.py:123`): `python async with admin_conn(pool) as conn: channel = await resolve_channel(conn, channel_kind, channel_account_id)` **ĂSTA e SINGURUL query pe `admin_conn`** (pool privilegiat). Trebuie să afli `business_id` **înainte** să știi tenantul — e operația care-l derivă. Orice alt query pe `admin_conn` = bug de izolare.
3. **status** (`consumer.py:137`): idempotent (pe `provider_msg_id`) → fără lock, scriere directă.

Lock per conversație (`consumer.py:157-169`)

```
sender_key = f"{channel_kind}:{channel_account_id}:{sender_external_id}"
got = await acquire_conv_lock(redis, business_id, sender_key, token, ttl_s=30)
if locked is False:
    # altă replică procesează aceeași conversație
    await _requeue_busy(redis, event, settings) # re-queue cu backoff
return
```

DE CE? 2 workeri + 2 mesaje rapide → ambele citesc același `state`, ambele scriu → race. Lock-ul serializează tururile **aceleiași** conversații. **Redis jos** → **fail-open** (procesează oricum: mai bine un risc mic de race decât blocarea traficului).

⚠ **Gaura #2 (NX-140):** `_requeue_busy` (`consumer.py:90`) — după `conv_lock_max_requeues` (default 10) reîncercări, evenimentul e **DROPAT** cu un log. A doua cale de pierdere tăcută.

reap_pending — plasa de siguranță (`consumer.py:234`)

`XAUTOCLAIM` reclamă mesajele citite de un worker care a **murit** înainte de ACK (ne-ACK-uite > 60s) și le reprocesează. Închide gaura „consumer mort între citire și ACK”.

7. handle_turn — inima unui tur

📄 `src/worker/processor.py:418`

Cel mai important flux. Fan-out 45 (cel mai mare din cod). Rulează pe o conexiune **deja tenant-scoped**.

Pas 0 — pregătire (linii 440-449)

```
turn_id = str(uuid4())
verified_customer_ref = event.get("verified_customer_ref") # login web (NX-129)
identity_external_id = verified_customer_ref or sender_external_id
```

`verified_customer_ref or sender_external_id`: dacă marginea a verificat o identitate stabilă (login eshop, JWT), rezolvăm contactul pe EA (client stabil peste device-uri). Altfel, pe `visitor_id`-ul anonim.

Pas 1 — Dedupe Layer 2 (linia 456)

```
if provider_msg_id and not await claim_inbound(conn, business.id, provider_msg_id):
    return TurnResult(..., deduped=True) # deja procesat
```

Al doilea strat de dedupe, **durabil în DB**. Prinde retry-uri Meta care scapă de Redis (FLUSHALL/restart). **Guard ÎNAINTE de orice scriere** — un duplicat nu produce nici mesaj, nici outbox.

Subtilitate (NX-86, claim-or-resume): `claim_inbound` marchează „în lucru” (nu „gata”). Dacă turul crapă la mijloc, `completed_at` rămâne NULL → reaper-ul îl poate relua. Abia `mark_inbound_completed` (în TX-ul de outbox) îl finalizează.

Pas 2 — Contact + Conversație + Mesaj (linii 460-488)

```
contact = await get_or_create_contact(conn, business.id, channel_kind, identity_external_id, ...)
conv = await get_or_create_conversation(conn, business.id, contact.id, channel_id, ...)
await insert_message(conn, ..., Direction.INBOUND, Author.CONTACT, body=event.get("body"), ...)
await touch_last_inbound(conn, business.id, conv["id"]) # alimentează fereastra 24h
```

`touch_last_inbound` setează `last_inbound_at = now()` → alimentează fereastra de 24h (Meta permite mesaje libere doar 24h de la ultimul inbound al clientului). E derivat, nu un flag.

Pas 3 — Construiește TurnContext + încarcă memoria (linii 490-510)

```
ctx = TurnContext(
    turn_id=turn_id, business=business, contact=contact, message=InboundMessage(...),
    conversation_id=conv["id"],
    history=await get_recent_messages(conn, ...), # ultimele 8 mesaje
    state=ConversationState.from_jsonb(conv["state"]), # memoria (ref-uri, defensiv)
    summary=await get_summary_for_context(conn, ...), # rezumat rolling
    language=conv["locale"] or business.default_locale,
    bot_active=conv["bot_active"], handoff_until=conv["handoff_until"], ...
)
```

Astea 3 (`history` + `state` + `summary`) = **toată memoria** botului. Vezi cap. 14.

Pas 4 — Cost guard (linii 523-525)

```
if redis and cost_guard_enabled:
    await seed_daily_cost(conn, redis, business.id) # reseed lazy din usage_daily
    llm = await _llm_within_budget(ctx, redis, business, channel_kind=channel_kind)
```

`_llm_within_budget` (`processor.py:308`) verifică plafonul zilnic (business + per-contact). Depășit → întoarce `None` .
Cu `llm=None` , pipeline-ul rulează **degradat**: gates + cache merg, stagiile LLM sar. Niciodată nu crapă.

Pas 5 — Rulează pipeline-ul (linia 528)

```
await run_pipeline(ctx, PipelineDeps(conn=conn, redis=redis, llm=llm, media=media), stages)
```

Intră cele 11 stagii (cap. 8). Când se întoarce, `ctx.reply` e (poate) setat.

Pas 6 — Fără reply (linii 534-545)

```
if ctx.reply is None:
    if ctx.halt: log.info("tăcere intenționată (handoff)") # om preia – CORECT
    else:      log.info("tur procesat fără reply")
    if provider_msg_id: await mark_inbound_completed(...) # finalizează claim-ul!
    return TurnResult(...)
```

`mark_inbound_completed` și aici: chiar dacă nu răspunzi, turul e DONE → altfel reaper-ul l-ar reprocessa la infinit.

Pas 7 — Disclaimer + fragmentare (linii 551-563)

```
reply_text = ensure_disclaimer(ctx.reply.text, ctx.language) # art. 50 AI Act, idempotent, azi OFF
is_rich = ctx.reply.rich is not None
has_products = bool(ctx.reply.products)
if is_rich or has_products or not deliver:
    fragments = [reply_text] # rich/carusel/sync = 1 fragment
else:
    fragments = split_reply(reply_text, limit=200) # text lung → max 2 (citire ușoară pe telefon)
```

De ce rich/carusel nu se sparge? Spargerea lead-in-ului ar strica ordinea cardurilor.

Pas 8 — TRANZACȚIA Sender (linii 565-667) — cel mai important bloc

Totul atomic — sau tot, sau nimic:

```

async with conn.transaction():
    # 8a. Merge canonic de state
    new_state = conv["state"]
    if (is_rich or has_products) and ctx.reply.products:
        new_state = {**conv["state"], "displayed_products": ctx.reply.products}
    new_state = {**new_state, "pending_question": ctx.reply.pending_question}
    new_state = {**new_state,
        "constraints": ctx.state.constraints,          # slot-fill din clarify
        "asked_intents": ctx.state.asked_intents,      # anti-loop
        "search_constraints": ctx.state.search_constraints, # stiva de căutare (NX-133)
    }
    if not (is_rich or has_products):
        new_state = {**new_state, "active_search": None} # reset sesiune căutare
    if ctx.state_patch:
        new_state = {**new_state, **ctx.state_patch} # Agent are ULTIMUL cuvânt (cart etc.)

    # 8b. Mesaje outbound + rânduri outbox
    for i, frag in enumerate(fragments):
        out_msg_id = await insert_message(..., Direction.OUTBOUND, Author.BOT, body=frag,
            status="queued" if deliver else "sent",
            **(_message_usage_kwargs(ctx.usage) if i == 0 else {})) # cost/tokeni pe primul frag
        if not deliver: continue
        payload = {"type": "text", "to": sender_external_id, "text": frag, ...}
        if i == 0 and is_rich: payload["rich"] = asdict(ctx.reply.rich)
        elif i == 0 and has_products: payload["type"]="carousel"; payload["products"]=ctx.reply.products
        outbox_id = await enqueue_outbox(conn, ..., f"{turn_id}:{i}", payload) # idempotency key

    # 8c. Patch state cu optimistic lock
    await patch_conversation_state(conn, ..., new_state, conv["state_version"], touch_outbound=True)

    # 8d. Finalizează dedupe-ul ÎN aceeași TX
    if provider_msg_id: await mark_inbound_completed(...)

```

DE CE totul într-o tranzacție? Analogie bancară: retragi din A, depui în B — dacă se întrerupe la mijloc, banii dispar. Aici: mesaj outbound + outbox + patch state + finalizare dedupe = atomice. Altfel ai avea un răspuns fără outbox (client nu primește nimic) sau state scris fără mesaj (memorie coruptă).

- `idempotency_key = f"{turn_id}:{i}"` (`processor.py:651`): outbox are UNIQUE pe cheie → un replay nu inserează dublul. `turn:0` și `turn:1` pentru cele 2 fragmente.
- **Optimistic lock pe** `state_version` (`processor.py:656`): scrie doar dacă `state_version` din DB e încă cel citit la început. Un „check-and-set” — a doua apărare peste lock (vezi cap. 15).

 **Bug clasic „botul uită”:** dacă un stagiou mută un câmp **in-place** pe `ctx.state` (ex. `constraints`), dar processor-ul nu-l listează în merge-ul canonic (8a), câmpul trăiește doar în memorie și se pierde tăcut la write-back. **Regula:** orice câmp mutat in-place de un stagiou TREBUIE listat explicit la 8a.

Pas 9 — Post-tur async (linii 689-705)

```
post_acc, post_token = usage.push() # acumulator de usage SEPARAT
try:
    await _cache_writeback(...)      # scrie în semantic_cache (dacă cacheable)
    await _summarize_if_needed(...)  # rezumat rolling (nano)
    await _extract_profile_and_score(...) # profil + lead_score (nano)
finally:
    usage.pop(post_token)
```

Rulează **DUPĂ** ce răspunsul e în outbox — nu întârzie livrarea. „Temele de casă”: învață clientul, comprimă istoricul, memorează răspunsul. Toate **best-effort** (crapă → log, turul a răspuns deja). Acumulator separat ca apelurile nano de fundal să nu scape contabilității de cost (`phase="post_turn"`).

8. PIPELINE-ul de 11 stagii

 `src/worker/runner.py:47` (motor), `runner.py:207` (`DEFAULT_STAGES`)

Motorul (`run_pipeline`)

```
for stage in stages:
    before = acc.snapshot()
    started = perf_counter()
    await stage(ctx, deps)                # rulează stagiul
    ctx.emit("stage_completed", stage=name, latency_ms=...) # măsoară
    _record_stage_delta(...)              # cost/tokeni per stagiul
    if ctx.reply is not None or ctx.halt:  # EARLY EXIT
        ctx.emit("pipeline_early_exit", stage=name); break
```

Genialitatea: stagiile sunt funcții pure `async def stage(ctx, deps) -> None`. Runner-ul le rulează în ordine, măsoară latența/tokenii **în jur** (`before` / `after`), iese la primul `reply` sau `halt`. **Stagiile nu știu că sunt măsurate** (principiul 10). Adugi un stagiul nou fără să atingi runner-ul.

 **Discrepanță doc↔cod:** docstring-ul (`runner.py:8-10`) încă zice „un singur stagiul real (echo_stage)”. E STALE. CLAUDE.md zice „9 stagii” + „validator = stagiul 8”. În realitate sunt **11 stagii** și validatorul e cod în interiorul agentului, nu un stagiul. Codul câștigă.

Ordinea (de la ieftin la scump)

Ținta: 40-60% din trafic se oprește **înainte** de LLM. Stagiile sunt de la gratis la scump:

#	Stagiu	Fișier	Ce face	Cost
1	gates_stage	stages/gates.py:439	filtre: activ? blocat? risc? moderare? poză? PII?	gratis (+moderare)
2	language_stage	stages/language.py:27	detectează RO/HU/EN	gratis
3	clarify_resume_stage	stages/clarify.py:28	reia un slot în așteptare	gratis
4	greeting_stage	stages/greeting.py:184	pur salut → welcome determinist	gratis
5	alias_stage	stages/alias.py:46	match EXACT pe fraze aprobate	gratis (index)
6	cache_stage	stages/cache.py:89	cache semantic	1 embed
7	faq_stage	stages/faq.py:45	răspuns din FAQ	1 embed
8	triage_stage	stages/triage.py:212	nano clasifică ruta	1 nano
9	handoff_stage	stages/handoff.py:39	escaladare la om	gratis
10	agent_stage	stages/agent.py:957	mini + tool loop + validator	1-4 mini
11	fallback_stage	runner.py:169	dacă nimic → clarificare (nu tăcere)	gratis

8.1 Gates (stages/gates.py:439)

Primul stadiu real. 8 porți, în ordine, fiecare cu early-exit:

```

if not ctx.bot_active:          ctx.halt_silent("bot_inactive"); return      # 1. kill-switch
if ctx.contact.is_blocked:      ctx.halt_silent("contact_blocked"); return  # 2. blocklist
if handoff_until > now():       ctx.halt_silent("handoff_active"); return  # 3. om a preluat
if await _rate_limited(...):    return                                  # 4. rate limit
if await _moderation_blocked(...): return                                # 5. moderare (toxic)
reason = detect_risk(ctx.message.body)                                # 6. risc (om/legal)
if reason and handoff_enabled_for(channel): request_human + reply; return
if content_type == "image":     await _route_image(...)                # 7. poză → Vision
_apply_input_guardrails(ctx)    # 8. clamp+PII+injection

```

Detalii importante: - **Rate limit** (`_rate_limited` , `gates.py:323`): >20 mesaje/60s → un mesaj de throttle O SINGURĂ dată, apoi tăcere pe restul burst-ului. Fail-open dacă Redis e jos. - **Moderare** (`_moderation_blocked` , `gates.py:348`): apel OpenAI moderation (gratuit). Flagged → răspuns neutru + contor de flag-uri pe 24h; peste prag (3) → blochează contactul. **Fail-open**: fără cheie/API jos → trece (indisponibilitatea moderării nu tace tot traficul). NICIODATĂ corpul în analytics — doar categoriile. - **Risc** (`detect_risk` , `gates.py:122`): pattern-uri **determinate** (nu LLM) RO+HU+EN pentru „vreau un om” și „avocat/reclamație”. Escaladează DOAR pe canale cu handoff (`handoff_enabled_for`); pe web (fără operator) NU escaladează — botul asistă singur. - **Vision** (`_route_image` , `gates.py:385`): poză → descriere text → devine `ctx.message.body` . NU setează reply, doar **îmbogățește** inputul → triajul rutează SALES. **Fail-soft**: orice eșec → păstrează caption-ul util al pozei ca text de căutare (nu aruncă intenția). „Imagine → text → search” (nu avem image-embedding). - **Guardrails de input** (`_apply_input_guardrails` , `gates.py:255`) — **AICI e detaliul „mesaj lung → păstrăm doar o parte”**: `python if len(body) > INBOUND_BODY_MAX: # 2000 caractere ctx.message.body = body[:INBOUND_BODY_MAX] # TRUNCHIERE, nu rejection ctx.emit("body_truncated", chars=len(body)) # DOAR lungimea, nu corpul (P12) masked, counts = mask_pii(ctx.message.body) # telefon/email/iban/card → [telefon] etc. n = screen_injection(ctx) # "ignore previous" etc. → doar contor` - **Trunchiere la 2000 caractere** ( `INBOUND_BODY_MAX` , `config.py:16`): un mesaj mai

lung e **tăiat**, nu respins (P6). Aliniat cu validarea web (`max_length=2000`). Nota: e cap de *caractere*, plasă structurală. - **Mascare PII** (`mask_pii` , `gates.py:175`): telefonul/email/IBAN/card liber-tastate de client → `[telefon]` etc., ca să NU intre în prompt/analytics. Ordine: email → iban → card → telefon. Cardul e validat cu **Luhn** + lungime (15/16/19) + prefix IIN, ca să NU prindă un cod EAN de produs. - **Ecran injection** (`screen_injection`): „ignore previous instructions” etc. → doar **contor** (nu tăcem, nu escaladăm — ar premia abuzul). Apărarea reală = validatorul agentului.

8.2 Language (`stages/language.py:27`)

Detectează RO/HU/EN, setează `ctx.language`, persistă `conversations.locale` (scriere directă best-effort). **DE CE aici, înaintea straturilor gratuite?** Toate lookup-urile în FAQ/cache/template includ `locale` — un cache hit în limba greșită e un bug, nu un hit (principiul 11).

8.3 Clarify resume (`stages/clarify.py:28`)

Dacă `state.pending_question` e setat (am întrebat ceva la turul trecut), răspunsul scurt al clientului („200 lei”) e consumat **determinist** aici: umple slotul, setează ruta de reluare. Dacă a re-întrebat prea des (`attempts > clarify_max_attempts`, default 2) → escaladează (HANDOFF pe slot critic / SALES altfel). Rulează ÎNAINTE de greeting/cache/triage ca „200 lei” să nu fie tratat ca salut sau re-triat de la zero.

8.4 Greeting (`stages/greeting.py:184`)

Pur salut („bună”, „salut”) → mesaj de welcome branded, **determinist** (fără LLM). Nu verifică ruta, doar textul. Numele botului + sugestiile din `businesses.settings[“welcome”]`.

8.5 Alias (`stages/alias.py:46`)

Match **EXACT** al frazei normalizate în `intent_aliases` (status='approved'). Cel mai ieftin strat (index B-tree, zero token). Trei ieșiri: FAQ target → servește răspuns (early-exit); route/product/category → setează **doar** `ctx.route` (fără reply), iar cache/FAQ/triaj îl **respectă** (skip); miss → mergi la cache. Gol pe demo (se populează din shadow mode).

8.6 Cache semantic (`stages/cache.py:89`)

Query repetat → răspuns gata, fără LLM de generare. L1 exact (hash) → L2 cosine (embed + prag `cache_tau_high` = 0.92). Două tiere: `static` (fără produse, TTL zile) și `dynamic` (recomandare cu produse, TTL minute + snapshot de preț). La lookup pe dynamic se face **price-check**: dacă prețul s-a schimbat, e miss (nu servim preț învechit). Query-urile „realtime/contextual” (ex. „mai ieftin”) sar cache-ul.

8.7 FAQ (`stages/faq.py:45`)

Întrebări de cunoștințe (retur/livrare/garanție) → răspuns din `faqs` (un embed, fără generare). Lookup ÎNTOTDEAUNA `business_id + locale + cosine`. Trei praguri: `faq_tau_high` (0.78), `faq_tau_policy` (0.45, DOAR când mesajul conține o întrebare clară de politică — regex), și fallback de locale opțional. Gol pe demo (`faqs=0`).



Bug istoric aici (silent dead layer): un codec pgvector greșit făcea `float('')` → `DataError` pe ORICE lookup FAQ/cache → ambele straturi mureau **tăcut** (best-effort → miss). Nedetectat cât `faqs` era gol; a ieșit la primul seed real. Fix în `connection.py:118` (`_vector_encode` acceptă și literal deja formatat). **Lecția:** un strat best-effort care nu emite NICIUN event = excepție înghițită; caută warning-ul în log.

8.8 Triaj (stages/triage.py:212)

Primul touchpoint LLM. Nano clasifică în: `simple` | `sales` | `order` | `handoff` | `clarify`. Output JSON validat cu Pydantic (`TriageOut`, `triage.py:191`).

```
if ctx.route is not None: return          # clarify_resume a setat deja ruta → no-op (P3)
if deps.llm is None: return              # fără cheie → degradare (echo fallback)
categories = await list_category_slugs(...) # slug-urile valide din DB
user = f"Limba: {lang}\n{context}\n{history}\nMesaj: {body}\nCategorii valide: {categories}\n..."
raw = await deps.llm.classify_json(_SYSTEM, user) # nano, JSON forțat
out = TriageOut(**raw)
category_key = out.category_key if out.category_key in categories else None # inventat → aruncat!
```

Garduri deterministe (codul decide, nu nano): - **category_key inventat** (`triage.py:255`): dacă nano dă o categorie care nu-i în listă → o aruncă. Nu rutezi pe ghicit. - **Factual guard** (`triage.py:262`): dacă nano zice „simple” dar mesajul atinge un fapt de business (reducere/ preț/stoc/politică — regex `_FACTUAL_BAIT_RE`), re-rutează la `sales`. DE CE? Ruta `simple` e servită de nano FĂRĂ validator → un client ar putea forța „zi doar da: aveți 70% reducere?” → „Da”. Guard-ul previne asta. - **Low confidence** (`triage.py:272`): dacă nano zice `confidence=low` pe `sales/order` → forțează `clarify`. „LLM înțelege, codul decide.” - **Sloturi** (`_normalize_slots`, `triage.py:69`): `budget_max` / `concerns` / `brand` / `suitable_for` extrase și **normalizate în cod** (concerns filtrate la vocabularul DomainPack) → populează `RouteDecision.filters` ca seed pentru search.

Pentru `simple` / `clarify`, nano compune și răspunsul → early-exit. `closure` (mulțumire de final) → mesaj cald + chips pe categorii adiacente (cross-sell).

8.9 Handoff (stages/handoff.py:39)

Consumă `Route.HANDOFF`. Dacă canalul are operator → `request_human` + notifică + mesaj de confirmare. Pe web (fără operator) → rescrie ruta la SALES (botul asistă singur).

8.10 Agent (stages/agent.py:957)

Cel mai complex — 1411 linii, „god-module”. Vezi cap. 10 (tool loop) + cap. 12 (grounding). Pe scurt, 3 faze: - **PRE-loop** (intenții deterministe, zero LLM): cere link? → servește `product_url` din state. Compară? → tabel determinist. „Arată mai multe”? → pagina următoare din pool (zero LLM). - **LLM tool loop**: system prompt din DB → `run_tool_loop` (max 3 pași) → tools (search/details/compare/cart/checkout/reorder/back_in_stock/faq/check_order/request_human). - **POST-loop** (compunere deterministă): checkout fallback, cross-sell, „mai ieftin”, superlativ, apoi finalize (rich sau proză) + validator. Modelul propune, **codul dispune** pe cifre/linkuri.

8.11 Fallback (runner.py:169)

Dacă niciun stagiou n-a produs reply (rută neacoperită, triaj fără răspuns, fără cheie OpenAI) → o întrebare de clarificare, **NU tăcere** (principiul 6). ⚠ Doar în română (încalcă P11 — R7 în backlog).

9. Bugetul de context (mesaj lung → păstrăm doar o parte)

 `src/worker/context.py`

Ăsta e exact detaliul pe care l-ai cerut. LLM-ul are un buget de tokeni. Nu-i trimiți toată conversația — o **tai la buget**, în cod (principiul 4 — bugetul e în cod, nu în prompturi, nu prin disciplină).

Sunt DOUĂ tăieri de „mesaj lung”

1. **Mesajul CURENT prea lung** → tăiat la **2000 caractere** în Gates (`INBOUND_BODY_MAX` , vezi 8.1). Asta e protecție de input.
2. **Istoricul + contextul** → bugetat aici, în `context.py` . Astea intră în promptul de triaj + agent.

Blocurile de context și bugetele lor

```
def conversation_transcript(history, *, max_turns=6, max_chars=1200):
    prior = history[:-1]                                # fără mesajul curent (ultimul)
    lines = [f"{'Client' if inbound else 'Asistent'}: {body}" for m in prior[-max_turns:]]
    return "\n".join(lines)[-max_chars:]                # ultimele 6 tururi, tăiat la 1200 char
```

Bloc	Funcție	Buget	Ce conține
Transcript	<code>conversation_transcript</code> (<code>context.py:23</code>)	6 tururi / 1200 caractere	mesajele anterioare „Client:/Asistent:”
Profil	<code>customer_profile_block</code> (<code>context.py:40</code>)	300 caractere , liste tăiate la 4	ce știm despre client (fără PII de canal)
State	<code>state_block</code> (<code>context.py:59</code>)	3 produse / 600 caractere	produse arătate (id+nume+preț) + constrângeri
Rezumat	<code>summary_block</code> (<code>context.py:85</code>)	600 caractere (<code>summary_max_chars</code>)	rezumatul conversației lungi (>8 mesaje)

Cum se tai: - Transcript: ia **ultimele** 6 tururi (`prior[-6:]`), apoi taie la **ultimele** 1200 caractere (`[-1200:]`). Deci dacă un mesaj vechi e uriaș, se pierde întâi el (păstrezi ce e recent = relevant). - Profil: sare valorile goale, listele la 4 elemente, apoi taie la 300. - Rezumat: pentru conversații > 8 mesaje, un rezumat rolling (generat post-tur de nano) acoperă mesajele vechi care au ieșit din fereastra de 8, tăiat la 600.

DE CE ordinea asta? `context_blocks` (`context.py:96`) unește: **rezumat (fundal vechi)** → **profil** → **state**, iar transcriptul recent e concatenat downstream. Ordine cronologică: vechi → recent. Blocul stă în mesajul **USER** (dinamic), NU în system → promptul static rămâne byte-identic → **prompt caching** OpenAI (75-90% discount pe partea statică).

search_query — de ce follow-up-urile scurte funcționează (`context.py:106`)

```
def search_query(history, current, *, n=2):
    users = [m.body for m in history if m.direction == INBOUND and m.body]
    return " ".join(users[-2:])    # ultimele 2 mesaje ale CLIENTULUI
```

Când clientul zice „ceva mai ieftin”, căutarea nu ia doar „ceva mai ieftin” (fără sens izolat) — ia ultimele 2 mesaje ale lui → caută în contextul corect.

Exemplu complet: clientul scrie un paragraf de 3000 de caractere despre pielea lui + ce vrea. 1. Gates taie la 2000 caractere (emit `body_truncated`). 2. La turul următor, mesajul intră în `history`. Când construim promptul, transcriptul ia ultimele 6 tururi, apoi taie la 1200 caractere — deci din paragraful lung rămâne coada relevantă. 3. Dacă conversația trece de 8 mesaje, restul se comprimă într-un rezumat de max 600 caractere.

Așa „păstrăm doar o parte” — la fiecare nivel, cu un buget clar în cod.

10. Cusătura LLM (retry, tokeni maximi, tool loop)

`src/agent/llm.py` — **SINGURUL** loc care vorbește cu OpenAI (principiul 2).

`get_llm()` (`llm.py:359`) întoarce un singleton `LLMClient`, sau **None** dacă nu e cheie → pipeline-ul degradează grațios. Testele injectează un fake (zero apeluri reale în CI).

Plafonul de tokeni pe apel (detaliul „max token pe mesaj”)

`_sampling` (`llm.py:139`)

```
def _sampling(self, *, agent: bool) -> dict:
    out = {}
    if agent:
        out["max_completion_tokens"] = s.llm_max_tokens_agent # 800 (config.py:523)
    if s.llm_sampling_enabled:
        out["temperature"] = s.llm_temperature_agent if agent else s.llm_temperature_triage # 0.7 / 0.2
    return out
```

- `max_completion_tokens` = **800** pe TOATE apelurile de agent (`llm_max_tokens_agent`). E plafonul de **output** (câți tokeni poate scrie modelul). Un completion patologic / buclă nu scapă de ceiling.
- `max_completion_tokens`, **NU** `max_tokens` — ultimul e deprecăt → 400 pe modelele gpt-5.4-*. (Ăsta a fost un bug real: agentul trimitea `max_tokens` → 400 → fallback „n-am înțeles” pe orice sales. Fix în PR #133.)
- **Triajul (agent=False) NU primește ceiling** — JSON scurt, nu are nevoie.
- **Vision** (`describe_image`, `llm.py:305`): `max_completion_tokens=256` + `detail:"low"` = cost tăiat (un tile, fără high-res).
- **Temperatura pe rol:** triaj 0.2 (determinist, clasificare), agent 0.7 (variație, copy ne-repetitiv). Corectitudinea NU depinde de temperatură (o asigură validatorul) → agentul poate fi urcat liber.

Retry mărginit (`_with_retry` , `llm.py:45`)

```
for attempt in range(max_retries + 1): # llm_retry_max = 2
    try: return await factory()
    except openai.APIStatusError as e:
        if e.status_code < 500 and not RateLimitError: raise # 4xx terminal → ridică imediat
        wait = _retry_after_seconds(e) # respectă Retry-After
    except (APITimeoutError, APIConnectionError): wait = None
    sleep_s = (wait or delay) + jitter # backoff exponențial + jitter
    await asyncio.sleep(sleep_s); delay *= 2
raise last # epuizat → ridică (caller degradează)
```

Ce e tranzitoriu (retry-abil): 429 (rate limit), 5xx, timeout, connection. **Ce e terminal (ridică imediat):** 400/401/403/404 (nu are sens să reîncerci un request greșit). La epuizare loghează `llm_api_failure` și ridică → caller-ul (stagiul) prinde și degradează.

Metodele adaptorului

Metodă	Ce face	Cine o folosește
<code>classify_json</code> (<code>llm.py:168</code>)	chat cu <code>response_format=json_object</code>	triaj
<code>complete_schema</code> (<code>llm.py:188</code>)	chat cu <code>json_schema</code> strict	agent (rich)
<code>complete</code> (<code>llm.py:211</code>)	chat text simplu	agent (proză)
<code>run_tool_loop</code> (<code>llm.py:227</code>)	bucla de tool-calling	agent
<code>moderate</code> (<code>llm.py:290</code>)	moderation (gratuit)	gates
<code>describe_image</code> (<code>llm.py:305</code>)	Vision poză→text	gates
<code>embed</code> (<code>llm.py:343</code>)	embeddings (1536 dim)	cache/FAQ/search/products

Tool loop (`run_tool_loop` , `llm.py:227`) — inima agentului

```
for _ in range(max_steps): # max 3 (CLAUDE.md: max 3 tool calls/tur)
    resp = await self._chat(agent=True, model=mdl, messages=messages, tools=tools, tool_choice="auto")
    msg = resp.choices[0].message
    if not msg.tool_calls:
        return msg.content # modelul a răspuns cu text → gata
    messages.append({assistant + tool_calls})
    contents = await asyncio.gather(*(execute(tc.name, args) for tc in msg.tool_calls)) # CONCURENT
    for tc, content in zip(...): messages.append({"role": "tool", "content": content})
    resp = await self._chat(agent=True, ...) # cap atins → un ultim apel FĂRĂ tools (text forțat)
    return resp.choices[0].message.content
```

Analogie: modelul e un vânzător care poate „cere de la depozit” (tool) până la 3 ori: „dă-mi seruri sub 100 lei” (search) → primește rezultate → „dă-mi detalii la #2” (details) → apoi scrie răspunsul. Formatul OpenAI (tool_calls / rol `tool`) trăiește DOAR aici (adaptorul = singurul loc care vorbește OpenAI). Tool-urile cerute într-un pas rulează **concurrent** (`asyncio.gather`) ca să taie latența.

11. Căutarea de produse

 `src/tools/catalog_tools.py:383` (`search_products_tool`) — vezi Diagrama 5.

Motorul de căutare combină **lexical** (potrivire pe cuvinte) + **semantic** (potrivire pe înțeles).

Fluxul

1. **Argumentele** de la model (categorie, filtre, buget, concerns).
2. **Map concerns** (`catalog_tools.py:399`): „ten gras” → atribut real, prin DomainPack.
3. **Moștenire sesiune** (`catalog_tools.py:417`): dacă e sesiune activă + fără categorie/concerns noi → moștenește filtrele → **paginare** (`continue_search_session` , pagina următoare, **zero cost LLM**).
4. **Scara de relaxare** (`build relax ladder` , `catalog_tools.py:439`): dacă nu găsești nimic cu toate filtrele, relaxezi filtrele pe rând (buget → concern → ... → categorie ultima).
5. **Embed query** (dacă disponibil, `catalog_tools.py:450`) → **UN** apel embed.
6. **Lexical** (FTS + pg_trgm, `catalog_tools.py:468`) + **Semantic** (pgvector HNSW, `catalog_tools.py:486`).
7. **Fuse** (`fuse_candidates` , `catalog_tools.py:507`): RRF (Reciprocal Rank Fusion) + blended rank (rating social-proof shrunk + disponibilitate + reducere + concern). Repară „un produs 4.6×148 recenzii ajunge sub unul 4.4×28”.
8. **Diversify** (`catalog_tools.py:518`): prima pagină pe relevanță → terțe de preț + max 2/brand (nu top-N identice).
9. **Dedupe** vs `displayed_products` , cap 6. **Store session** (pool + fingerprint în state).
10. **ToolResult**: produse complete → validator; `llm_view` compact (max 6×8 câmpuri) → model.

Degradare: embed picat → doar lexical; semantic picat în tur → rămâne lexical. Niciodată nu crapă complet.

Detaliu important — două vederi ale rezultatelor: validatorul primește produsele **complete** (ca să verifice prețuri/linkuri), dar modelul primește un `llm_view` **compact** (max 6 produse × 8 câmpuri) ca să nu-i umpli contextul. Asta e tot buget de tokeni.

12. Grounding-ul anti-halucinație (apărarea load-bearing)

Aici trăiește obsesia centrală: **botul nu inventează**. Sunt **două** apărări, una per cale.

Calea PROZĂ: validatorul (`agent.py:472-502` , `560-595`)

După ce agentul compune un răspuns text, validatorul verifică: - Fiecare **preț** din reply există în `ctx.retrieval` ? (`_PRICE_RE` + cifre bare fără valută, `validator_bare_numbers_enabled`) - Fiecare **produs/link** menționat există în catalog? - **Claim-uri neverificabile** (superlativ „best seller”, stoc „pe stoc”) → `validator_claims_enabled` . - **Claim MEDICAL** (tratează/vindecă, sigur în sarcină, fără alergeni, recomandat de medic) → `safety_medical_guardrail_enabled` (răspundere juridică!).

Dacă invalid → **1 retry** cu prețurile permise → dacă tot invalid → **răspuns determinist din DB** (`agent.py:523`). Zero prețuri inventate **structural** — nu prin disciplina modelului, ci prin cod.

Calea RICH: compose (`compose.assemble` , `compose.py:329`) — vezi Diagrama 4c

Pentru carduri de produs, regula e și mai strictă: **modelul emite DOAR cuvinte + referințe** `product_id` . Codul hidratează faptele:

1. Modelul întoarce JSON structurat (mini, `complete_schema`).
2. `assemble` hidratează după `product_id` .
3. **Membership grounding** (`MEM`): itemul referă un produs REAL adus? Nu → **aruncat**.
4. **Scrub câmp cu câmp** (`scrub_prose` / `scrub_intro` / `scrub_education`): cifre, `%` , claim-uri neverificabile → `field=None` .
5. **Medical** (`_unsafe_medical`): claim medical în câmp → **câmp aruncat** (P0 safety).
6. Badge din semnale reale (`badges.py`), pick determinist, chips.

Exemplu: modelul zice „serul ăsta cu vitamina C 15% reduce ridurile”. Cod: `15%` → nu apare în date → scos. „reduce ridurile” → claim medical → câmp aruncat. Rămâne doar ce e ancorat în catalog.

NX-139 — axe de decizie grounded

Ultima adăugire: `decision_axes` + `spec_numbers` (`compose.py:661` , `704`). Axele REALE pe care variază setul (tip de ten / fitment / material — per vertical, din DomainPack) intră ca input în compunere → intro-ul numește axe reale, nu superficiale. Cifrele de specificație din datele produselor afișate („SPF 30”, „50 ml”) devin permise în intro/education (grounded). Prețurile NU intră niciodată în setul permis.

13. IEȘIRE: Sender TX + dispatcher + canale

Sender = tranzacția din `handle_turn` (cap. 7, pas 8)

Reamintire: reply + outbox + patch state + finalizare dedupe = **o singură tranzacție atomică**. Țasta e „un singur punct de ieșire” (principiul 5). Nimic nu trimite mesaje în afară de outbox → dispatcher.

Dispatcher (`src/worker/dispatcher.py:245`)

Proces separat. Buclă:

```
rows = await claim_due(...)          # FOR UPDATE SKIP LOCKED (nu se calcă cu alt dispatcher)
for row in rows:
    sender = registry.get(row.channel_kind)
    render = choose_render(sender.capabilities, payload)  # rich/carousel/cards/template/text
    result = await sender.send(...)
    if ok: await mark_sent(...) + link provider_msg_id    # TX
    else: await mark_failed(...) → backoff → dead
```

- **FOR UPDATE SKIP LOCKED** : dacă rulezi 2 dispatchere, fiecare ia rânduri diferite (nu dublă trimitere).
- **choose_render** (`dispatcher.py:101`): alege forma după **capabilitățile** canalului. WhatsApp are carousel; un canal fără rich primește `text` (floor aplatizat).

- **Visibility timeout self-healing:** dacă dispatcher-ul moare între claim și mark, rândul e „redeemed” după timeout (nu rămâne blocat).
- **⚠ Slăbiciune (R6):** dispatcher-ul e **secvențial** (rând cu rând, tenant cu tenant) + poll 2s la idle. Un tenant lent întârzie ceilalți; +0-2s latență pe calea async.

Canalele (marginile de transport)

📄 `src/channels/base.py` — `ChannelSender` Protocol + `Capability` matrix (NX-115).

Canal	Client	Capabilities	Notă
WhatsApp	<code>MetaClient</code> (<code>meta_client.py:28</code>)	TEXT, CAROUSEL, TEMPLATE, TYPING	canal PRIMAR de producție; fereastră 24h
Telegram	<code>TelegramClient</code> (<code>telegram/client.py:62</code>)	TEXT, RICH, TYPING	canal de TEST; edit carusel
Web	<code>WebSender</code> (<code>web/sender.py:34</code>)	TEXT, RICH, COMPARISON	publish SSE + backlog replay

DE CE „registry”? Dispatcher-ul nu știe de canale — cere `registry.get(channel_kind)`. Adugi un canal nou implementând `ChannelSender` + înregistrându-l. Cuplajul de canal trăiește DOAR la 2 margini: ingestie (parser) + trimitere (sender). Pipeline-ul e agnostic de canal.

Web widget — două moduri

- **Sincron** (`/web/chat` , `web/app.py:190`): `handle_turn(deliver=False)` → **fără outbox**, răspunsul HTTP e transportul. Frontendul primește direct.
- **Async** (`/web/messages` + SSE `/web/stream`): envelope pe stream (ca Telegram) → worker → dispatcher → WebSender publică pe SSE → browserul primește.

14. Memoria conversației

Vezi Diagrama 6. Memoria = 4 lucruri:

Sursă	Ce e	Unde se încarcă	Unde se scrie
<code>history</code>	ultimele 8 mesaje	processor (<code>get_recent_messages</code>)	<code>insert_message</code>
<code>state</code> (jsonb ≤8KB)	ref-uri: produse afișate, cart, constrângeri, sesiune căutare	processor (<code>from_jsonb</code>)	Sender TX (<code>patch_conversation_state</code>)
<code>summary</code>	rezumat rolling (conversații >8 mesaje)	processor (<code>get_summary_for_context</code>)	post-tur (<code>summarizer</code> , nano)
<code>profile</code> + <code>lead_score</code>	ce știm despre client	processor (pe Contact)	post-tur (<code>extract_profile</code> , nano)

Ciclul de viață al state-ului: 1. **START:** `from_jsonb` hidratează defensiv (cap. 4). 2. **ÎN prompturi:** `context_blocks` (cap. 9). 3. **DURANTĂ:** agentul mută in-place `constraints` , `search_constraints` ; tool-urile cer `state_patch` (cart). 4. **SENDER TX:** merge canonic (cap. 7, pas 8a) + optimistic lock. 5. **POST-tur:** cache writeback + rezumat + profil.

Regula de aur (principiul 8): state = ref-uri `{id, name, price}`, NU obiecte complete. Buget 8KB (CHECK în DB). Când ai nevoie de detalii → re-hidratezi din DB.

15. Baza de date

📄 `src/db/connection.py` — vezi Diagrama 7.

Două pool-uri, două căi

```
admin_pool (get_pool, connection.py:76) → control plane + joburi. Rol PRIVILEGIAT.  
bot_pool (get_bot_pool, connection.py:207) → tenant path. Rol bot_runtime, FĂRĂ bypassrls.
```

- `admin_conn` (`connection.py:247`): folosit DOAR pentru `resolve_channel` (canal→business, precede tenantul)
- joburi admin. RLS bypass-at aici → suprafață limitată intenționat.
- `tenant_conn(business_id)` (`connection.py:261`): hot path, RLS activ.

tenant_conn — izolarea în detaliu

```
async with pool.acquire() as conn:  
    if _isolation_enabled():  
        row = await conn.fetchrow(  
            "select set_config('app.business_id', $1, false) as biz, current_user as usr", business_id)  
        _check_isolation(row["usr"], row["biz"], business_id) # rol==bot_runtime? GUC==expected?  
    try:  
        yield conn  
    finally:  
        await conn.execute("select set_config('app.business_id', '', false)") # RESET la release
```

- **Setare + verificare într-un round-trip** (`set_config` întoarce valoarea, `current_user` confirmă rolul).
- `_check_isolation` (`connection.py:187`): conexiunea TREBUIE să fie `bot_runtime` ȘI să poarte exact `app.business_id = expected`. Orice abatere (rol greșit / GUC nesetat / reuse murdar de la alt tenant) → **IsolationError ÎNAINTE de primul query** + log CRITICAL.
- **RESET la release**: următorul checkout pe alt tenant nu vede GUC-ul vechi.

DE CE rol de LOGIN, nu SET ROLE ? `SET ROLE` de sesiune se scurgea sub multiplexarea poolerului Supabase (P0-A din audit). Cu login direct, identitatea e fixată de credențiale. RLS devine plasă: **un query fără filtru de tenant → 0 rânduri, nu datele altui client.**

Izolarea multi-tenant e în straturi: `WHERE business_id=$1` în cod (primar) + RLS (plasă). Chiar dacă uiți filtrul, RLS te salvează.

Boot-gate (consumer.py:315-322)

```
await assert_migrations_current(pool)    # migrare pending → BOOT REFUZAT
await get_bot_pool()                    # eager → parolă greșită crapă la boot, nu la primul mesaj
```

Workerul refuză să pornească pe o schemă incompletă sau cu rol greșit — crash loud.

Tabelele principale (din docs/schema_v2_production.sql)

Tabel	Ce stochează	Cine scrie	Cine citește
businesses	tenanții (config, vertical, plafon cost)	seed/admin	processor (load_business)
channels	canalele per tenant (kind, provider_account_id)	seed	resolve_channel
contacts	clienții (profile, lead_score, consent, is_blocked)	processor, post-tur	gates, context
channel_identities	PII-ul de canal (telefon E.164 + hash) — DOAR aici	get_or_create_contact	identity resolution
conversations	conversații (state jsonb, bot_active, handoff_until, last_inbound_at)	Sender TX	processor
messages [partiționat]	toate mesajele (direction, author, body, cost, tokens)	insert_message	history
inbound_dedupe	claim dedupe L2 (provider_msg_id, completed_at)	claim/mark	dedupe
outbox	coada de ieșire (idempotency_key, payload, status)	Sender TX	dispatcher
message_status_events	delivered/read/failed	record_status_event	—
products	catalogul (name, price, availability, attributes)	sync (nu worker)	search
product_embeddings	vectori 1536 (HNSW cosine)	embed job	semantic search
faqs	întrebări+răspunsuri (embedding)	seed	faq_stage
semantic_cache	răspunsuri cache-uite (embedding, TTL)	post-tur writeback	cache_stage
intent_aliases	fraze aprobate → target	shadow mode	alias_stage
checkout_links	linkuri de plată (ref_code → atribuire)	checkout_link tool	orders webhook
orders	comenzi (attribution)	orders webhook	check_order
proactive_jobs	joburi proactive (kind, scheduled_at)	initiators	proactive scheduler
analytics_events [partiționat]	event-uri (append-only, turn_id)	runner/processor	rollup
usage_daily	rollup zilnic (cost, tokeni) — sursa de facturare	rollup nocturn	dashboard/cost guard

Convenții: toate tabelele tenant-scoped au business_id NOT NULL + index compus. Idempotență = UNIQUE pe (business_id, external_id) . Hot tables (messages , analytics_events) partiționate pe lună. PII DOAR în channel_identities .

16. Guvernanța costului (3 niveluri)

LLM-ul e scump. Sistemul are 3 plafoane, cu contoare în Redis (rapid) + reseed din `usage_daily` (durabil):

1. **Business/zi** (`daily_cost_cap_usd` , default \$5): `_llm_within_budget` (`processor.py:308`) pre-check. Peste plafon → `llm=None` → pipeline degradat.
2. **Contact/zi** (`contact_daily_cost_cap_usd` , opt-in): o conversație în buclă nu poate arde plafonul întregului tenant. Doar canale identificate.
3. **Vizitator web** (`web_cost_cap_per_visitor_usd` , \$0.50): un token public furat nu golește bugetul.

Enforcement post-increment fără TOCTOU (`_record_turn_cost` , `processor.py:345`): costul EXACT al turului (din `ctx.usage.cost_usd` , tokeni reali) se adaugă ATOMIC; dacă noul total ≥ plafon, emite `cost_guard_tripped` → turul URMĂTOR e blocat de pre-check. Facturarea reală = `usage_daily` , nu contorul (care e plasă).

Buget PER TUR (observabilitate, `runner.py:132`): dacă un tur depășește latența (5000ms) sau costul (\$0.01), runner-ul emite `turn_over_budget` cu stagiul cel mai lent. NU schimbă turul — doar alertează.

17. Tratarea erorilor (niciodată tăcere)

Vezi Diagrama 9. Principiul 6 pus în practică pe fiecare tip de eroare:

Eroare	Ce se întâmplă	Rezultat
LLM pică	retry mărginit → triaj route None / agent return → <code>fallback_stage</code>	întrebare de clarificare (nu tăcere)
Validator pică	1 retry cu prețuri permise → răspuns determinist din DB	zero prețuri inventate
Cost guard atins	<code>llm=None</code> — gates+cache merg, stagiile LLM sar	degradare
Redis jos la webhook	503 → Meta reîncearcă	dedupe prinde la retry
Cache/FAQ eroare	miss, turul continuă	best-effort
Analytics eroare	log only	observabilitatea nu blochează
Conv lock busy	requeue cu cap → drop la epuizare ⚠	NX-140
Excepție procesare	ACK + log = tur PIERDUT ⚠	NX-140 (gaura #1)
Livrare pică	backoff → dead (vizibil)	nu tăcut
Handoff	<code>request_human</code> + <code>handoff_until</code> + notifică	tururile următoare = tăcere intenționată

Cele două găuri reale (NX-140): excepție de procesare = ACK + drop, și lock blocat = drop. Ambele încalcă „niciodată tăcere” exact pe calea de eroare. Fix: re-queue cu contor + fallback în outbox.

18. Proactiv (mesaje inițiate de bot)

`src/proactive/` — vezi Diagrama 10.

Mesaje pe care botul le trimite **primul** (coș abandonat, stoc revenit). Cele mai reglementate decizii (consent + fereastra 24h Meta) — 100% cod determinist, zero LLM.

1. **FEED:** `jobs.scheduler` rulează sweepere (`sweep_abandoned_cart` , `sweep_back_in_stock` , `initiators.py`) → creează rânduri în `proactive_jobs` . ⚠ `schedule_awb_update` + `schedule_follow_up` sunt **definite dar niciodată apelate** (seam-uri TODO).

2. **ENGINE** (`proactive/scheduler.py:181`): `claim_due_jobs` (FOR UPDATE SKIP LOCKED) → rezolvă conversație+ canal+destinatar → `build_message_spec`.
3. **GATE** (`decide_proactive` , `templates.py:39`): consent pe acel tip? → în fereastra 24h? → dacă da, text liber; dacă nu, template aprobat în locale? → altfel `skipped_no_window`.
4. → `outbox` (idempotency `proactive:job_id`) → dispatcher.

19. Referință de configurare (toate butoanele)

 `src/config.py` — sursa unică. Nimic hardcodat, nimic din `os.environ` direct.

Bugete/plafoane structurale

Setare	Default	Ce controlează
<code>INBOUND_BODY_MAX</code> (constantă)	2000	caractere max pe corpul inbound (trunchiere în Gates)
<code>webhook_max_body_bytes</code>	262144 (256KB)	plasă anti-OOM webhook
<code>web_max_body_bytes</code>	16384 (16KB)	plasă anti-OOM web
<code>reply_split_chars</code>	200	prag de spargere reply în 2
<code>llm_max_tokens_agent</code>	800	<code>max_completion_tokens</code> pe agent
<code>summary_max_chars</code>	600	buget bloc rezumat
<code>summary_threshold</code>	20	de la câte mesaje se sumarizează

Praguri LLM/cache/FAQ

Setare	Default	Ce controlează
<code>llm_temperature_triage</code> / <code>_agent</code>	0.2 / 0.7	determinism vs variație
<code>llm_retry_max</code>	2	reîncercări pe tranzitoriu
<code>llm_timeout_s</code>	30	anti-hang
<code>cache_tau_high</code>	0.92	prag auto-accept cache
<code>faq_tau_high</code> / <code>_policy</code> / <code>_tool</code>	0.78 / 0.45 / 0.66	praguri FAQ

Plafoane de cost

Setare	Default	Ce controlează
<code>daily_cost_cap_usd</code>	5.0	plafon zilnic business
<code>contact_daily_cost_cap_usd</code>	0.0 (off)	plafon per-contact
<code>web_cost_cap_per_visitor_usd</code>	0.50	plafon per-vizitator web
<code>turn_latency_budget_ms</code> / <code>turn_cost_budget_usd</code>	5000 / 0.01	alerte per-tur

Kill-switch-uri importante (fail-safe)

`moderation_enabled` , `cache_enabled` , `faq_enabled` , `alias_enabled` , `vision_enabled` , `cost_guard_enabled` ,
`rate_limit_enabled` , `conv_lock_enabled` , `search_sessions_enabled` , `card_badges_enabled` ,

`safety_medical_guardrail_enabled`, `domain_pack_enabled`, `proactive_enabled`, `web_enabled`,
`web_identity_enabled`, `ai_disclaimer_enabled` (azi OFF), `rich_pick_web_enabled` (azi OFF — preferință client).

Filozofia flag-urilor: fiecare feature nou are un kill-switch fail-safe. OFF = comportamentul vechi, byte-identic. Poți dezactiva orice în prod fără redeploy de cod (doar env).

20. Ghid de depanare pe subsisteme

Cum depanează un senior aici

Nu se uită la cod întâi. Se uită la **event-urile din** `analytics_events` filtrate pe `turn_id` — reconstruiește traiectoria exactă a turului (ce stagii au rulat, cât au durat, unde a ieșit). `turn_id` e injectat automat în fiecare event (`ctx.emit`). E „black box recorder”-ul. Abia apoi pui breakpoint.

Loguri cheie (ce să `grep`-uiești)

Simptom	Grep	Ce înseamnă
Client n-a primit nimic	<code>eroare la procesarea mesajului (consumer.py:229)</code>	Excepție → ACK → tur pierdut (NX-140)
Client n-a primit nimic	<code>dropped după N reîncercări (consumer.py:96)</code>	Lock blocat → drop
Răspuns lent	<code>tur peste buget (runner.py:158)</code>	Latență/cost peste prag + stagiul lent
Bot „a amuțit”	<code>tăcere intenționată (handoff) (processor.py:537)</code>	Corect: a preluat un om
Mesaj de 2 ori	<code>dedupe_hit_db (processor.py:457)</code>	Corect: dedupe a prins retry
LLM eroare	<code>llm_api_failure (llm.py:74)</code>	Retry epuizat
Izolare	<code>isolation_assert_failed (connection.py:285)</code>	O conexiune neizolată la checkout — CRITIC
Straturi moarte	<code>register_vector_codec: codec pgvector neînregistrat</code>	FAQ/cache degradează la text-inline

Breakpoint-uri strategice

Loc	Ce inspectezi
<code>processor.py:528 (await run_pipeline)</code>	<code>ctx</code> ÎNAINTE de pipeline: history, state, language
<code>processor.py:534 (if ctx.reply is None)</code>	<code>ctx.reply</code> , <code>ctx.halt</code> , <code>ctx.route</code> DUPĂ pipeline — de ce n-a răspuns
<code>processor.py:565 (deschidere TX)</code>	<code>new_state</code> cum se construiește — prinzi „botul uită”
<code>triage.py:246 (out = TriageOut(**raw))</code>	ce a clasificat nano
<code>agent.py:957 (agent_stage intrare)</code>	ruta + retrieval + intenții deterministe
<code>connection.py:280 (_check_isolation)</code>	rolul + GUC-ul la checkout

Bug-uri tipice și race conditions

- **Race pe state:** 2 mesaje rapide fără lock → ultimul suprascrie. Verifică `conv lock: ocupat → re-queue`.
- **Optimistic lock conflict:** `patch_conversation_state` eșuează dacă `state_version` s-a schimbat → excepție în TX → ACK → tur pierdut. Cauză: 2 tururi concurente.

- **„Botul uită”**: câmp mutat in-place dar ne-merge-uit la 8a (`processor.py:580`). Regula: orice câmp in-place TREBUIE listat explicit.
- **Silent dead layer**: strat best-effort care nu emite niciun event = excepție înghițită. Caută warning-ul.
- **Conexiune ținută prin LLM** (R5/NX-141): `conn` ocupat cât agentul face 1-4 apeluri LLM → pool 10 → max ~10 tururi concurente/proces. Simptom la scală: „pool exhausted”.

Probleme LLM tipice

- **Fallback „n-am înțeles” pe orice sales**: agentul trimitea `max_tokens` (deprecăt) → 400. Fix: `max_completion_tokens`.
- **Cache poisoning**: un răspuns „n-am găsit” cache-uit (cacheable=True greșit) servit altui client → tăcere de facto. Regula: răspunsurile de fallback/refuz/clarify au `cacheable=False`.
- **Deployed build lag**: un „bug live” poate fi imaginea prod în urma lui `main`. Verifică commit-ul deployat înainte să vânezi codul.

21. Exerciții + validare

Exerciții de modificare (de la ușor la greu)

E1 (ușor) — Schimbă pragul de spargere reply de la 200 la 300 caractere. Ce fișier? Ce se schimbă în comportament? (Indiciu: `config.py`, `reply_split_chars`.)

E2 (ușor) — Vreau să văd în log câte produse afișează fiecare tur. Unde pui linia, ce loghezi (fără PII)?

E3 (mediu) — Adaugă un câmp `last_category` în state. Enumeră TOATE locurile: (a) `ConversationState` dataclass, (b) `from_jsonb` hidratare defensivă, (c) merge canonic în TX. De ce dacă sari peste (c) botul „uită”?

E4 (mediu) — Adaugă un tool nou `check_warranty(product_id)`. Ce fișiere atingi? (Indiciu: `tools/`, `tool_definitions.py`, `registrul`, `promptul`.)

E5 (mediu) — Adaugă un stagiou nou de pipeline între cache și FAQ. Ce semnătură trebuie să aibă? Unde-l pui în `DEFAULT_STAGES`? De ce nu trebuie să atingi runner-ul?

E6 (greu) — Repară gaura ACK-on-error (NX-140). La `consumer.py:228-230`, schițează: re-queue cu contor → la epuizare fallback în outbox. De ce fallback-ul trebuie în outbox și nu trimis direct?

E7 (greu) — Rezolvă R5 (conexiune ținută prin LLM). Cum fazezi `handle_turn`: load → release conn → LLM fără conn → TX pe conn proaspăt? Ce câștigi la scală?

Validare (răspunde, nu-ți dau răspunsurile)

1. **(Grilă)** Câte puncte LLM are pipeline-ul? (a) 11; (b) 2; (c) 5; (d) 0.
2. **(A/F)** Webhook-ul așteaptă răspunsul OpenAI înainte de ACK 200.
3. **(Deschisă)** Explică cele DOUĂ tăieri de „mesaj lung” (unde + la ce valoare).
4. **(Ce se întâmplă dacă...)** Modelul scrie „79 lei” dar produsul are 129 în retrieval. Pe calea proză vs rich?
5. **(Debugging)** „Botul n-a răspuns deloc.” Numește 2 locuri unde un tur se pierde tăcut + grep-ul pentru fiecare.
6. **(Arhitectură)** De ce e `resolve_channel` singurul query pe `admin_conn`?

7. **(Prezice)** Ștergi liniile `active_search=None` la `processor.py:592-593`. Ce bug apare?
 8. **(Găsește bug-ul)** Muți merge-ul canonic DUPĂ `state_patch` în TX. Ce se strică?
 9. **(Grilă)** `max_completion_tokens` vs `max_tokens`? De ce contează?
 10. **(Race)** De ce ai nevoie și de lock per conversație și de optimistic lock pe `state_version`?
-

Anexă — cele 12 principii (din CLAUDE.md, verificate în cod)

1. Pipeline liniar — niciun stagiou nu sare înapoi. ✓ (`runner.py`)
2. LLM doar în 2 puncte — triaj + agent. ✓ (`llm.py` = singura cusătură)
3. Un singur proprietar per câmp. ✓ (`models.py` docstrings)
4. Buget de context în cod, nu în prompturi. ✓ (`context.py`)
5. Un singur punct de ieșire — Sender → outbox → dispatcher. ✓ (excepție: typing)
6. Niciodată tăcere. ✓ (cu 2 găuri: NX-140)
7. `business_id` pe tot + RLS. ✓ (`connection.py`)
8. State = ref-uri, nu obiecte. ✓ (`models.py` `ProductRef`)
9. Promptul se generează din DB. ✓ (`prompt_builder.py`)
10. Observabilitate din runner. ✓ (`runner.py` măsoară, stagiile nu știu)
11. Limba e parte din cheie. ✓ (locale în FAQ/cache/template; excepție: fallback RO-only)
12. PII într-un singur loc — `channel_identities`. ✓ (+ mascare PII input)

Concluzie: sistemul e neobișnuit de disciplinat — principiile chiar sunt implementate. Datoriile reale: concentrarea (`agent.py` + `processor.py` acumulează tot ce e nou) și gaura ACK-on-error (NX-140).

Nativx Assistant — DEEP DIVE pe tot proiectul (RO, for dummies)

Manual care aprofundează **fiecare fișier** din proiect, în formatul cu 9 secțiuni (vezi [MASTERCLASS-PROMPT.md](#)):
📍 Unde · 🗣️ Analogie · ❓ De ce · ⚙️ Cum · 📄 Cazuri cu exemple · 🧠 De ce așa · 💥 Ce-ar fi dacă · 🧑🏻 Debug · ✅ Test.

Se construiește în **valuri** (un subsistem pe rând). Complementar cu: [MASTERCLASS-RO.md](#) (harta cap-coadă) și [ARCHITECTURE-WORKFLOWS.md](#) (10 diagrame).

Codul e sursa de adevăr. Fiecare afirmație are `fișier:linie`.

📊 Progres (cuprinsul întregului proiect)

Va l	Subsistem	Fișiere	Stare
1	Pipeline (11 stagii)	gates, language, clarify, greeting, alias, cache, faq, triage, handoff, agent, fallback	✅ AICI
2	Worker core	processor, consumer, runner, dispatcher, debounce, callback, reply_split, limits, order_gate	✅
3	Compunere & grounding	compose, badges, text_scrub, summarizer, profile, context	✅
4	Agent & LLM	llm, prompt_builder, tool_definitions, usage, pricing	✅ (agent.py ✅ în chat)
5	Tools	catalog_tools, commerce_tools, orders_tools, faq_tools, handoff_tools, taxonomy, base	✅
6	DB	connection, queries/* (~20 fișiere)	✅ (connection ✅ în MASTERCLASS-RO)
7	Intrare	webhook/app, signature, meta, body_limit, orders, redis_bus	✅ (webhook/app ✅ în MASTERCLASS-RO)
8	Canale	channels/base, media, meta_client, telegram/, web/	✅
9	Web gateway	web/app, session, identity	✅
10	Domain & config	domain/*, config, models, lang/detect, cache/canonical	✅ (config+models ✅ în MASTERCLASS-RO)
11	Proactiv & joburi	proactive/, jobs/, gdpr/*	✅

🎉 **TOATE** cele 11 valuri complete — tot proiectul e acoperit.

Legendă: ✅ = deep-dive scris · □ = urmează. Marcajele „✅ în chat / în MASTERCLASS-RO” = deja explicat detaliat altundeva; îl consolidez aici la valul lui.

VALUL 1 — Pipeline: cele 11 stagii

Reamintire (motorul): fiecare stagiul e o funcție `async def stage(ctx, deps) -> None`. Runner-ul ([runner.py:47](#)) le rulează în ordine, măsoară, și **iese la primul** `ctx.reply` sau `ctx.halt`. Ordinea = de la ieftin la scump (ținta: 40-60% opresc înainte de LLM).

```
1 gates → 2 language → 3 clarify_resume → 4 greeting → 5 alias → 6 cache  
→ 7 faq → 8 triage → 9 handoff → 10 agent → 11 fallback
```

Firul roșu al tuturor stagiilor „gratuite” (3-9): dacă un stagiul anterior a setat `ctx.route`, stagiile de mai jos îl **respectă** (`if ctx.route is not None: return`) — un singur scriitor pe tur (principiul 3). Și toate sunt **best-effort**: o eroare → miss, NU rupe turul (principiul 6).

Stagiul 1 — `gates_stage` (filtrele de la intrare)

0. 📌 **Unde:** [gates.py:439](#). Primul stagiul real. Nodurile `G1 - G5` din Diagrama 4a. **Scrie:** `ctx.halt` (tăcere), `ctx.reply` (risc/moderare), și `ctx.message.body` (Vision + guardrails).

1. 🛡️ **Analogie:** bodyguard-ul de la ușa clubului. Verifică pe rând: ești pe listă? ești interzis? te porți urât? ai adus o poză? — și abia apoi te lasă înăuntru.

2. ❓ **De ce:** înainte să cheltui vreun ban pe LLM, trebuie să decizi determinist **dacă botul are voie să răspundă**. Un client blocat, o preluare de om, un mesaj toxic — toate se opresc aici, gratis.

3. ⚙️ **Cum (8 porți, în ordine, [gates.py:439-487](#)):**

```
if not ctx.bot_active:          halt_silent("bot_inactive")      # 1 kill-switch per conversație  
if ctx.contact.is_blocked:      halt_silent("contact_blocked")  # 2 abuse blacklist  
if handoff_until > now():       halt_silent("handoff_active")    # 3 om a preluat  
if await _rate_limited(...):    return                          # 4 prea multe mesaje  
if await _moderation_blocked(...): return                        # 5 mesaj toxic  
reason = detect_risk(body)      # 6 „vreau un om” / „avocat”  
if reason and handoff_enabled: request_human + reply; return  
if content_type=="image": await _route_image(...)              # 7 poză → Vision → text  
_apply_input_guardrails(ctx)    # 8 trunchiere + PII + injection
```

4. 📄 **Cazuri cu exemplu:** | Poartă | Exemplu | Ce se întâmplă | |---|---|---| | 1 bot inactiv | admin a oprit botul pe conv | tăcere (omul scrie din inbox) | | 3 handoff activ | un om a preluat acum 5 min | tăcere până la `handoff_until` | | 4 rate limit | 25 mesaje în 60s | un mesaj „primesc multe mesaje...” o dată, apoi tăcere | | 5 moderare | insultă gravă | răspuns neutru + contor flag; la 3 flag-uri/24h → blocat | | 6 risc | „vreau să vorbesc cu un om” | escaladează + „te conectez cu un coleg” (doar pe WA/TG) | | 7 poză | poză cu o cremă | Vision → „cremă hidratantă X” devine body → search | | 8 mesaj lung | 3000 caractere | tăiat la 2000 (`INBOUND_BODY_MAX`) | | 8 PII | „sună-mă la 0712345678” | „sună-mă la [telefon]” (nu intră în prompt) |

5. 🧠 **De ce ordinea asta:** ieftin înainte de scump (rate limit e un check Redis, moderarea e un apel API → rate limit primul). Moderarea ÎNAINTE de risc: abuzul primește răspuns neutru, nu un om. Vision DUPĂ rate-limit: nu cheltui Vision pe un contact throttled.

6. 🚨 **Ce-ar fi dacă scoți** `handoff_enabled_for` **la poarta 6:** pe web (fără operator), „vreau un om” ar escalada în gol → botul ar tăcea, dar n-ar veni nimeni → tăcere de facto. De aceea pe web ruta e lăsată să curgă (botul asistă singur).
7. 🐞 **Debug:** `grep gate_halt` (cu `reason`) → de ce a tăcut. `grep message_moderated` → ce a fost flagged (doar categoriile, nu corpul). `grep body_truncated / input_pii_masked` → guardrails.
8. ✅ **Test:** Client trimite o poză cu caption „mai aveți crema asta?”, dar Vision pică (API jos). Ce devine `ctx.message.body` ? (indiciu: fail-soft păstrează caption-ul).
-

Stagiul 2 — `language_stage` (detectarea limbii)

0. 📌 **Unde:** `language.py:27`. **Scrie:** `ctx.language` + persistă `conversations.locale`. Nu setează reply/halt.
1. 🗣️ **Analogie:** recepționarul care aude în ce limbă vorbești și trece toată discuția pe limba aia.
2. ❓ **De ce AICI (după Gates, înainte de Cache):** toate straturile de mai jos (cache, FAQ, triaj) caută pe `locale`. Un cache hit în limba greșită = **bug**, nu hit (principiul 11). Deci limba trebuie corectă ÎNAINTE de ele.
3. ⚙️ **Cum:**
- ```
if len(supported) <= 1: return # tenant mono-lingv → zero DB
detected = detect_language(ctx.message.body, supported) # cod PUR, zero LLM
if detected is None or detected == ctx.language: return # fără semnal / deja corect → păstrează
ctx.language = detected
await set_conversation_locale(...) # se „lipește” pt tururile următoare
```
4. 🗨️ **Cazuri:** | Situație | Rezultat | |---|---| | tenant doar RO | return imediat (nimic de detectat) | | mesaj clar în maghiară | `ctx.language = hu`, persistat | | mesaj ambiguu (1 cuvânt) | `detected=None` → păstrează limba curentă (precision-first) | | persist DB pică | limba rămâne pe `ctx` pentru ACEST tur (best-effort) |
5. 🧠 **De ce „precision-first” (păstrează la incertitudine):** mai bine rămâi pe limba precedentă decât să sari pe o detectare nesigură și să răspunzi în limba greșită.
6. 🚨 **Ce-ar fi dacă rulează DUPĂ cache:** un client maghiar ar putea primi un cache hit românesc (cheia de cache s-ar face pe limba veche). Poziția în pipeline nu e cosmetică.
7. 🐞 **Debug:** `grep language_detected` (cu `from / to`).
- 

## Stagiul 3 — `clarify_resume_stage` (reia o întrebare în așteptare)

---

0. 📌 **Unde:** `clarify.py:28`. **Scrie:** `ctx.route`, `ctx.state.constraints[field]`, `ctx.state.asked_intents`. Nodurile CLR / CONS / RESUME din 4a.
1. 🗣️ **Analogie:** ai întrebat clientul „ce buget ai?” la tura trecută. Acum el zice „200 lei”. Nu o iei de la capăt ca și cum ar fi un mesaj nou — știi că „200 lei” e **răspunsul la întrebarea ta**.
2. ❓ **De ce:** răspunsul la o întrebare pe care NOI am pus-o nu trebuie să coste un apel de triaj (nano), nici să fie re-clasificat izolat („200 lei” → „ambiguu”). E slot-filling determinist.

### 3. ⚙️ Cum:

```
pq = ctx.state.pending_question
if not isinstance(pq, dict): return # nimic în așteptare
if ctx.message.content_type == "image": return # o poză NU e răspuns la un slot de text
answer = ctx.message.body.strip()
ctx.state.constraints[pq["field"]] = answer # 1. umple slotul
ctx.state.asked_intents.append(pq["field"]) # anti-loop
attempts = int(pq.get("attempts") or 0)
if attempts >= clarify_max_attempts: # 2. anti-bucă → escaladează
 ctx.route = RouteDecision(HANDOFF if field=="intent" else SALES); return
ctx.route = RouteDecision(Route(pq["resume_route"])) # 3. rutează determinist (triaj = no-op)
```

4. 📄 **Cazuri cu exemplu:** | Situație | Rezultat | |---|---| | Am întrebat „ce buget?” → client: „200 lei” | `constraints[budget]="200 lei"`, rută SALES, agentul caută cu bugetul | | Am întrebat, client trimite o POZĂ | `return` (poza nu umple slotul de text; slotul rămâne) | | Am re-întrebat de 2 ori același slot „intent” | escaladare la HANDOFF (nu întrebăm la infinit) | | Am re-întrebat de 2 ori „buget” | escaladare best-effort SALES (agentul lucrează cu ce are) |
5. 🧠 **De ce `attempts` (anti-bucă):** dacă întrebi de 3 ori „ce cauți?” și clientul tot nu e clar, o buclă infinită încalcă principiul 6. După N încercări, escaladezi.
6. 🚨 **Ce-ar fi dacă rulează DUPĂ greeting:** „200 lei” nu e salut, dar dacă `pending_question` era setat și clarify rula după alte stagii, riști ca „da” (răspuns la slot) să fie tratat ca altceva. De aceea e devreme, imediat după limbă.
7. 🐞 **Debug:** `grep clarify_resumed / clarify_escalated` (P12: fără `answer`, poate fi PII).
8. ✅ **Test:** `pending_question={field:"budget", attempts:2}`, `clarify_max_attempts=2`, client: „nu știu”. Ce se întâmplă?

## Stagiul 4 — `greeting_stage` (welcome determinist)

0. 📌 **Unde:** `greeting.py:184`. **Scrie:** `ctx.reply` (early-exit). Nodurile `GREET` / `WELCOME` din 4a.
1. 🗣️ **Analogie:** vânzătorul care spune „Bună! Sunt X, cu ce te pot ajuta?” când intri — fără să cheme managerul pentru un simplu salut.
2. ❓ **De ce:** un pur salut („bună”) nu trebuie să coste un apel de triaj. Free layer, determinist, branded (numele botului + sugestii), configurabil per business.
3. ⚙️ **Cum:**

```
if not enabled: return
if not is_greeting(ctx.message.body): return # PUR salut? (normalizat, exact în set)
text = build_welcome(business, language, bot_name, suggestions)
ctx.set_reply(text, cacheable=False)
```

`is_greeting` (`greeting.py:139`) normalizează (lowercase, fără diacritice, doar litere) și verifică **match exact** într-un set de saluturi RO/EN/HU.

4. 🚩 **Cazuri cu exemplu:** | Mesaj | `is_greeting` ? | Rezultat | |---|---|---| | „Bună ziua!” | da ( → `buna ziua` ∈ set) | welcome branded + sugestii | | „salut, caut o cremă” | **nu** (nu e pur salut) | no-op → pipeline continuă → triaj → sales | | „hi” | da | welcome | | „szia” | da (HU) | welcome în maghiară |
5. 🧠 **De ce match EXACT și conservator:** dacă mesajul curățat NU e exact un salut, nu trănțești welcome. Mai bine ratezi un salut (îl duce triajul) decât să pui welcome peste o întrebare de produs.
6. 💣 **Ce-ar fi dacă ai folosi „conține salut” în loc de „exact”:** „bună, unde e comanda mea?” ar primi welcome în loc de status comandă. Exactitatea previne asta.
7. 🐛 **Debug:** `grep welcome_sent`. Configurabil din `businesses.settings["welcome"]` (nume bot, sugestii, on/off).

## Stagiul 5 — `alias_stage` (match exact pe fraze aprobate)

0. 📌 **Unde:** `alias.py:46`. **Scrie:** `ctx.reply` (hit FAQ) SAU `ctx.route` (hit route/product/category). Nodurile `ALIAS` / `AFAQ` / `AROUTE` din 4a.

1. 🗣️ **Analogie:** o listă de „comenzi rapide” învățate. Dacă clientul scrie exact o frază știută („program magazin”), răspunzi instant din memorie, fără să gândești.

2. ❓ **De ce:** cel mai IEFTIN strat — lookup pe index B-tree, **zero token** (nici măcar embed). Frazele se populează din shadow mode (NX-93). Gol pe demo, dar mecanismul e acolo.

3. ⚙️ **Cum:**

```
if ctx.route is not None: return # deja rutat (clarify/alias) → nu suprascrie
phrase_norm, _ = canonicalize(body) # ACEEAȘI normalizare cu care se scriu aliasele
alias = await lookup_alias(conn, business_id, phrase_norm)
if alias.kind == "faq": ctx.set_reply(get_faq_answer(...)) # early-exit
elif alias.kind == "route": ctx.route = RouteDecision(route) # scurtcircuitază triajul
else: ctx.route = RouteDecision(SALES, category_key=...) # product/category → agent
```

4. 🚩 **Cazuri:** | Alias | `target_kind` | Rezultat | |---|---|---| | „program magazin” → FAQ orar | faq | servește răspunsul (cacheabil) | | „vreau retur” → route order | route | `ctx.route=ORDER`, triaj sărit | | „vitamina c” → category serums | category | `ctx.route=SALES, category=serums` → agent | | FAQ lipsă în limba curentă | faq | miss grațios (principiul 11) |

5. 🧠 **De ce `canonicalize` refolosit:** aliasele se SCRIU cu aceeași normalizare cu care se CAUTĂ. Dacă ar diferi, match-ul exact ar rata mereu. Un mic detaliu care face sau strică stratul.

6. 💣 **Ce-ar fi dacă alias ar suprascrie `ctx.route` deja setat:** ar călca peste `clarify_resume` → doi scriitori pe `route` → bug de design. De aceea `if ctx.route is not None: return`.

7. 🐛 **Debug:** `grep alias_lookup` (hit + `target_kind` + reason). P12: NICIODATĂ `phrase_norm` în log.

## Stagiul 6 — `cache_stage` (cache semantic)

0. 📌 **Unde:** `cache.py:89`. **Scrie:** `ctx.reply`, `ctx.from_cache`. Nodul `CACHE` / `CHIT` din 4a.

1. 📖 **Analogie:** un caiet cu răspunsuri deja date. Dacă vine aceeași întrebare (sau una foarte asemănătoare), citești din caiet în loc să gândești din nou.
2. ❓ **De ce:** query-uri repetate („cât e livrarea?") nu trebuie să coste un apel de generare LLM. Serviți din cache = rapid + gratis.
3. ⚙️ **Cum (2 straturi + 3 tiere de volatilitate):**

```
volatility = classify_volatility(body) # static / dynamic / realtime / contextual
if volatility in ("realtime","contextual"): return # comandă/personal SAU „mai ieftin" → bypass
canonical, hash = canonicalize(body)
hit = await exact_lookup(...) # L1: hash → O(1), zero false-positive
if hit and await _serve(...): return
embedding = await llm.embed([canonical]) # L2: semantic
cand = await semantic_lookup(...)
if cand and similarity >= cache_tau_high (0.92) and await _serve(...): return
```

4. ✂️ **Cazuri cu exemplu:** | Query | Ce se întâmplă | |---|---| | „cât e livrarea?" (identic cu unul cache-uit) | L1 exact → servit instant, `from_cache=True` | | „în cât timp ajunge coletul?" (parafrază) | L2 semantic  $\geq 0.92$  → servit | | „unde e comanda mea?" | `realtime` → bypass (răspuns specific userului) | | „ceva mai ieftin" | `contextual` → bypass (relativ la setul afișat — cache-ul ar servi alt baseline) | | hit `dynamic` cu preț schimbat | price-check pică → **evict lazy** + tratat ca miss (regenerează cu preț proaspăt) |
5. 🧠 **De ce price-check pe `dynamic` ( `_is_fresh_dynamic` , `cache.py:46`):** un răspuns cu produse cache-uit poate avea prețuri vechi. Înainte să-l servești, verifici prețurile curente + `data_version` . Diferit → nu servi preț învechit (self-healing).
6. 💣 **Ce-ar fi dacă n-ai bypass la `contextual` :** „mai ieftin" al clientului A (baseline 200 lei) ar fi servit din cache clientului B (baseline 50 lei) → răspuns greșit. Cache poisoning.
7. 🐛 **Debug:** `grep cache_lookup` (layer: exact/semantic/miss/**stale\_evict**), `cache_bypass` . Bug istoric: codec pgvector greșit → cache murea tăcut (fix connection.py).
8. ✅ **Test:** De ce e L1 (exact) verificat înaintea lui L2 (semantic)?

---

## Stagiul 7 — `faq_stage` (răspuns din baza de cunoștințe)

---

0. 🔑 **Unde:** `faq.py:45`. **Scrie:** `ctx.reply` . Nodul `FAQ / FHIT` .
1. 📖 **Analogie:** un dosar cu întrebări frecvente (retur/livrare/garanție). Cauți în el înainte să chemi vânzătorul.
2. ❓ **De ce separat de cache:** separarea proprietarilor — `cache_stage` deține `semantic_cache` , `faq_stage` deține `faqs` . FAQ-urile sunt curate (editate de client), deci prag puțin mai relaxat.
3. ⚙️ **Cum (cu praguri inteligente):**

```

canon = canonicalize(body)[0] # paritate cu seed-ul FAQ
emb = await llm.embed([canon])
hit = await semantic_lookup(..., locale=ctx.language, model=model)
msg_is_policy = _POLICY_RE.search(canon) # întrebare de livrare/plată/retur/garanție?
faq_is_policy = _POLICY_RE.search(hit.question)
is_policy = msg_is_policy and faq_is_policy
tau = faq_tau_policy (0.45) if is_policy else faq_tau_high (0.78) # prag relaxat DOAR pe politică
if hit and sim >= tau: ctx.set_reply(hit.answer, cacheable=(sim >= faq_tau_high))

```

4.  **Cazuri cu exemplu:** | Query | Prag folosit | Rezultat | |---|---|---| | „cum returnez un produs?” (întrebare clară de politică, FAQ de politică) | 0.45 (relaxat) | servit chiar la cosine mic | | „ce ser recomanzi?” (produs) | 0.78 (strict) | doar dacă e potrivire bună | | „rituals sună bine, aveți livrare?” (MIXT) | 0.45 pe FAQ livrare, dar cacheable=False | servit, dar nu otrăvește cache-ul | | user HU, FAQ doar RO (fallback gated) | 0.85 (strict) | servește cunoștința RO (nu traduce) |

5.  **De ce pragul relaxat DOAR pe politică + gate pe tipul FAQ-ului (NX-138):** un mesaj mixt (produs + livrare) „diluează” embedding-ul → similaritatea la FAQ-ul de livrare cade sub 0.78 → întrebarea de livrare n-ar prinde niciodată, iar agentul re-recomandă („bug copy-paste”). Regexul dă precizia care permite pragul jos. Dar pragul jos se aplică **doar dacă și FAQ-ul potrivit e de politică** — altfel un FAQ de consultanță ar „salva” pe un mesaj mixt și ar deflecta cererea de produs.

6.  **Ce-ar fi dacă hit-ul relaxat pe mesaj mixt ar fi** `cacheable=True` : query-ul mixt ar otrăvi `semantic_cache` pentru alte mesaje similare → de aceea `cacheable=(sim >= faq_tau_high)`.

7.  **Debug:** `grep faq_hit` (similarity + policy flag), `faq_lookup layer=miss`, `locale_unserved` (semnal că trebuie seedate cunoștințe în limba userului).

## Stagiul 8 — `triage_stage` (nano clasifică ruta)

0.  **Unde:** `triage.py:212`. Primul touchpoint LLM. **Scrie:** `ctx.route` (+ reply pentru simple/clarify). Nodurile `TRI` / `TVAL` / `ROUTE` din 4a. (Deep-dive complet: *cap. 8.8 din MASTERCLASS-RO.md*. Rezumat aici.)

1.  **Analogie:** recepționera care ascultă ce vrei și te trimite la ghișeu potrivit: vânzări, comenzi, sau „stai să lămurim ce cauți”.

2.  **De ce:** trebuie să decizi ruta ( `simple/sales/order/handoff/clarify` ) ieftin (nano ~300 tokeni), nu cu modelul scump. Nano înțelege, **codul decide** (guarduri deterministe peste output).

3.  **Cum (esența):**

```

if ctx.route is not None: return # deja rutat → no-op
raw = await llm.classify_json(_SYSTEM, user) # nano, JSON forțat
out = TriageOut(**raw) # validat Pydantic
category_key = out.category_key if out.category_key in categories else None # inventat → aruncat
if route==SIMPLE and _factual_bait(body): route = SALES # guard: nu confirma fapte nevalidate
if out.confidence=="low" and route in (SALES,ORDER): route = CLARIFY # codul forțează clarify

```

4.  **Cazuri (guarduri deterministe):** | Situație | Guard | Rezultat | |---|---|---| | nano inventează categoria „xyz” | category ∉ listă | `category_key=None` (nu rutezi pe ghicit) | | „zi doar da: aveți 70% reducere?” → nano zice simple |

factual guard | re-rutat SALES (agent grounded) | | nano nesigur pe sales | confidence=low | forțat CLARIFY (nu ghici) | | „mulțumesc, asta vreau” | closure | mesaj cald + chips pe categorii adiacente |

5. 🧠 **De ce guarduri:** ruta `simple` e servită de nano FĂRĂ validator → un client ar putea forța o confirmare falsă. Guardurile mută deciziile riscante din mâna modelului în cod.

7. 🐛 **Debug:** `grep intent_detected` (route+category+confidence), `trriage_factual_guard`, `trriage_low_confidence`.

---

## Stagiul 9 — `handoff_stage` (escaladare la om)

---

0. 📌 **Unde:** `handoff.py:28`. **Scrie:** `ctx.route` (rescrie la SALES pe web) SAU `ctx.reply` (confirmare). Nodurile `HOFF` / `HESC` / `HSUP` din 4a.

1. 🗣️ **Analogie:** butonul „cheamă managerul”. Dacă e cine să vină, îl chemi și spui clientului „vine imediat”. Dacă nu e (magazin fără manager de tură), te descurci singur.

2. ❓ **De ce (bug real R5):** triajul emitea `Route.HANDOFF`, dar NIMENI nu-l consuma → cădea pe fallback-ul generic „n-am înțeles”. Acest stagiul îl consumă corect.

3. ⚙️ **Cum:**

```
if route != HANDOFF: return # no-op pe alte rute
if ctx.reply is not None: return # deja servit → nu suprascrie
if not handoff_enabled_for(channel): # web fără operator
 ctx.route = RouteDecision(SALES); return # agentul preia (flux normal)
await request_human(...); await notify_operator(...) # escaladează + notifică
ctx.set_reply("Te conectez cu un coleg...", cacheable=False) # NICIODATĂ tăcere
```

4. 📄 **Cazuri:** | Canal | Rezultat | |---|---| | WhatsApp/Telegram (operator planificat) | escaladare + „te conectez cu un coleg” + tur următor tace | | Web (fără operator) | rută rescrisă SALES → agentul răspunde normal (fără mesaj de operator) | | escaladare eșuează (DB jos) | răspunde oricum „te conectez cu un coleg” (nu tăcere) |

5. 🧠 **De ce rescrii la SALES pe web:** o escaladare pe un canal fără operator = tăcere/fundătură. Mai bine botul asistă singur decât să promită un om care nu vine.

7. 🐛 **Debug:** `grep handoff_requested` / `handoff_suppressed`.

---

## Stagiul 10 — `agent_stage` (mini + tool loop + validator)

---

0. 📌 **Unde:** `agent.py:957`. 1411 linii. **Scrie:** `ctx.reply`, `ctx.retrieval`, `ctx.state_patch`. Diagrama 4b + 4c.

(Deep-dive COMPLET livrat în chat — Modulul 3, 7 unități: gărzi · intenții PRE-loop · merge\_constraints · prompt+tool loop · compunere POST-loop · arborele de finalize · VALIDATORUL. Se consolidează aici la Valul 4.)

**Rezumatul de reținut:** 3 faze — PRE-loop (intenții deterministe \$0: link/compare/show\_more) → tool loop (max 3, `execute` ține evidența) → POST-loop (checkout fallback/cross-sell/cheaper/superlativ → finalize rich/proză → **validator:** retry → fallback determinist). „Modelul propune, codul dispune.”

---

## Stagiul 11 — `fallback_stage` (plasa finală)

0. 📍 **Unde:** `runner.py:169`. ULTIMUL stagiu. **Scrie:** `ctx.reply`.

1. 🛡️ **Analogie:** plasa de siguranță de sub trapez. Dacă toate stagiile au ratat (nimeni n-a răspuns), prinzi clientul cu o întrebare de clarificare — **niciodată tăcere**.

2. ❓ **De ce:** garanția principiului 6. Dacă ajungi aici, ceva n-a fost acoperit (rută order/handoff neacoperită, triaj fără răspuns, fără cheie OpenAI). Mai bine o întrebare decât tăcere.

3. ⚙️ **Cum:**

```
ctx.set_reply("Hmm, n-am înțeles exact 😊 Cauți un produs anume, ai o întrebare despre o "
"comandă, sau altceva?", cacheable=False)
```

4. 📄 **Cazuri:** ajungi aici DOAR dacă niciun stagiu de sus n-a setat `reply`. Ex.: `llm=None` (cost guard) + mesaj care nu-i salut/cache/FAQ/alias → toate ratează → fallback.

5. 🧠 **De ce** `cacheable=False`: e un non-răspuns specific contextului — nu vrei să-l servești altui client din cache.

6. 💣 **Ce-ar fi dacă îl ștergi:** un tur necoperit ar ieși din pipeline cu `ctx.reply=None` → processor-ul logează „tur fără reply” → **client nu primește nimic** (tăcere). Fallback-ul e plasa.

7. ⚠️ **Slăbiciune (R7):** e doar în română. Un client HU/EN ajuns aici primește RO. Încalcă P11.

## Recap Valul 1 (schema mentală a pipeline-ului)

|          |                                                                            |
|----------|----------------------------------------------------------------------------|
| gates    | → poate botul răspunde? (activ/blocat/handoff/rate/moderare/risc/poză/PII) |
| language | → RO/HU/EN (înainte de straturile locale-keyed)                            |
| clarify  | → răspuns la o întrebare a noastră? umple slotul determinist               |
| greeting | → pur salut? welcome branded \$0                                           |
| alias    | → frază exactă știută? răspuns din index \$0                               |
| cache    | → query repetat? servit din cache (price-check pe dynamic)                 |
| faq      | → întrebare de cunoștințe? răspuns din faqs (1 embed)                      |
| triage   | → nano clasifică ruta (guarduri: category inventat/factual/low-conf)       |
| handoff  | → escaladare la om (sau SALES pe web)                                      |
| agent    | → mini + tools + validator (inima vânzării)                                |
| fallback | → nimic? întrebare de clarificare (niciodată tăcere)                       |

**Cele 2 reguli care leagă totul:** (1) un stagiu care setează `ctx.route` e respectat de cei de jos (un scriitor/tur, P3); (2) toate sunt best-effort — eroare = miss, nu rupe turul (P6).

**Test de val:** trasează mesajul „bună, caut un ser cu vitamina C sub 100 lei” prin toate cele 11 stagii. La care se oprește și de ce? (răspunde-mi în chat)

# VALUL 2 — Worker core (motorul care duce turul cap-coadă)

Astea sunt „organele interne” ale workerului: cine scoate mesajul din coadă, cine-l orcheestrează, cine măsoară, cine trimite răspunsul afară, plus utilitarele (debounce, spargere, cost, poartă de comandă).

`processor.py`, `consumer.py`, `runner.py` sunt explicate LINIE cu LINIE în Modulul 2 (chat) + [MASTERCLASS-RO.md](#) cap. 6-8. Aici le pun condensat, ca manualul să fie complet, și dau tratament COMPLET fișierelor noi.

## `runner.py` — motorul benzii rulante (condensat)

🔑 [runner.py:47](#). 🤖 dirijorul: rulează stagiile în ordine, măsoară fiecare, oprește la primul `reply / halt`. ? ca stagiile să fie funcții pure care nu știu nimic despre măsurare (principiul 10). ⚙️ `for stage in stages: await stage(ctx,deps); emit(...); if ctx.reply or ctx.halt: break`. 🧠 adaugi un stagiou nou fără să atingi runner-ul (doar în lista `DEFAULT_STAGES`). 🐛 `grep stage_completed` (latența fiecărui stagiou), `pipeline_early_exit`.

## `processor.py` — inima turului (condensat)

🔑 [processor.py:418](#) (`handle_turn`). 🤖 managerul de tură: dedupe → contact/conv → încarcă memoria → pipeline → **tranzacția Sender** → post-tur. ? un singur loc care orcheestrează tot turul, tranzacțional. ⚙️ vezi Modulul 2. **Punctul cheie — TX-ul atomic** ([processor.py:565-667](#)): mesaj outbound + outbox + patch state + finalizare dedupe = totul sau nimic. 🐛 `grep "tur procesat" / dedupe_hit_db / tăcere intenționată`.

## `consumer.py` — bucla Redis (condensat)

🔑 [consumer.py:275](#). 🤖 ospătarul care ia bonuri de pe cui (`XREADGROUP`) și le duce la bucătărie. ⚙️ citește → rulează (`message/status/order/callback`) → rezolvă tenant (`admin_conn`, singurul query privilegiat) → lock conversație → `handle_turn`. ⚠️ cele 2 găuri NX-140: excepție = ACK+drop ([consumer.py:228](#)); lock blocat = drop ([consumer.py:96](#)). 🐛 `grep "eroare la procesarea"`.

## `debounce.py` — coalesce mesaje rapide

0. 🔑 **Unde:** [debounce.py:32](#) (`Debouncer`). Folosit de consumer ([consumer.py:295](#)). Nodul `DEB` din Diagrama 3.
1. 🤖 **Analogie:** ospătarul care așteaptă 3 secunde după ce te oprești din vorbit, ca să ia TOATĂ comanda odată — nu fuge la bucătărie după fiecare cuvânt.

2.  **De ce:** clientul scrie des în rafală („vreau un ser" / „...cu vitamina C" / „...sub 100 lei"). Fără debounce ai avea 3 tururi (3× cost, 3 răspunsuri, context spart). Cu debounce = 1 tur cu tot contextul.

### 3. Cum:

```
async def add(self, event, msg_id):
 key = (channel, account, sender) # per expeditor
 self._buffers[key].append((event, msg_id))
 old_timer.cancel() # fiecare mesaj nou RESETEAZĂ timerul
 self._timers[key] = create_task(self._flush_later(key))

async def _flush_later(self, key):
 await asyncio.sleep(3.0) # 3s de liniște
 combined = self._combine(events) # body = mesajele lipite cu \n
 await self._process(combined) # procesează UN tur
 await self._ack(msg_ids) # ACK DUPĂ flush (durabilitate!)
```

4.  **Cazuri cu exemplu:** | Situație | Rezultat | |---|---| | 3 mesaje în 2 secunde | timer resetat de 2 ori → 1 tur cu body = „vreau un ser\n...cu vitamina C\n...sub 100 lei" | | 1 mesaj, apoi liniște 3s | flush → 1 tur normal | | flush eșuează (excepție în process) | **fără ACK** → mesajele rămân pending (PEL) → reaper le reia | | worker moare între buffer și flush | mesajele ne-ACK-uite → reaper (XAUTOCLAIM) le recuperează |

5.  **De ce ACK DUPĂ flush și nu la citire (NX-87):** durabilitate. Dacă ai face ACK la citire și apoi workerul moare, mesajele s-ar pierde (Redis crede că-s gata). ACK-after-flush = mesajul e „gata" doar când a fost procesat cu succes.

6.  **Ce-ar fi dacă add NU resetează timerul:** primul mesaj ar declanșa flush-ul după 3s, iar al 2-lea și al 3-lea ar face tururi separate → exact ce voiai să eviți.

7.  **Debug:** `grep "procesarea lotului a eșuat"` (flush fără ACK → pending). Buffer-ul e în memorie (best-effort); durabilitatea stă pe ACK, nu pe buffer.

8.  **Test:** Clientul scrie „bună" apoi, după 5 secunde, „caut un ser". Câte tururi? De ce?

---

## reply\_split.py — spargerea reply-ului lung

0.  **Unde:** `reply_split.py:27` ( `split_reply` ). Funcție **pură**, fără I/O. Folosit de processor (`processor.py:563`). Nodul FRAG din 3.

1.  **Analogie:** în loc să-ți trimită un perete de text, prietenul îți scrie în 2 bule scurte. Se citește mai ușor pe telefon, pare „om care scrie".

2.  **De ce:** un răspuns > 200 caractere e greu de citit pe mobil. Îl spargi la o **graniță naturală** (paragraf → linie → propoziție → spațiu), fără să rupi cuvinte.

3.  **Cum (cascada de granițe):**

```

if len(text) <= limit: return [text] # sub 200 → un fragment
for sep in ("\n\n", "\n"): # 1) paragraf/linie
 if head.rfind(sep) >= min_head: return [cap1, cap2]
end = max(rfind("."), rfind("!"), rfind("?")) # 2) sfârșit de propoziție
if end >= min_head: return _bullet_safe([...])
idx = head.rfind(" ") # 3) ultimul spațiu
if idx >= min_head: return _bullet_safe([...])
return [text[:limit], text[limit:]] # 4) fallback dur

```

4.  **Cazuri cu exemplu:** | Text | Rezultat | |---|---| | „Da, avem.” (10 car.) | [“Da, avem.”] — un fragment | | 300 car. cu 2 paragrafe | taie la `\n\n` → 2 bule curate | | 300 car. o singură propoziție lungă | taie la ultimul spațiu ≤ 200 | | listă cu bullet-uri | `_bullet_safe` mută un „•” orfan în fragmentul 2 |
5.  **De ce `min_head` (`limit//4`):** ca să nu ai un prim fragment minuscul („Da.”) + al doilea uriaș. O graniță prea devreme e sărită pentru una mai târzie.
6.  **Ce-ar fi dacă ai rupe la exact 200 mereu:** ai tăia cuvinte la jumătate („recoman-dare”). Urât. De aceea cauți granițe naturale, cu fallback dur doar dacă nu există niciuna.
7.  **Debug:** `grep reply_split` (event `parts=2`). NB: rich/carusel NU se sparge (ar strica ordinea cardurilor — vezi processor pas 7).

## callback.py — navigarea caruselului

0.  **Unde:** `callback.py:36` ( `handle_callback` ). Apelat de consumer pe `kind=callback` (`consumer.py:179`). Nodul CAROUSEL din 3.

1.  **Analogie:** clientul apasă săgeata ► la un carusel de produse. Nu e o întrebare nouă — e doar „arată-mi cardul următor”. Nu chemi vânzătorul, doar întorci pagina.
2.  **De ce:** o apăsare ◀/▶ e **UI deterministă**, NU trebuie să treacă prin triaj/agent (cost + LLM degeaba). Citești setul afișat din state și editezi cardul.
3.  **Cum:**

```

idx = parse_nav(event["data"]) # "car:nav:3" → 3
products = conv["state"]["displayed_products"] # setul persistat de Sender
if not 0 <= idx < len(products): return None # index invalid → no-op
payload = {"type":"edit_media", "card_message_id":..., "products":products, "index":idx}
outbox_id = await enqueue_outbox(..., f"cb:{provider_msg_id}", payload) # EDIT prin outbox

```

4.  **Cazuri:** | Situație | Rezultat | |---|---| | apasă ► la index valid | editează cardul (`edit_media` prin `outbox`) | | callback necunoscut (nu `car:nav:`) | no-op, log | | card expirat (state fără produse) | no-op (index în afara setului) | | re-livrare Telegram a aceleiași apăsări | idempotent ( `cb:{provider_msg_id}` ) → nu dublează |
5.  **De ce tot prin outbox (nu edit direct):** principiul 5 — **un singur punct de ieșire**. Chiar și o editare de card iese prin outbox → dispatcher. Zero logică de trimitere duplicată.
6.  **Ce-ar fi dacă nu verifici `0 <= idx < len`:** un index din afara setului (card vechi) ar da `IndexError` sau ar afișa produsul greșit. Guard-ul face no-op curat.

7. 🐞 **Debug:** `grep carousel_navigated (to_idx, total, product_id)`.

---

## `limits.py` — cost guard + rate limit (contoare Redis)

---

0. 📌 **Unde:** `limits.py`. Folosit de gates (rate limit) + processor (cost). Vezi și cap. 16 din [MASTERCLASS-RO.md](#).
  1. 🗨️ **Analogie:** contorul de la intrare + casa de marcat. Numără câte mesaje trimiți (rate limit) și cât ai cheltuit azi (cost), ca să nu depășești bugetul.
  2. ❓ **De ce:** tool-calling-ul face 2-4× apeluri mini/tur → costul poate scăpa. Redis e guard-ul REALTIME (rapid); facturarea reală rămâne `usage_daily`. Contoarele de aici sunt o plasă.
  3. ⚙️ **Funcțiile cheie:** | Funcție | Ce face | |---|---| | `rate_limit_count` (:50) | INCR mesaje/fereastră fixă → gates decide throttle | | `cost_over_budget` (:59) | pre-check read-only: azi ≥ plafon? | | `cost_add_and_total` (:75) | **INCR atomic + întoarce totalul** (fără TOCTOU) | | `spend_over_cap` (:139) | plafon per-contact (fereastră 24h) | | `web_cost_over_visitor_cap` (:93) | plafon per-vizitator web | | `seed_daily_cost` (:161) | reseed din `usage_daily` (supraviețuiește FLUSHALL) |
  4. 🚧 **Cazuri (cele 3 plafoane):** | Plafon | Cheie Redis | Scop | |---|---|---| | business/zi | `cost:{biz}:{today}` | un tenant nu depășește \$5/zi | | contact/zi | `spend:{biz}:contact:{id}` | o conversație în buclă nu arde plafonul tenantului | | vizitator web | `webcost:{biz}:{visitor}:{today}` | un token furat nu golește bugetul |
  5. 🧠 **De ce „POST-increment fără TOCTOU”** (`cost_add_and_total`): dacă ai `cost_over_budget` (citește) apoi `cost_add` (scrie) separat, între ele pot trece N tururi concurente care toate văd „sub plafon”. Increment atomic + compară DUPĂ → chiar dacă scapă câteva, turul următor e blocat determinist.
  6. 💣 **Ce-ar fi dacă contoarele ar prinde erorile Redis:** ar bloca traficul când Redis e jos. De aceea funcțiile NU prind erori — caller-ul le tratează **fail-open** (guard indisponibil ≠ trafic blocat).
  7. 🐞 **Debug:** `grep cost_guard_tripped / contact_spend_capped`. ⚠️ `estimate_turn_cost` (:186) e **cod mort** (înlocuit de costul exact din tokeni, NX-125) — de șters.
  8. ✅ **Test:** De ce contorul zilnic are TTL de 2 zile (nu 1)?
- 

## `order_gate.py` — poarta de comandă conștientă de identitate

---

0. 📌 **Unde:** `order_gate.py`. Folosit de agent + tool-urile de comandă.
1. 🗨️ **Analogie:** ca să-ți verific comanda, am nevoie să știu CINE ești. Pe WhatsApp știu (numărul = contul). Pe web anonim NU știu (n-ai cont) → te rog să te loghezi.
2. ❓ **De ce (bug real NX-128):** pe web nu există cont (contact throwaway) → `check_order` (scoped pe `contact_id`) NU găsește nimic, oricât de corect ar fi numărul. Mesajul vechi „n-am găsit pe acest cont” era înșelător + intra în buclă (modelul cerea nr/email pe care tool-ul nu le putea folosi).
3. ⚙️ **Cum:**

```
def web_unidentified(ctx):
 # canal anonim fără identitate verificată?
 if channel_kind in IDENTIFIED_CHANNELS: return False # WhatsApp/Telegram = identificat
 return not ctx.verified_customer_ref # web fără login = True
mesaje deterministe per-locale: login_required / no_orders
```

4.  **Cazuri cu exemplu:** | Canal | „unde e comanda mea?” | Rezultat | |---|---|---| | WhatsApp | numărul = contul | `check_order` caută normal | | Web anonim | fără cont | mesaj de login („intră în cont și revino”) | | Web cu login passthrough (JWT) | `verified_customer_ref` setat | trece de poartă (NX-129) | | WhatsApp, dar fără comenzi | — | „nu găsesc comenzi pe contul tău” (onest) |

5.  **De ce „FAQ-first” (NX-128+ +):** zidul de login NU e pe toată ruta ORDER. „Cum returnez?” (proces) se răspunde fără cont (FAQ). Zidul apare DOAR când modelul cheamă `check_order` (lookup ce chiar cere cont).

6.  **Ce-ar fi dacă n-ai `web_unidentified`:** clientul web ar primi „n-am găsit comanda” într-o buclă infinită (modelul re-cere numărul) → frustrare + tur pierdut.

7.  **Debug:** mesajul de login apare `cacheable=False` (context-relativ). `with_handoff` se adaugă DOAR dacă tenantul are `request_human` activ (nu promiți un coleg inexistent).

## `dispatcher.py` — IEȘIREA (outbox → canale)

0.  **Unde:** `dispatcher.py:245` ( `run_dispatcher` ). Proces SEPARAT. Nodurile `DISP` / `RENDER` / `SEND` din Diagrama 3. **Singurul care trimite efectiv** (principiul 5).

1.  **Analogie:** oficiul poștal. Sender-ul pune scrisori în cutie (outbox); poștașul (dispatcher) le ia, alege plicul potrivit fiecărui destinatar (canal) și le livrează, marcând „trimis” sau „eșuat”.

2.  **De ce proces separat:** ca livrarea (care depinde de rețea/Meta) să nu blocheze procesarea (worker). Decuplare prin `outbox`. Dispatcher-ul poate face retry independent.

3.  **Cum (bucla):**

```
business_ids = await business_ids_with_due_outbox(conn) # control plane: cine are scadente
for business_id in business_ids:
 async with tenant_conn(business_id) as conn:
 rows = await claim_due(conn, business_id) # FOR UPDATE SKIP LOCKED
 for row in rows: await dispatch_row(...) # trimite fiecare
```

`dispatch_row` (`dispatcher.py:129`):

```
sender = registry.get(channel_kind) # NX-60: transportul din registru
branch = choose_render(payload, ptype, sender.capabilities) # NX-115: ce formă randez?
provider_id = await sender.send...(...) # trimite (rich/carousel/template/text)
async with conn.transaction(): # atomic: nu rămâne 'sent' fără id
 await mark_sent(...); await set_message_provider_id(...)
```

4.  **Cazuri — `choose_render` (degradare grațioasă, `dispatcher.py:101`):** | Payload | Canal are capability? | Ramura | |---|---|---| | `rich` | WhatsApp are RICH? | `rich` (carduri native) | | `rich` | canal fără RICH | `text` (floor

aplatizat) | | `template` (proactiv) | WhatsApp TEMPLATE | `template` (Meta randează) | | `template` | canal fără TEMPLATE | `text` (degradare vizibilă) | | `carousel` | canal fără CAROUSEL/CARDS | `text` (lead-in ca text) | | `edit_media` | canal fără EDIT | `edit_unsupported` → **dead** (nu degradează — e UI) |

5. 🧠 **De ce** `FOR UPDATE SKIP LOCKED` : dacă rulezi 2 dispatchere, fiecare ia rânduri **diferite** (nu dublă trimitere). Iar `claim` împinge `next_attempt_at` (visibility timeout) → un dispatcher mort între claim și mark nu pierde rândul (redevine scadent). Self-healing.

6. 💣 **Ce-ar fi dacă** `mark_sent` + `set_message_provider_id` **NU erau în aceeași TX**: ai putea avea outbox „sent” dar mesajul fără `provider_msg_id` → statusurile delivered/read n-ar mai găsi mesajul.

7. 🐛 **Debug**: `grep "outbox .* trimis"` (succes), `render_path` (degradare rich→text VIZIBILĂ, NX-127), `mark_failed`. ⚠️ Slăbiciune R6: dispatcher secvențial + poll 2s → un tenant lent întârzie restul.

8. ✅ **Test**: Un reply `rich` merge spre un canal fără RICH. Ce primește clientul? Se pierde ceva?

## Recap Valul 2

```
consumer → scoate din coadă, rutează, rezolvă tenant, lock
debounce → coalesce mesaje rapide (3s) → 1 tur
processor → orchestrează turul + TX atomic (mesaj+outbox+state+dedupe)
runner → rulează cele 11 stagii, măsoară, early-exit
reply_split → sparge text lung în 2 bule (granițe naturale)
callback → navigare carusel (edit prin outbox, non-LLM)
limits → cost guard (3 plafoane) + rate limit (contoare Redis fail-open)
order_gate → poarta de comandă (web anonim → login)
dispatcher → outbox → canale (choose_render + degradare grațioasă + self-healing)
```

**Firul roșu**: decuplare prin coadă (Redis) și outbox (DB); atomicitate în TX; self-healing la fiecare graniță (dedupe, ACK-after-flush, visibility timeout); degradare grațioasă peste tot (niciodată tăcere).

## VALUL 3 — Compunere & grounding (paza anti-halucinație)

Aici trăiește obsesia centrală a proiectului: **botul nu inventează nimic**. Modelul emite doar cuvinte + referințe `product_id`; **codul hidratează faptele** (preț, rating, link) și **curăță proza** de orice cifră/claim neverificabil. Plus utilitățile care „învață” clientul (profil, rezumat) fără PII.

`compose.py` — **grounding-ul căii RICH (cel mai important)** 🎯

0. 📍 **Unde**: `compose.py:329` ( `assemble` ). **Pur** (fără I/O, fără LLM, fără DB). Diagrama 4c. Apelat de `_finalize_rich` (agent) + randorul web.

1. 🗨️ **Analogie:** un editor sever care primește textul unui copywriter entuziast. Copywriter-ul (modelul) scrie „serul ăsta cu 15% vitamina C reduce ridurile și e cel mai vândut!”. Editorul (compose) taie: „15%” (nu-i în date) → scos; „reduce ridurile” (medical) → scos; „cel mai vândut” (neverificabil) → scos. Rămâne doar ce e ancorat în fapte reale.

2. ❓ **De ce:** calea rich (carduri) e cea care servește MAJORITATEA recomandărilor. E echivalentul validatorului, dar pentru carduri: garanția că niciun preț/produs/claim inventat nu ajunge la client.

3. ⚙️ **Cum funcționează `assemble`:**

```
facts = {p["id"]: p for p in retrieved} # dicționar de fapte reale
1. MEMBERSHIP: doar product_id care sunt în retrieval (id inventat → DROP tăcut)
for it in j["items"]:
 if it["product_id"] in facts and not dup: llm_items[pid] = it
2. ORDINE = rankingul de retrieval (determinist), nu ordinea liberă a modelului
ordered_ids = [p["id"] for p in retrieved if p["id"] in llm_items] # position bias eliminat
3. BUILD fiecare card: fapte din cod, proză scrubuită
RichItem(price=eff, rating=..., url=..., badge=derive_badge(...), # ← din DATE
 reason=_join_reason(scrub_prose(it["fit_clause"]), anchor)) # ← proză SCRUBUITĂ
4. PICK determinist = produsul cel mai bine clasat (items[0]), nu alegerea modelului
5. INTRO/EDUCATION scrubuite (cifrele clientului permise, restul DROP)
```

4. 🗨️ **Fiecare mecanism de grounding, cu exemplu:** | Mecanism | Funcție | Exemplu | |---|---|---| | Set-membership | `facts` lookup | model referă produs #999 (nu în retrieval) → cardul e **aruncat tăcut** | | Prețuri din cod | `RichItem(price=eff)` | modelul NU scrie prețul; codul pune `float(p["price"])` din DB | | Ordine deterministă | `ordered_ids` | modelul pune cel ieftin primul (bias) → codul reordonează pe ranking | | Scrub proză | `scrub_prose` | „reduce ridurile 20%” → `None` (cifră + claim) | | Scrub medical | `_unsafe_medical` | „tratează acneea” → câmp DROP (răspundere juridică) | | Off-category | `_off_category` | ceri „fond de ten” pe catalog skincare → intro onest „nu am exact, dar...” + pick suprimat |

5. 🧠 **De ce ordine + pick DETERMINISTE (ARCH-2026 P0):** modelul are „position bias” — pune primul produsul care apare primul în prompt sau cel mai ieftin. Dar produsul cel mai bine clasat (4.6★×148 recenzii) ar trebui primul, nu cel slab (4.4★×28). Codul decide ordinea (din ranking-ul blended); modelul doar **narează** (justificare). „Recomandarea mea” = `items[0]`, nu alegerea liberă a modelului.

5b. 🧠 **De ce scrub la nivel de PROPOZIȚIE la `education` (`scrub_education`, `compose.py:268`):** vechea versiune arunca TOT paragraful la o singură propoziție „murdară” — un „SPF 30” ucidea și sfatul util de lângă. Granular = păstrezi propozițiile sigure, arunci doar pe cele cu cifre nepermise. Așa botul „consultă” ca iZi, nu doar listează.

6. 💥 **Ce-ar fi dacă scoți membership-ul ( `if pid in facts` ):** un `product_id` halucinat de model ar deveni card cu preț/link inventate → clientul comandă un produs inexistent. Membership-ul e apărarea imbatabilă (nu poți hidrata fapte pentru un id care nu există în retrieval).

7. 🐛 **Debug:** `grep pick_suppressed` (off-category), `rich_downgraded` (rich→proză: `all-items-dropped-by-membership` = toate produsele au picat la membership, `structured-call-failed` = apelul structurat a crăpat).

8. ✅ **Test:** modelul emite `intro="Ai ceva bun sub 80 lei"` iar clientul a scris „sub 80 lei”. Trece `scrub_intro`? De ce (indiciu: `_allowed_client_numbers`)?

## Funcțiile-satelit din `compose`

- `flatten` (:466) — aplatizează `RichReply` în TEXT (floor pentru WhatsApp/cache).
- `flatten_framing` (:499) — pentru web (cardurile fac enumerarea; textul e doar framing + education).

- `build_comparison (:722)` — tabel comparativ 100% determinist (fiecare celulă = fapt real, zero LLM).
- `decision_axes / spec_numbers (:661/:704)` — NX-139: axele reale pe care variază setul + cifrele de specificație grounded.

---

## `text_scrub.py` — detectoarele de claim (creierul scrub-ului)

---

0. 📌 **Unde:** `text_scrub.py`. **Pur.** Partajat de `compose` (calea rich) și `agent._valid` (calea proză) — un singur loc canonic, fără drift de pattern.
1. 🗨️ **Analogie:** un radar care detectează 4 tipuri de „minciună periculoasă” în text: cifre inventate, procente, superlative, și claim-uri medicale.
2. ❓ **De ce un fișier separat:** dacă pattern-urile ar fi duplicate în `compose` și `agent`, s-ar desincroniza. Aici sunt o dată; ambele căi le folosesc.
3. ⚙️ **Cele 4 predicate:** | Predicat | Prinde | Cine-l folosește | |---|---|---| | `has_marketing_claim (:48)` | procent + claim + superlativ (FĂRĂ cifre) | `scrub_intro` (permite cifrele clientului) | | `has_unverifiable_claim (:56)` | cifre + procente + claim + superlativ | `scrub_prose` (calea rich) | | `has_text_claim (:64)` | claim + superlativ (fără cifre) | `agent._claims_ok` (calea proză) | | `has_stock_claim (:73)` | „pe stoc / disponibil” POZITIV | validat availability-aware | | `has_medical_claim (:134)` | claim MEDICAL (4 categorii) | P0-safety pe ambele căi |
4. 📄 **Cazuri cu exemplu — `has_medical_claim` (cel mai delicat):** | Text | Prinde? | De ce | |---|---|---| | „tratează acneea” | DA | verb terapeutic ( `trateaz` ) + afecțiune ( `acne` ) | | „hidratează tenul uscat” | NU | claim cosmetic legitim | | „sigur în sarcină” | DA | verdict de siguranță ( `_PREG_SAFE` ) | | „pentru acnee, consultă un dermatolog” | NU | redirect SIGUR (fără verb terapeutic) | | „fără alergeni” | DA | garanție absolută de inocuitate | | „recomandat de dermatologi” | DA | falsă autoritate medicală | | „testat dermatologic” | NU | claim cosmetic uzual |
5. 🧠 **De ce `has_stock_claim` sare peste negație:** „nu mai e pe stoc” / „revine pe stoc” sunt răspunsuri ONESTE de indisponibilitate. `_STOCK_NEG` (fereastră de 24 caractere înainte) le exclude — altfel ai respinge un răspuns corect.
6. 💣 **Ce-ar fi dacă `has_medical_claim` ar prinde „hidratează”:** ai scoate claim-uri cosmetice legitime → răspunsuri sterile, inutile. Detectorul e calibrat să prindă DOAR periculosul, nu tot.
7. 🐛 **Debug:** vezi `validator_rejected kind=claim` (agent) sau câmpurile devenite `None` (`compose`).

---

## `badges.py` — badge-uri de card din semnale reale

---

0. 📌 **Unde:** `badges.py:36` ( `derive_badge_kind` ). **Pur.** Gated de `card_badges_enabled` .
1. 🗨️ **Analogie:** eticheta „Top Vânzări” / „Reducere” de pe raft — dar pusă doar când e ADEVĂRATĂ, nu ca truc de marketing.
2. ❓ **De ce:** badge-urile („Top Favorit”, „Super Preț”) cresc conversia, dar NU au voie să fie inventate. Se **derivă** din date: rating + recenzii → „top”; reducere reală → „deal”.
3. ⚙️ **Cum:**

```
if list_price > price and (list_price-price)/list_price*100 >= 20%: return "deal" # reducere reală
if rating >= 4.7 and review_count >= 50: return "top" # top real
return None
```

4. 🚩 **Cazuri:** | Produs | Badge | De ce | |---|---|---| | preț 80, list\_price 120 (33% off) | „Super Preț” (deal) | reducere ≥ 20% | | 4.8★ cu 148 recenzii | „Top Favorit” (top) | rating + recenzii peste prag | | 5.0★ cu 1 recenzie | niciunul | recenzii < 50 (evită badge fals) | | reducere + rating mare | „Super Preț” | prioritate: deal > top |

5. 🧠 **De ce** `top_reviews=50` (nu 1): un produs 5★ cu o singură recenzie NU e „top favorit” — e zgomot. Pragul de recenzii previne badge-ul fals.

6. 🚩 **Ce-ar fi dacă badge-urile ar veni din model:** ar pune „Top” pe orice → încredere distrusă. De aceea sunt **derivate din date**, gated fail-safe (OFF → doar badge-uri pre-seedate curate).

## summarizer.py — rezumatul rolling (memoria conversațiilor lungi)

0. 📌 **Unde:** `summarizer.py:66` ( `generate_summary` ). Rulează POST-TUR async (nu blochează livrarea). LLM = **NANO**.

1. 🗣️ **Analogie:** la o ședință lungă, secretarul rezumă ce s-a discutat până acum într-un paragraf, ca să nu recitești tot procesul-verbal.

2. ? **De ce:** o conversație de 30 de mesaje nu încapă în prompt (buget de tokeni). Comprimi mesajele vechi (cele care ies din fereastra de 8) într-un rezumat scurt care ține firul.

3. ⚙️ **Cum:**

```
system = "rezumă factual: ce caută, produse/prețuri discutate, decizii, constrângeri, obiecții"
text = await llm.complete(system, user, model=llm.model_triage) # NANO
return _redact_pii(text) # scoate telefoanele (P12)
```

4. 🚩 **Cazuri:** vezi `processor._summarize_if_needed` — rulează doar la > 20 mesaje total, și re-rulează doar la ≥ 12 mesaje noi peste watermark (anti-regenerare). Watermark = cel mai nou mesaj INCLUS (onest).

5. 🧠 **De ce NANO (nu mini):** e o sarcină simplă (comprimare), nu vânzare. Nano e ieftin. Și e post-tur, deci nu întârzie răspunsul. Al treilea apel nano acceptat (ca profilul), nu un al treilea punct LLM sincron.

6. 🚩 **Ce-ar fi dacă n-ai** `_redact_pii`: un telefon menționat în conversație ar ajunge în rezumat → apoi în prompt → PII scurs din `channel_identities`. Redactarea defensivă e plasa peste instrucțiunea din prompt.

## profile.py — botul „învață” clientul (fără PII)

0. 📌 **Unde:** `profile.py:138` ( `extract_profile` ) + `:209` ( `compute_lead_score` ). POST-TUR async. LLM = **NANO**.

1. 🗣️ **Analogie:** vânzătorul care, după discuție, notează în fișa clientului „ten uscat, buget ~200 lei, îi place CeraVe” — dar NU are voie să noteze telefonul (ăla stă în alt registru securizat).

2. ? **De ce:** ca botul să-și amintească preferințele clientului între conversații ( `contacts.profile` ) + să estimeze cât de aproape e de cumpărare ( `lead_score` ).

### 3. Cum (două apărări):

```
delta = await extract_profile(llm, history, message, language) # nano extrage semnale
patch, dropped = filter_profile_patch(delta.profile_patch, vertical) # 1. WHITELIST de chei
new_score = compute_lead_score(delta.lead_signals, ctx) # 2. FORMULĂ, nu numărul LLM
```

4.  **Cazuri cu exemplu:** | Ce extrage nano | `filter_profile_patch` | Rezultat | `|---|---|---|` | `{skin_type: "uscat"} (beauty)` | `skin_type` ∈ whitelist | păstrat în DB || `{phone: "0712..."}` | `phone` ∉ whitelist | **aruncat** (PII nu intră în DB) || `{random_key: "x"}` | necunoscut | aruncat + semnal `profile_key_dropped` || `lead_signals ready_to_buy=true` | `compute_lead_score` | scor +20 (bază etapă + ponderi) |

5.  **De ce whitelist + formulă (nu numărul LLM-ului):** - **Whitelist:** modelul NU poate scrie chei arbitrare (sau PII) în DB, oricât ar încerca. Cheile de contact (telefon/email/nume) nu sunt în whitelist **prin construcție** → imposibil de scris. - **Formulă deterministă pentru scor:** `lead_score` intră în contracte/export CRM → trebuie **explicabil** și **stabil** între versiuni de model. Un număr inventat de LLM ar varia aleator.

6.  **Ce-ar fi dacă ai scrie direct `delta.profile_patch` în DB:** modelul ar putea pune `{"client_phone": "0712..."}` → PII în `contacts.profile` → încălcarea principiului 12. Whitelist-ul e apărarea structurală.

7.  **Debug:** `grep profile_updated (keys_set), profile_key_dropped` (ce a încercat modelul, redactat), `lead_score_updated (old→new)`.

8.  **Test:** De ce `lead_score` are un bonus de +5 din COD ( `_engaged_products` ) pe lângă semnalele nano?

---

## context.py — bugetul de context (condensat)

 **context.py.** Deja explicat COMPLET în [MASTERCLASS-RO.md](#) cap. 9. Rezumat: taie istoricul/profilul/state-ul/rezumatul la buget de tokeni impus în COD (transcript 6 tururi/1200 car., profil 300, state 3 produse/600, rezumat 600). Blocul stă în mesajul USER (dinamic) ca promptul static să rămână byte-identic → prompt caching OpenAI. `search_query` ia ultimele 2 mesaje ale clientului (follow-up-urile scurte caută în context). **Vezi cap. 9 pentru detaliile complete.**

---

## Recap Valul 3

```
compose → grounding rich: membership + ordine determinist + scrub + medical + badge + pick
text_scrub → detectoarele de claim (partajate rich+proză): marketing/unverifiable/text/stock/MEDICAL
badges → Top Favorit / Super Preț DERIVATE din date (nu inventate)
summarizer → rezumat rolling (nano, post-tur, PII redactat)
profile → învață clientul: whitelist chei + formulă scor (nu numărul LLM), zero PII
context → bugetul de tokeni în cod (transcript/profil/state/rezumat)
```

**Firul roșu:** „modelul propune, codul dispune”. Faptele (preț/rating/link/badge/scor) vin din COD/date; proza modelului e mereu SCRUBUITĂ; PII-ul e blocat structural (whitelist + redactare).

---

# VALUL 4 — Agent & LLM (cum vorbește cu OpenAI + cum e generat promptul)

Cusătura cu OpenAI + fabrica de prompturi + contabilitatea de tokeni. `agent.py` (stagiul care CONDUCE totul) e explicat integral în chat (Modulul 3) și rezumat la Valul 1.

## `llm.py` — cusătura cu OpenAI (condensat)

📌 `llm.py`. **SINGURUL** loc care vorbește cu OpenAI (principiul 2). Deja explicat COMPLET în [MASTERCLASS-RO.md](#) cap. 10. Rezumat: `get_llm()` → singleton sau `None` (degradare); `_sampling` pune `max_completion_tokens=800` pe agent (NU `max_tokens`, deprecat → 400) + temperatură per rol (triaj 0.2 / agent 0.7); `_with_retry` = retry mărginit pe tranzitoriu (429/5xx/timeout), respectă `Retry-After`, 4xx terminale ridică imediat; `run_tool_loop` = bucla de tool-calling (max 3, tools concurente via `gather`). **Vezi cap. 10 pentru linie cu linie.**

## `prompt_builder.py` — fabrica de prompturi **GENERATE** din DB 🏭

0. 📌 **Unde:** `prompt_builder.py:279` (`build_agent_system`). **Pur** (zero I/O, zero DB — primește datele deja citite). Principiul 9. Apelat de agent (`_load_prompt_inputs`).
1. 🗨️ **Analogie:** o fabrică de fișe de instrucțiuni pentru vânzător. Pentru fiecare magazin (tenant), generează un „manual” personalizat din categoriile lui, dar cu aceleași reguli de bază (nu inventa prețuri, max 3 tool-uri, siguranță medicală).
2. ❓ **De ce GENERAT din DB (nu hardcodat):** proiectul e multi-tenant, multi-vertical. Un magazin de cosmetice și unul de HVAC au categorii diferite. Promptul se compune din `categories` + `intent_aliases` per business → zero „beauty” hardcodat (principiul 9).
3. ⚙️ **Cum (3 prompturi, toate cu antet generat + reguli comune):**

```
@lru_cache(maxsize=256) # memoizat per (business, locale)
def build_agent_system(inp: PromptInputs) -> str:
 return f"_{store_header(inp)}\n_{TOOLS_BLOCK}\n_{SAFETY_RULES}"
_store_header = "Ești consultant pentru {business}, magazin de {vertical}. Vinzi: {categorii}."
_TOOLS_BLOCK = descrierea celor 10 tool-uri + reguli (IDENTIC pe tenanți)
_SAFETY_RULES = interdicția claim-urilor medicale (IDENTIC)
```

| Prompt                          | Când                           | Ce conține                                              |
|---------------------------------|--------------------------------|---------------------------------------------------------|
| <code>build_agent_system</code> | bucla de tool-calling          | antet + tools + safety                                  |
| <code>build_reco_system</code>  | retry proză                    | antet + „recomandă 2-3, prețuri reale” + safety         |
| <code>build_rich_system</code>  | recomandarea structurată (iZi) | antet + <code>_RICH_RULES</code> (consultativ) + safety |

4. 📄 **Cazuri cu exemplu:** | Business | `_store_header` generat | ---|---| | Sole Demo (beauty) | „Ești consultant pentru Sole Demo, magazin de beauty. Vinzi: seruri, creme, parfumuri...” | | un magazin HVAC | „...magazin de hvac.

Vinzi: centrale, aer condiționat, boilere..." || moneda EUR (DomainPack) | „prețul EXACT (euro)" în loc de „(lei)" |

5. 🧠 **De ce** `lru_cache` + **sortare deterministă (PROMPT CACHING)**: OpenAI reduce ~50% costul input-ului dacă **prefixul system e BYTE-IDENTIC** între apeluri ( $\geq 1024$  tokeni). De aceea: categoriile/aliasurile se sortează determinist (`PromptInputs.build`), tot ce e per-tur (mesaj/produse) stă în USER (nu în system), și rezultatul se memoizează per (business, locale). Determinismul prefixului = bani economisiți.

6. 🚫 **Ce-ar fi dacă ai pune produsele clientului în system**: prefixul s-ar schimba la fiecare tur → prompt caching-ul n-ar mai prinde → cost dublu pe input. De aceea system = STATIC, USER = dinamic.

7. 🐞 **Debug**: `_RICH_RULES` (4798 caractere) e cel mai lung prompt — conține toată logica consultativă „model iZi" (intro pe axe, fit\_clause anti-tautologic, education în 3 mișcări, mod detaliu/superlativ, chips).

---

## `tool_definitions.py` — schemele tool-urilor (contractul cu modelul)

---

0. 🔑 **Unde**: `tool_definitions.py:348` (`tool_schemas`). Prefix STATIC (ordine fixă → prompt caching).

1. 🗨️ **Analogie**: meniul cu ce poate „comanda" vânzătorul de la depozit. Fiecare tool are un formular strict (ce argumente, ce tipuri) pe care modelul trebuie să-l completeze corect.

2. ? **De ce scheme separate**: OpenAI function-calling cere scheme JSON. `strict: True` (Structured Outputs) → argumentele vin valide **din construcție** (mai puține retry-uri). `business_id` NU apare în scheme — se ia din `ctx` în tool (izolare, principiul 7).

3. ⚙️ **Cele 10 tool-uri**: | Tool | Argumente cheie | Ce face | |---|---|---| | `search_products` | query, price\_max, category, brand, concerns, sort\_mode, product\_name, variant\_label | caută în catalog | | `get_product_details` | product\_id | detalii + recenzii | | `compare_products` | product\_ids[] | compară 2-3 | | `cart_add` | product\_id, variant\_id, quantity | adaugă în coș | | `checkout_link` | cart\_items[] | creează link de plată | | `reorder` | (fără) | re-comandă ultima | | `subscribe_back_in_stock` | product\_id, variant\_id | notificare la restock | | `faq_lookup` | query | fapt de business | | `check_order` | order\_ref | status comandă | | `request_human` | reason | escaladare |

4. 🚧 **Cazuri**: `enabled_tools(business, route)` decide care tool-uri sunt active per rută (sales vs order). `sort_mode` enum forțează modelul: „cel mai ieftin" → `price_asc`, „cel mai bun" → `rating_desc`.

5. 🧠 **De ce** `strict: True` + `additionalProperties: False`: modelul NU poate inventa argumente noi sau tipuri greșite → mai puține erori de parsare, mai puține retry-uri. Contractul e rigid intenționat.

6. 🚫 **Ce-ar fi dacă** `business_id` **ar fi în schemă**: modelul l-ar putea seta greșit (alt tenant) → breșă de izolare. De aceea vine mereu din `ctx`, niciodată din args-urile modelului.

---

## `usage.py` — contabilitatea de tokeni (observabilitatea de cost)

---

0. 🔑 **Unde**: `usage.py`. Folosit de llm (`record_chat`) + runner (`push` / `pop`).

1. 🗨️ **Analogie**: casa de marcat care numără fiecare bon (apel LLM): câți tokeni, cât a costat, cât s-a economisit din cache.

2. ? **De ce (bug real)**: răspunsurile OpenAI poartă `usage` (tokeni), dar adaptorul nu le citea → `usage_daily.cost_usd` era mereu 0, iar economia din prompt caching invizibilă. Fix: adaptorul raportează fiecare apel aici.

### 3. ⚙️ Cum (elegant, respectă principiul 10):

```
def push() -> (acc, token): return UsageAccumulator(), _current.set(acc) # runner deschide
def record_chat(resp, model): # adaptorul raportează
 acc = _current.get() # ContextVar (izolare per tur)
 acc.add(model, prompt_tokens, completion_tokens, cached_tokens)
 # runner emite UN event llm_usage la final; processor un al doilea (post_turn)
```

4. 🛠️ **Cazuri:** | Situație | Rezultat | |---|---| | apel chat cu usage | tokeni + cost adunați în acumulator | | fake din teste (fără usage) | no-op (nu rupe turul) | | tool-uri concurente ( `gather` ) | toate copiile ContextVar văd ACEEAȘI instanță → se adună corect | | post-tur (summarizer+profil) | al doilea acumulator → `llm_usage phase=post_turn` |

5. 🧠 **De ce ContextVar (nu variabilă globală):** izolare la concurență. `asyncio.gather` copiază contextul, dar toate copiile văd aceeași instanță de acumulator (o mutăm, nu re-legăm var-ul) → tokenii sub-apelurilor concurente se adună corect, fără să se amestece între tururi.

6. ⚡ **Ce-ar fi dacă ai folosi o globală:** două tururi concurente și-ar amesteca tokenii → cost atribuit greșit. ContextVar-ul păstrează izolarea per tur.

## pricing.py — tarifele LLM (sursa unică de prețuri)

0. 📍 **Unde:** `pricing.py:106` ( `cost_for` ). Sursa UNICĂ de prețuri.

1. 🗨️ **Analogie:** lista de tarife a furnizorului (OpenAI): cât costă 1 milion de tokeni per model, cu reducere pentru tokenii serviți din cache.

2. ? **De ce separă tokenii cached:** prompt caching-ul dă discount pe prefixul static. `cost_for` taxează tokenii cached la `cached_input` (≈10% din `input` ) → arată direct economia adusă de prefixul byte-identic.

3. ⚙️ **Cum:**

```
def cost_for(model, prompt_tokens, cached_tokens, completion_tokens):
 cached = min(cached_tokens, prompt_tokens)
 full = prompt_tokens - cached
 return (full*r.input + cached*r.cached_input + completion*r.output) / 1_000_000
```

4. 🛠️ **Cazuri:** | Model | input | cached | output (USD/1M) | |---|---|---|---| | gpt-5.4-mini (agent) | 0.25 | 0.025 | 2.00 | | gpt-5.4-nano (trial) | 0.05 | 0.005 | 0.40 | | embeddings | 0.02 | 0.02 | 0 | | moderation | 0 | 0 | 0 (gratuit) |

5. 🧠 **De ce override din env ( `LLM_PRICING_JSON` ):** tarifele se pot schimba fără redeploy de cod. Best-effort: JSON invalid → WARNING + tarifele implicite (nu rupi botul pentru o setare greșită). `savings_for` calculează banii economisiți de prompt caching (vizibilitate NX-78).

6. ⚡ **Ce-ar fi dacă un model necunoscut ar da cost 0:** ai subestima costul tăcut. De aceea fallback-ul e prudent ( `_DEFAULT` = tariful mini), nu 0.

## Recap Valul 4

```
llm → cusătura OpenAI: retry mărginit + max_completion_tokens=800 + tool loop (max 3)
prompt_builder → prompturi GENERATE din DB (multi-tenant) + lru_cache pt prompt caching (prefix byte-identic)
tool_definitions → schemele celor 10 tool-uri (strict, business_id din ctx nu din args)
usage → contabilitate tokeni per tur (ContextVar, izolare la concurență)
pricing → tarife USD/1M (sursă unică, cached separat, override din env)
```

**Firul roșu:** LLM-ul e izolat la o singură margine (llm.py); promptul e generat din DB (nu hardcodat) și optimizat pentru prompt caching (prefix static); fiecare token e contabilizat exact.

## VALUL 5 — Tools (mâinile agentului)

Tool-urile sunt „mâinile” pe care le folosește vânzătorul (modelul): caută în catalog, adaugă în coș, creează linkuri, verifică comenzi. Toate sunt **cod determinist**, scoped pe `business_id` (din `ctx`, NU din args — izolare, principiul 7).

### `base.py` — framework-ul de tool-uri (contractul comun)

0. 📌 **Unde:** `base.py`. `ToolResult` + `register` + `run_tool` + `enabled_tools`.

1. 🗨️ **Analogie:** regulamentul depozitului: fiecare „unealtă” are aceeași formă de comandă și același formular de răspuns, ca vânzătorul să știe la ce să se aștepte.

2. ❓ **De ce:** un contract uniform (`async def tool(ctx, deps, args) -> ToolResult`) ca toate tool-urile să fie interschimbabile. Agentul le cheamă la fel, indiferent care.

3. ⚙️ **Cum:**

```
@dataclass
class ToolResult:
 ok: bool
 products: list = [] # COMPLETE → ctx.retrieval + validator
 llm_view: str = "" # COMPACT → model (≤6×8, fără PII)
 links: list = [] # linkuri generate de bot (checkout) → validator le acceptă
 prices: list = [] # sume din DB (total comandă) → validator le acceptă
 state_patch: dict = {} # mutație de state (cart) → processor persistă
 relevance: Relevance = None # semnal off-category → compose

@register("search_products") # decorator → TOOL_REGISTRY
async def run_tool(ctx, deps, name, args):
 fn = TOOL_REGISTRY.get(name)
 if fn is None: return ToolResult(ok=False) # tool inexistent → nu crapă
 try: return await fn(ctx, deps, args)
 except: return ToolResult(ok=False) # tool eșuat → degradare grațioasă (P6)
```

4. 🚧 **Cazuri** — `enabled_tools` (**toolset per rută**): | Rută | Tool-uri active | |---|---| | sales | search, details, compare, cart\_add, checkout, reorder, back\_in\_stock, faq | | order | check\_order, faq\_lookup (DOAR — focus pe status) | | opt-in | request\_human (doar dacă tenantul îl activează în settings) |
5. 🧠 **De ce** `ToolResult` **separă** `products` (**complete**) de `llm_view` (**compact**): validatorul are nevoie de produsele COMPLETE (ca să verifice prețuri/linkuri), dar modelul primește o vedere COMPACTĂ (6×8) ca să nu-i umpli contextul cu tokeni. `links` / `prices` = fapte grounded de bot (checkout/total) pe care validatorul le acceptă în plus.
6. 💣 **Ce-ar fi dacă** `run_tool` **n-ar prinde excepțiile**: un tool care crapă (DB jos) ar rupe tot turul. Guard-ul `try/except` → `ToolResult(ok=False)` → agentul continuă cu ce are (P6).
7. 🐛 **Debug**: `grep tool_call` (name, ok, args whitelisted, n\_results, latency, error).

## `catalog_tools.py` — motorul de căutare hibridă (cel mai complex tool) 🔍

0. 📍 **Unde**: `catalog_tools.py:382` ( `search_products_tool` ). Diagrama 5. **Scrie**: `ctx.state_patch["active_search"]` (sesiunea de paginare).
1. 🧠 **Analogie**: un bibliotecar expert care caută în DOUĂ feluri simultan: după cuvinte (lexical, „găsește cărți cu «vitamina C» în titlu”) ȘI după înțeles (semantic, „găsește cărți despre luminozitatea pielii”), apoi combină cele două liste inteligent.
2. ? **De ce hibrid**: doar lexical ratează sinonime („luminozitate” vs „glow”). Doar semantic ratează nume exacte. Împreună = recall bun. Plus: relaxare progresivă (dacă nu găsești nimic cu toate filtrele, relaxezi), diversificare (nu top-N clone), și sesiuni de paginare (zero cost la „mai arată-mi”).
3. ⚙️ **Cum (fluxul complet)**:

```
a = SearchArgs(**args) # validare Pydantic
concern_keys = map_concerns(domain_pack, a.concerns) # „ten gras” → „oily”
CONTINUARE sesiune: aceleași filtre (fp) + pool stocat → pagina următoare ($0 embed)
if sess["fp"] == fp and sess["pool"]: return continue_search_session(...)
SESIUNE NOUĂ:
ladder = _relax_ladder(...) # trepte de relaxare
query_vec = await llm.embed([a.query]) # UN apel embed (sau None → lexical-only)
for f in ladder: # relaxează până iese ceva
 lexical = await search_products_lexical(...) # FTS + pg_trgm
 vector = await search_products_semantic(...) # pgvector HNSW
 ranked = fuse_candidates(lexical, vector, weights=rank_weights) # RRF + blended rank
 if ranked: break
ranked_final = diversify_pool(ranked, limit) # terțe preț + max 2/brand
pool_ids = [p["id"] for p in ranked_final][:24] # sesiunea (ref-uri)
products = first_page(pool_ids) # prima pagină (unseen-dedup)
```

4. 🚧 **Cazuri cu exemplu (fiecare mecanism)**: | Mecanism | Funcție | Exemplu | |---|---|---| | Map concerns | `map_concerns` | „ten gras” → filtru real `oily` în DB | | Relax ladder | `_relax_ladder` | „ser vitamina C sub 50 lei ten mixt” — nimic → relaxează concern → relaxează category | | Preț DUR | `_relax_ladder` | cu `search_sort_mode_enabled`, prețul NU se relaxează (bug „sub 80 → 149” reparat) | | Fuse | `fuse_candidates` | lexical

+ semantic → RRF + blended (rating shrunk + reducere) || Diversify | `diversify_pool` | prima pagină = ieftin/mediu/scump + max 2/brand (nu 6 clone) || Sesiune | `continue_search_session` | „mai arată-mi” → pagina 2 din pool, **\$0 embed** || Brand not found | — | ceri brand X, catalog n-are → „nu lucrăm cu X” (nu prezenta alt brand) || Produs numit inexistent | `_named_product_found` | „aveți Hidra Boost Ultra?” — nu există → „nu ca atare, dar astea-s alternative” |

**5. 🧠 De ce relaxarea NU atinge prețul ( `search_sort_mode_enabled` ):** bug real — „ser sub 80 lei” fără rezultate → vechea relaxare scotea bound-ul de preț → întorcea un 149.99 (exact ce n-a cerut). Acum prețul + stocul sunt DURE; se relaxează doar softul (concern → category). Onestitate: dacă a relaxat, `llm_view` conține o notă „relaxat: nu e match exact” → agentul e sincer.

**5b. 🧠 De ce sesiuni de căutare ( `continue_search_session` ):** „mai arată-mi” nu trebuie să re-caute (cost embed + risc de drift). Pool-ul (24 id-uri) + cursor se stochează în state (ref-uri, P8). Fingerprint-ul ( `fp` ) al filtrelor decide: aceleași filtre → paginează; filtre schimbate → sesiune nouă.

**6. ✂ Ce-ar fi dacă pool-ul sesiunii ar fi dedup-uit de produsele afișate:** produsele deja arătate ar fi excluse PERMANENT + epuizare falsă (bug review #1). De aceea pool-ul e ordinea completă; dedup-ul vs displayed se face la fiecare pagină ( `_next_page` ), nu la semănare.

**7. 🐞 Debug:** `grep product_search` (mode=semantic/lexical, relaxed, relax\_depth, top\_cosine\_distance, diversified), `search_session` (new/page/exhausted), `named_product_not_found` .

**8. ✅ Test:** Client caută „ser cu retinol sub 60 lei”, catalogul n-are retinol sub 60. Ce trepte de relaxare se aplică (cu `search_sort_mode_enabled` ON)? Ce primește clientul?

## Celelalte 2 tool-uri de catalog

- `get_product_details_tool` (:637) — detalii + recenzii + variante (id+preț real).
- `compare_products_tool` (:649) — 2-3 produse (< 2 valide → eroare).

---

## `commerce_tools.py` — tool-urile care SCRIU (coș, checkout, restock)

---

**0. 📌 Unde:** `commerce_tools.py`. WRITE tools. Diagrama „buclo de bani”.

**1. 🛒 Analogie:** casa de marcat + coșul de cumpărături. Aici se face banul: adaugi produse, creezi linkul de plată, te abonezi la „revine pe stoc”.

**2. ? De ce validare dură contra catalogului:** nu linkuiești/adaugi produse inexistente. Fiecare produs + variantă se verifică în DB înainte.

**3. ⚙ Cele 4 tool-uri:** | Tool | Ce face | Grounding | |--|--|--| | `checkout_link` | creează link `?ref=turn_id` + scrie `checkout_links` | `prices=[total]` + `links=[url]` → validator || `cart_add` | acumulează coșul în `state.cart` (merge pe product+variant) | `state_patch={"cart":...}` || `reorder` | propune re-comanda ultimei comenzi | `prices` din `orders` (nu catalog) || `subscribe_back_in_stock` | abonare la restock (idempotent) | citit de proactiv (NX-70) |

**4. 📄 Cazuri cu exemplu — `checkout_link` :** Situație | Rezultat | |--|--|--| | coș valid, base URL configurat | link `?ref=turn_id` + total grounded || base URL gol | „linkul nu e configurat — dar coșul FUNCȚIONEAZĂ (`cart_add`)” (nu refuz total) || `variant_id` fabricat („nuața 03”) | `variant_not_found` (nu linkuim variantă inexistentă) || toate produsele inactive | `no_valid_products` |

5. 🧠 **De ce** `ref=turn_id` (**idempotent per tur**): dacă turul se rejucă, `ref_code = turn_id` → nu creezi 2 linkuri. Iar `ref` leagă linkul de conversație → când vine comanda (webhook), o atribui (bucla de atribuire).
- 5b. 🧠 **De ce** `cart_add` **merge pe (product\_id, variant\_id)**: re-apel pe același produs → crește cantitatea, nu duplică linia. Coșul se acumulează între mesaje (state), până la checkout.
6. 💣 **Ce-ar fi dacă** `checkout_link` **gol ar da un** `llm_view` **gol**: modelul ar generaliza refuzul („nu pot nici adăuga în coș”), deși `cart_add` nu depinde de URL (bug real NX-137). De aceea `llm_view` e instructiv: spune explicit ce funcționează.
7. 🐞 **Debug**: `grep checkout_link_created` (items, value), `cart_updated`, `variant_rejected`, `back_in_stock_subscribed`.
- 

## `orders_tools.py` — `check_order` (status comandă, izolare dură)

---

0. 🔑 **Unde**: `orders_tools.py:61`. Read-only.

1. 🗨️ **Analogie**: ghișeu „unde e coletul meu?”. Îți arată statusul comenzii TALE — dar nu poți vedea comanda altcuiva nici ghicind numărul.

2. ❓ **De ce izolare dură**: lookup-ul e scoped pe `ctx.contact.id` (din `channel_identities`, NU din args). Un `order_ref` doar îngustează; nu poți vedea comanda altui client.

3. ⚙️ **Cum**:

```
if web_unidentified(ctx): return login_required # web anonim → login (nu „n-am găsit”)
customer_ref = ctx.verified_customer_ref # web cu login → comenzile reale
orders = await get_orders_status(business_id, external_id=order_ref,
 contact_id=None if customer_ref else ctx.contact.id, external_customer_ref=customer_ref)
if not orders: return no_orders_message(ctx) # onest, conștient de canal
return ToolResult(prices=_order_totals(orders), llm_view=_orders_view(orders)) # fără PII
```

4. 📄 **Cazuri**: | Situație | Rezultat | |---|---| | WhatsApp, „status ORD-123” | caută pe contul lui (contact\_id) → status + AWB + ETA | | Web anonim | mesaj de login (nu poate avea comenzi legate de contact throwaway) | | Web cu login verificat | caută pe `customer_ref` (comenzile reale din eshop) | | comandă inexistentă / a altcuiva | `not_found` IDENTIC (nu divulgă existența) |

5. 🧠 **De ce** `not_found` **identic pentru „inexistent” și „a altcuiva”**: izolare — nu vrei să afle un atacator că „ORD-123 există dar nu e a ta” (ar putea enumera comenzi). Răspuns identic = zero informație scursă.

6. 💣 **Ce-ar fi dacă** `order_ref` **din args ar fi cheia de lookup**: un client ar putea ghici numere și vedea comenzi străine. De aceea cheia vine din `ctx` (verificat), `order_ref` doar filtrează.

7. 🐞 **Debug**: `grep order_lookup_gated` (web anonim). Fără PII în `llm_view` (status/AWB/total — `orders` n-are telefon/adresă).

---

## faq\_tools.py — faq\_lookup (cunoștințe în mijlocul vânzării)

📌 [faq\\_tools.py:32](#). 🗂️ dosarul de reguli pe care vânzătorul îl consultă când clientul întreabă o politică („pot plăti ramburs?") în mijlocul recomandării. ⚙️ refolosește ACELAȘI `semantic_lookup` ca stratul gratuit FAQ, dar cu prag mai relaxat ( `faq_tau_tool=0.66` — agentul parafrazează oricum). Miss → „verific cu un coleg" (nu inventa reguli). 🧠 `business_id / locale` din `ctx` (P7, P11), nu din args.

## handoff\_tools.py — request\_human + notify\_operator

📌 [handoff\\_tools.py:54](#). 🗂️ butonul „cheamă un coleg" al vânzătorului. ⚙️ `set_handoff` (botul tace turul următor) + `notify_operator` (POST webhook, **fără PII** — doar slug + conversation\_id + reason). 📧 pe canale fără operator (web) → no-op, modelul continuă să asiste (nu promiți un coleg inexistent). 🧠 OPT-IN per business ( `enabled_tools` ); `reason` ajunge la operator, NU în event (P12: token fix `agent_request` ).

## taxonomy.py — map\_concerns (traducător de nevoi)

0. 📌 **Unde:** [taxonomy.py:59](#) ( `map_concerns` ). **Pur, zero LLM.**

1. 🗂️ **Analogie:** un traducător care transformă vorba clientului („ten gras") în termenul din baza de date („oily"). Fără el, filtrul n-ar prinde nimic.

2. ❓ **De ce:** clientul zice „ten gras", dar în `products.attributes->'concerns'` scrie „oily". Fără mapare, filtrul jsonb `?|` n-ar potrivi nimic → rezultat gol fals.

3. ⚙️ **Cum:**

```
table = domain_pack.concern_map # din DomainPack (per-vertical, nu hardcodat)
out = [table[_norm(c)] for c in raw if _norm(c) in table] # necunoscutele se IGNORĂ
return sorted(dict.fromkeys(out)) # unice, ordine stabilă
```

4. 📧 **Cazuri:** | Input client | `concern_map` (beauty) | Output | `---|---|---` | „ten gras" | oily | `["oily"]` | | „piele sensibilă" | sensitive | `["sensitive"]` | | „ceva verde" (necunoscut) | — | `[]` (ignorat, nu filtru fals) | | DomainPack lipsă | — | `[]` (fără crash, fără filtru) |

5. 🧠 **De ce necunoscutele se IGNORĂ (nu filtru fals):** mai bine zero filtru decât unul greșit care golește rezultatul (P6 indirect). Și maparea vine din DomainPack (per-vertical) — HVAC „zgomotos" → „low\_noise" merge fără deploy, nu doar beauty.

## Recap Valul 5

```

base → contractul comun (ToolResult) + registry + run_tool (izolare + degradare)
catalog_tools → căutare HIBRIDĂ (lexical+semantic) + relax ladder + diversify + sesiuni de paginare
commerce_tools → WRITE: checkout (ref=turn_id) + cart_add (merge) + reorder + back_in_stock
orders_tools → check_order (izolare dură pe contact_id, not_found identic)
faq_tools → faq_lookup (cunoștințe în vânzare, prag relaxat)
handoff_tools → request_human + notify_operator (fără PII, opt-in)
taxonomy → map_concerns (nevoia clientului → cheie DB, din DomainPack)

```

**Firul roșu:** tool-uri deterministe, scoped pe `business_id` din `ctx` (nu din `args` = izolare); `products` complete pentru validator + `llm_view` compact pentru model; grounding prin `links` / `prices`; degradare grațioasă la orice eșec.

## VALUL 6 — DB (cum vorbește codul cu Postgres)

Stratul de date. **Regula de aur (principiul 7):** fiecare query are EXPLICIT `where business_id = $1` (mecanism primar de izolare); RLS pe rolul `bot_runtime` e plasa. `conn` e mereu tenant-scoped ( `tenant_conn` ).

`connection.py` (pool-uri + RLS + izolare) e explicat COMPLET în [MASTERCLASS-RO.md](#) cap. 15. Aici: query-urile cu logică reală (SQL de căutare, RRF, optimistic lock, price-check, claim) în format complet; CRUD-urile simple, grupate.

### `catalog.py` — SQL-ul de căutare (inima retrievalului) 🔍

0. 📌 **Unde:** `catalog.py`. Read-only. Folosit de `catalog_tools`.

1. 🤖 **Analogie:** motorul de căutare al bibliotecii — știe să caute după titlu (nume), după raft (categorie), după preț, și după „despre ce e” (semantic). Cu reguli istețe: prețul real e pe variantă, un 5★-cu-1-recenzie nu bate un 4.6★-cu-200.

2. ❓ **De ce e complex:** căutarea trebuie să fie corectă (prețul exact pe care-l vede clientul), robustă (typo/SKU), și inteligentă (rating cold-start, reducere). Plus izolare pe tenant fără excepție.

3. ⚙️ **Cum — două concepte SQL cheie:**

**Prețul efectiv** ( `_EFFECTIVE_PRICE` , `catalog.py:21`):

```
coalesce(vp.price, p.sale_price, p.price) -- min(variantă) → sale_price → price
```

De ce? Prețul REAL e pe variantă (50ml vs 100ml). Validatorul trebuie să vadă exact ce vede clientul.

**Rating shrunk (Bayesian)** ( `_SHRUNK_RATING` , `catalog.py:25`):

```
(n*rating + 30*4.0) / (n + 30) -- prior C=30 spre media 4.0
```

De ce? Un 5.0★ cu 1 recenzie devine ~4.03 (tras spre medie); un 4.6★ cu 200 rămâne ~4.59. Cold-start reparat.

4. 🚩 **Cele 3 căutări (+ 4 helper-e), cu rol:** | Funcție | Ce face | |---|---| | `search_products_lexical` (:248) | FTS (`websearch_to_tsquery`) + `pg_trgm` (typo/SKU) → `rang = ts_rank_cd + similarity` | | `search_products_semantic` (:536) | `pgvector` `embedding <=> query` (cosine) + aceleași filtre dure | | `get_products_by_ids` (:368) | re-fetch după id, **ordinea cerută păstrată** (`array_position`) | | `search_cheaper_than` (:394) | „mai ieftin” — strict sub preț, aceeași categorie, doar în stoc | | `get_complementary_products` (:429) | cross-sell: același brand SAU concern comun, categorie DIFERITĂ | | `sibling_categories` (:482) | categorii surori (chips de închidere) | | `list_category_slugs / names` | groundarea triajului + promptului |
5. 🧠 **De ce filtre DURE parametrizate ( `placeholder` ):** SQL injection-safe. Filtrele se combină cu AND; brandul = filtru DUR și pe vector (un brand inexistent → zero, NU produse semantic-apropiate de la alt brand — bug „avem Chanel”). `sort_mode` = `allowlist ( _VALID_SORT )`, nu param bindabil (e structural).
- 5b. 🧠 **De ce `array_position` la `get_products_by_ids`:** deixis ordinal — „compară primele două” / „a doua” trebuie să rezolve produsul corect în ORDINEA afișată, nu în ordinea DB.
6. 💣 **Ce-ar fi dacă prețul ar fi `p.price` (nu efectiv):** validatorul ar vedea 199 (`products.price`) când clientul vede 149 (varianta) → ar respinge un preț corect ca „inventat”. Prețul efectiv = paritate client↔validator.
7. 🐛 **Debug:** SQL-ul se construiește dinamic (condiții + params). La un bug de căutare, loghează `sql` + `params` și rulează-l direct pe DB. `search_tsv` = coloană tsvector (migrația 015).
8. ✅ **Test:** de ce `search_cheaper_than` exclude produsele deja afișate (`p.id <> all($2)`)?

---

## `fusion.py` — RRF + blended rank (combinarea căutărilor)

---

0. 🔑 **Unde:** `fusion.py:215` ( `fuse_candidates` ). **PUR** (zero DB, zero LLM). Folosit de `catalog_tools`.
1. 🤖 **Analogie:** doi bibliotecari caută separat (unul pe cuvinte, unul pe înțeles) și-ți dau două liste. `fusion` le combină inteligent: o carte care apare în AMBELE liste urcă în top.
2. ? **De ce:** ai două liste ranguite (lexical + semantic). Trebuie o singură listă. RRF (Reciprocal Rank Fusion) = standardul: un produs în ambele liste acumulează scor din amândouă.
3. ⚙️ **Cum — RRF + blended:**

```
def rrf_scores(lexical, vector):
 # RRF: $\sum 1/(k + rang)$, k=60
 scores[pid] += 1.0 / (60 + rank) # în ambele liste → sumă

def blended_rerank(products, scores, weights):
 # ARCH-2026 P0: scor blended
 score = w["relevance"]*rrf_norm + w["rating"]*rating_norm # relevanță (1.0) primară
 + w["availability"]*in_stock + w["sale"]*on_sale + w["concern"]*overlap
```

4. 🚩 **Cazuri:** | `sort_mode` | Ce se aplică | |---|---| | relevance (blended ON) | RRF + rating shrunk + availability + sale + concern (ponderi din DomainPack) | | relevance (blended OFF) | `deterministic_rerank` (RRF pur, rating doar pe tie) | | price\_asc/desc, rating\_desc | `_merge_by_sort` (re-sort determinist, paritate SQL) |
5. 🧠 **De ce blended (ARCH-2026 P0):** cu RRF pur, ratingul conta DOAR la egalitate (≈niciodată) → un produs 4.6★×148 se îngropa sub unul 4.4★×28. Blended dă social-proof-ului voce în scorul PRIMAR, dar relevanța rămâne dominantă (pondere 1.0 + normalizare min-max pe set). **Modelul NU clasează niciodată — codul o face.**
- 5b. 🧠 **De ce re-calculezi `_shrunk_rating` în Python (nu doar SQL):** paritate. Re-sortul pe `rating_desc` trebuie să reproducă EXACT ordinea SQL (tie-break-uri identice) → altfel paginarea ar drifta.

6. 🚨 **Ce-ar fi dacă un semnal ar avea pondere prea mare (ex. sale=1.0):** produsele la reducere ar domina, împingând relevanța jos → clientul primește reduceri irelevante. Ponderile mici (0.08-0.35) țin relevanța rege.

---

## conversations.py — optimistic lock pe state

---

0. 🔑 **Unde:** `conversations.py:108` (`patch_conversation_state`).

1. 🗨️ **Analogie:** un caiet partajat cu un număr de versiune pe copertă. Când scrii, verifici „e încă versiunea pe care am citit-o?”. Dacă altcineva a scris între timp, refuzi (nu suprascrii orbește).

2. ❓ **De ce:** `conversations.state` (memoria agentului) e scris de Sender. Două tururi concurente ar putea suprascrie unul pe altul. Optimistic lock previne.

3. ⚙️ **Cum:**

```
update conversations set state=$4, state_version = state_version + 1
where business_id=$1 and id=$2 and state_version = $3 # DOAR dacă versiunea e cea citită
returning state_version
if new_version is None: raise StateConflict # altă scriere a intervenit
```

4. 🚨 **Cazuri:** | Situație | Rezultat | |---|---| | versiunea din DB = cea așteptată | UPDATE prinde → versiune++ | | altă scriere a intervenit | zero rânduri → `StateConflict` (turul reia, nu suprascrie) | | primul mesaj al unui contact nou (race) | `get_or_create_conversation` cu `ON CONFLICT` → o singură conversație |

5. 🧠 **De ce lock per conversație (consumer) + optimistic lock (aici) = DOUĂ apărări:** lock-ul serializează tururile ÎNAINTE de procesare (evită concurența); optimistic lock-ul e plasa dacă lock-ul a eșuat (Redis jos, fail-open). Bretele + curea.

5b. 🧠 **De ce** `set_handoff / set_conversation_locale` **NU ating** `state_version`: ca să nu intre în conflict cu patch-ul Sender-ului din același tur. Scriu alte coloane, nu state-ul.

6. 🚨 **Ce-ar fi fără optimistic lock:** două tururi concurente → ultimul suprascrie tot state-ul primului (pierzi produse afișate, constrângeri). „Botul uită” din cauza unei curse.

---

## semantic\_cache.py — cache + price-check self-healing

---

0. 🔑 **Unde:** `semantic_cache.py`. Folosit de `cache_stage` + `_cache_writeback`.

1. 🗨️ **Analogie:** un caiet cu răspunsuri, dar cu „dată de expirare” pe fiecare — iar pentru răspunsurile cu prețuri, verifici că prețul e încă valabil înainte să-l servești.

2. ❓ **De ce price-check:** un răspuns cu produse cache-uit poate avea prețuri vechi. Înainte să-l servești, verifici prețurile curente. Diferit → nu servi preț învechit.

3. ⚙️ **Cum:**

```
def exact_lookup(hash): ... # L1: O(1) pe canonical_hash
def semantic_lookup(embedding, embedding_model): ... # L2: cosine, filtru pe MODEL (NX-124a)
def current_prices(product_ids): return {id: eff_price} # price-check: prețurile curente
def purge_by_product(product_id): ... # invalidare la scoatere din stoc
```

4.  **Cazuri:** | Situație | Rezultat | |---|---| | hit static (fără produse) | servit direct | | hit dynamic, preț neschimbat | servit (price-check trece) | | hit dynamic, preț schimbat | `delete_entry` + tratat ca miss (regenerează) | | vectori de alt model embed | filtru `embedding_model` → nu amestecă spațiile |

5.  **De ce filtru OBLIGATORIU pe `embedding_model` (L2):** ordonarea cosine pe vectori din alt model (dimensiune/spațiu diferit) e ZGOMOT. Un upgrade de embeddings nu mai amestecă spațiile (principiul 11).

6.  **Ce-ar fi fără price-check:** un client vede prețul de acum 3 zile (produsul s-a scumpit) → comandă la preț greșit. Price-check-ul = self-healing (cache-ul se auto-invalidază).

## `outbox.py` — claim + dispatch (coada de ieșire)

0.  **Unde:** `outbox.py`. Scris de Sender, citit de dispatcher.

1.  **Analogie:** cutia poștală cu bonuri numerotate. Poștașul (dispatcher) ia bonurile scadente, dar un bon pe care-l ia altcineva e „blocat” (nu-l iei tu). Dacă poștașul dispăre cu bonul, bonul re apare după un timeout.

2.  **De ce:** decuplare (Sender scrie, dispatcher trimite) + idempotență + paralelism sigur + self-healing.

3.  **Cum — 3 mecanisme:**

```
def enqueue_outbox(idempotency_key): # UNIQUE → re-enqueue = no-op
 on conflict (business_id, idempotency_key) do nothing
def claim_due(): # FOR UPDATE SKIP LOCKED
 for update of o2 skip locked # 2 dispatchere → rânduri diferite
 set next_attempt_at = now() + 120s # visibility timeout (self-healing)
def mark_failed(attempts): # backoff [5,30,120,300]s
 if attempts >= 6: status='dead' # vizibil, nu pierdut tăcut
```

4.  **Cazuri:** | Situație | Rezultat | |---|---| | re-enqueue același `turn:0` | `ON CONFLICT DO NOTHING` → nu dublează | | 2 dispatchere claim simultan | `SKIP LOCKED` → fiecare ia rânduri diferite | | dispatcher moare între claim și mark | `next_attempt_at` în viitor → redevine scadent după 120s | | 6 eșecuri consecutive | status `dead` (vizibil în coadă, nu pierdut) |

5.  **De ce `FOR UPDATE SKIP LOCKED` (nu doar `SELECT`):** paralelism sigur. `SKIP LOCKED` sare rândurile pe care alt worker le ține → zero dublă trimitere, fără să blochezi. Visibility timeout = reaper implicit (fără coloană separată).

6.  **Ce-ar fi fără idempotency\_key UNIQUE:** un retry de pipeline ar crea un al doilea mesaj → clientul primește răspunsul de 2 ori.

7.  **Debug:** `status='dead'` = mesaje ne-livrabile (verifică `last_error`). `business_ids_with_due_outbox` rulează pe `admin_conn` (control plane — dispatcher-ul nu știe dinainte ce tenanți au de trimis).

# CRUD-urile simple (grupate)

Restul query-urilor sunt wrappere SQL directe, scoped pe `business_id` . Le caracterizez pe grupe:

| Fișier                                    | Ce face                                                                    | Funcții cheie                                                                                                   |
|-------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>channels.py</code>                  | canal → business (SINGURUL lookup pre-tenant, pe <code>admin_conn</code> ) | <code>resolve_channel</code>                                                                                    |
| <code>businesses.py</code>                | încarcă tenantul + <code>DomainPack</code>                                 | <code>load_business</code> , <code>get_data_version</code>                                                      |
| <code>contacts.py</code>                  | contacte + identity resolution (PII în <code>channel_identities</code> )   | <code>get_or_create_contact</code> , <code>block_contact</code> , <code>update_contact_profile_and_score</code> |
| <code>messages.py</code><br>[partiționat] | mesajele (istoric, sumar)                                                  | <code>insert_message</code> , <code>get_recent_messages</code> , <code>count_messages</code>                    |
| <code>inbound_dedupe.py</code>            | dedupe L2 (claim-or-resume NX-86)                                          | <code>claim_inbound</code> , <code>mark_inbound_completed</code>                                                |
| <code>message_status.py</code>            | delivered/read/failed → <code>messages.status</code>                       | <code>record_status_event</code>                                                                                |
| <code>aliases.py</code>                   | alias lookup (strat gratuit)                                               | <code>lookup_alias</code> , <code>get_faq_answer</code>                                                         |
| <code>faqs.py</code>                      | FAQ semantic lookup                                                        | <code>semantic_lookup</code>                                                                                    |
| <code>summaries.py</code>                 | rezumate rolling                                                           | <code>get_summary_for_context</code> , <code>insert_conversation_summary</code>                                 |
| <code>commerce.py</code>                  | comenzi + checkout + <code>back_in_stock</code>                            | <code>get_orders_status</code> , <code>create_checkout_link</code> , <code>subscribe_back_in_stock</code>       |
| <code>proactive.py</code>                 | joburi proactive ( <code>claim_due</code> FOR UPDATE SKIP LOCKED)          | <code>claim_due_jobs</code> , <code>mark_*</code>                                                               |
| <code>analytics.py</code>                 | event-uri (append-only, INSERT)                                            | <code>insert_events</code>                                                                                      |
| <code>usage.py</code>                     | rollup zilnic (sursa de facturare)                                         | <code>rollup</code>                                                                                             |
| <code>wa_templates.py</code>              | template-uri WhatsApp aprobate                                             | lookup template                                                                                                 |
| <code>gdpr.py</code>                      | ștergere/export (security definer)                                         | <code>gdpr_erase_contact</code>                                                                                 |

**Firul roșu al tuturor:** `where business_id = $1` explicit + RLS ca plasă; idempotență prin UNIQUE; hot tables (`messages` , `analytics_events` ) partiționate pe lună; PII DOAR în `channel_identities` .

## Recap Valul 6

catalog

→ SQL de căutare: preț efectiv (variantă) + rating shrunk + FTS/trgm/pgvector

fusion

→ RRF + blended rank (relevanță primară + social proof), codul clasează, nu modelul

conversations

→ optimistic lock pe `state_version` (a doua apărare peste lock-ul de conversație)

semantic\_cache

→ L1/L2 + price-check self-healing + filtru pe `embedding_model`

outbox

→ enqueue idempotent + claim (SKIP LOCKED + visibility timeout) + backoff → dead

CRUD-uri

→ wrappere scoped pe `business_id`, PII în `channel_identities`, partiționare hot tables

**Firul roșu:** izolare `business_id` + RLS peste tot; determinism (tie-break-uri stabile → cache/golden stabile); self-healing (optimistic lock, price-check, visibility timeout); idempotență prin UNIQUE.

---

## VALUL 7 — Intrare (marginile care primesc de la lume)

---

Cum intră un mesaj în sistem, în siguranță și rapid. `webhook/app.py` (endpointurile) e explicat COMPLET în [MASTERCLASS-RO.md](#) cap. 5; aici: securitatea (semnătură), parserul, plasa anti-OOM, comenzile și backbone-ul Redis.

---

### `signature.py` — verificarea semnăturii HMAC

---

0. 🔑 **Unde:** `signature.py`. Folosit de `webhook/app` la fiecare POST.

1. 🗂️ **Analogie:** un sigiliu de ceară pe scrisoare. Doar cine cunoaște secretul poate face un sigiliu valid peste EXACT acest conținut. Verifici sigiliul înainte să deschizi plicul.

2. ❓ **De ce peste corpul BRUT:** dacă ai reserializa JSON-ul, ai schimba octeții → semnătura n-ar mai potrivi. Verifici `raw`, nu `payload`. Principiul 7: nu ai încredere în input extern.

3. ⚙️ **Cum:**

```
expected = hmac.new(secret, raw_body, sha256).hexdigest()
return hmac.compare_digest(expected, received) # timp CONSTANT (anti timing-attack)
```

4. 📧 **Cazuri:** | Situație | Rezultat | |---|---| | semnătură corectă | True → procesează | | semnătură greșită / lipsă | False → 403 (fail-closed) | | fără secret configurat | False (preferăm să respingem) |

5. 🧠 **De ce `compare_digest` (nu `==`):** comparația normală se oprește la primul octet diferit → un atacator ar putea măsura timpul și ghici semnătura octet cu octet (timing attack). `compare_digest` compară în timp constant.

5b. 🧠 **De ce HMAC și nu un secret-header static:** un secret static scurs din loguri/proxy autentifică orice. Cu HMAC, un secret scurs NU ajută — atacatorul tot nu poate semna un corp pe care nu-l cunoaște.

6. 💣 **Ce-ar fi fără verificare:** oricine ar putea trimite mesaje false → botul ar răspunde la spam / ar procesa comenzi false. Semnătura e poarta de autenticitate.

---

### `meta.py` — parserul payload-ului Meta

---

0. 🔑 **Unde:** `meta.py:50` ( `parse_webhook` ). **Fără DB** (webhook subțire).

1. 🗂️ **Analogie:** un traducător care ia formularul complicat de la Meta și scoate din el doar mesajele, în forma noastră simplă (envelope neutru).

2. ❓ **De ce:** structura Meta e imbricată ( `entry[].changes[].value.messages[]` ). O aplatizezi într-o listă plată de `InboundEvent`. NU atinge DB (asta e treaba workerului) → webhook rămâne <50ms.

3. ⚙️ **Cum:**

```
for entry in payload["entry"]:
 for change in entry["changes"]:
 for msg in value.get("messages", []): # ignoră statuses (parse_statuses separat)
 body, media_id = _extract_body(msg, content_type) # text/image/button/interactive
 if len(body) > INBOUND_BODY_MAX: body = body[:2000] # trunchiere la 2000
 events.append(InboundEvent(channel_kind="whatsapp", ...))
```

4.  **Cazuri:** | Tip mesaj | Ce extrage | |---|---| | text | `text.body` | | image/audio/... | caption + `media.id` (pentru Vision) | | button | `button.text` | | interactive | titlul reply-ului (buton/listă) | | mesaj fără id/expeditor | sărit (inutilizabil) | | payload doar cu statuses | listă goală |
5.  **De ce parsare DEFENSIVĂ (chei lipsă → sărim):** Meta poate trimite structuri parțiale/neașteptate. Un `change` malformat nu trebuie să crape tot webhook-ul → sari peste el, nu crăpa.
6.  **Ce-ar fi dacă ai atinge DB aici:** webhook-ul ar depăși 50ms → Meta ar face retry → dublă procesare. DB-ul trăiește în worker, nu la margine.

## `body_limit.py` — plasa anti-OOM

 `body_limit.py:16` ( `enforce_body_cap` ).  portarul care cântărește coletul înainte să-l lase înăuntru.  pe un VPS mic fără swap, un POST de mulți MB bufferizat înainte de verificare poate OOM-ui procesul (pică TOȚI tenanții).  verifică `Content-Length` → 413 dacă lipsește/prea mare; apoi **stream-limit** prinde un `Content-Length` mincinos (declară mic, trimite mult).  respingi ÎNAINTE de a citi corpul integral → nu bufferizezi niciodată zeci de MB.

## `orders.py` — webhook comenzi → atribuire (bucla de bani)

0.  **Unde:** `orders.py:64` ( `process_order` ). Rulează în WORKER (DB writes).
1.  **Analogie:** contabilul care primește o comandă de la magazin și verifică „a venit prin recomandarea botului?”. Dacă comanda poartă eticheta botului ( `?ref=` ), o atribuie botului (bani făcuți).
2.  **De ce:** închide bucla de bani. Botul creează un link `?ref=turn_id` ; când clientul cumpără prin el, comanda vine cu acel `ref` → o atribui („assisted”) → dovada valorii botului.
3.  **Cum:**

```
o = OrderIn(**order) # validare Pydantic (neutru de platformă)
if o.ref:
 link = await get_checkout_link_by_ref(o.ref) # match pe checkout_links
 if link: attribution = "assisted"; contact_id = link["contact_id"]
row = await upsert_order(..., attribution=attribution) # idempotent pe (business, external_id)
if inserted and o.items: insert_order_items(...) # items DOAR la insert nou (nu dubla)
if checkout_link_id: mark_checkout_converted(...)
```

4.  **Cazuri:** | Situație | Attribution | |---|---| | comandă cu `?ref=` cunoscut | `assisted` (botul a ajutat) | | comandă fără ref / ref necunoscut | `none` | | re-livrarea aceleiași comenzi | idempotent (upsert pe `external_id`, items nu se

dublează) |

5. 🧠 **De ce idempotent pe** `(business_id, external_id)` : platforma poate re-trimite aceeași comandă (retry). Upsert-ul → nu creezi comenzi duplicate; items doar la insert nou.

6. ⚡ **Ce-ar fi fără atribuire:** n-ai putea dovedi cât a vândut botul → n-ai putea factura clientul pe valoare. `?ref=` + match = ROI-ul agenției.

---

## `redis_bus.py` — backbone-ul de coadă + dedupe + lock

---

0. 📌 **Unde:** `redis_bus.py`. Client Redis partajat.

1. 🗨️ **Analogie:** banda transportoare a bucătăriei + registrul de bonuri văzute + lacătul per masă.

2. ❓ **De ce:** decuplare (webhook XADD, worker XREAD) + dedupe rapid (retry Meta) + serializare per conversație.

3. ⚙️ **Cele 3 mecanisme:** | Funcție | Ce face | `|---|---` | `enqueue_inbound (:66)` | XADD pe stream `inbound` (maxlen ~100k, trim aproximativ) | `seen_before (:53)` | dedupe L1: SET NX EX pe `(account, msg_id)`, TTL 48h | `acquire / release_conv_lock` | lock per conversație (SET NX EX + release Lua compare-del) |

4. 🚨 **Cazuri:** | Situație | Rezultat | `|---|---` | retry Meta (același msg\_id) | `seen_before` True → skip | lock luat de alt worker | `acquire` False → re-queue | worker moare cu lock | TTL 30s → lock expiră (nu blochează pe veci) | release după expirare | Lua compare-del: tokenul nu mai e al nostru → no-op (nu ștergi lock-ul altuia) |

5. 🧠 **De ce release cu Lua compare-del:** dacă lock-ul tău a expirat la TTL și alt worker l-a luat, un `DEL` naiv ar șterge lock-ul ALTUIA. Lua verifică atomic „tokenul e al meu?” înainte de del.

5b. 🧠 **De ce PII de canal DOAR în cheia de lock (nu în log):** `sender_key` conține id-ul expeditorului (PII). E efemer (TTL), NICIODATĂ logat (P12).

6. ⚡ **Ce-ar fi fără maxlen pe stream:** sub backlog uriaș, stream-ul ar crește nelimitat → OOM Redis. Trim-ul aproximativ e cap de siguranță (dar ⚠️ cele mai vechi se pierd sub backlog — vezi Diagrama 9).

---

## Recap Valul 7

---

```
signature → HMAC pe corpul brut (timp constant, fail-closed) – autenticitate
meta → parser Meta → InboundEvent neutru (defensiv, fără DB, <50ms)
body_limit → plasă anti-OOM (413 înainte de bufferizare)
orders → atribuire (?ref → assisted) idempotent – bucla de bani
redis_bus → XADD + dedupe L1 + lock conversație (Lua compare-del)
```

---

## VALUL 8 — Canale (marginile de transport)

---

Cuplajul de canal trăiește DOAR aici (NX-60). Pipeline-ul e agnostic: la INTRARE fiecare canal produce un

`InboundEvent` neutru; la IEȘIRE dispatcher-ul cere un `ChannelSender` din registru. Adaugi un canal = o clasă + o înregistrare.

---

## base.py — contractul de canal (Capability matrix)

---

0. 📌 **Unde:** `base.py`. `Capability`, `InboundEvent`, `ChannelSender` Protocol, registry-uri.
  1. 📌 **Analogie:** fișa de specificații a fiecărui curier: ce poate livra (text? colete? plicuri urgente?). Dispecerul citește fișa și alege ce trimite, degradând la „scrisoare simplă” dacă curierul nu poate mai mult.
  2. ? **De ce Capability matrix (NX-115):** în loc de scară `if hasattr(sender, "send_rich")`, fiecare sender DECLARĂ ce poate (`capabilities`). Dispatcher-ul rutează table-driven. Un canal nou = declară capacități, nu editezi `if/elif`-uri.
  3. ⚙️ **Capabilitățile:** | Capability | Metodă | Ce e | |---|---|---| | TEXT | `send_text` | OBLIGATORIU pentru orice sender | | RICH | `send_rich` | recomandare structurată (carduri + chips) | | CARDS | `send_products` | listă compactă cu butoane | | CAROUSEL | `send_carousel_card` | carusel navigabil | | EDIT | `edit_message_media` | editează cardul (navigare) | | TYPING | `mark_typing` | „scrie...” | | MEDIA | `fetch_media` | download inbound (Vision) | | TEMPLATE | `send_template` | proactiv în afara ferestrei 24h | | OFFER/COMPARISON | (în `send_rich`) | buton CTA / tabel |
  4. ✂️ **Envelope-urile neutre:** `InboundEvent` (mesaj), `StatusEvent` (delivered/read), `CallbackEvent` (apăsare buton). Toate au `to_dict()` cu `kind` → consumer-ul rutează pe `kind`.
  5. 🧠 **De ce IDENTIFIED\_CHANNELS = (whatsapp, telegram):** pe astea id-ul de canal ESTE userul (telefon/chat). Web e anonim → identitatea vine doar din login passthrough (NX-129). Un singur loc de adevăr (cost per-contact, poarta de comandă).
  6. ✂️ **Ce-ar fi dacă un sender n-ar declara TEXT:** dispatcher-ul n-ar avea la ce degrada → mesajul ar rămâne blocat. TEXT e obligatoriu = floor-ul garantat (niciodată tăcere).
- 

## meta\_client.py — WhatsApp Cloud API (send + typing + media)

---

0. 📌 **Unde:** `meta_client.py:28`. Implementează `ChannelSender` + `MediaFetcher`.
  1. 📌 **Analogie:** curierul WhatsApp. Livrează text, arată „scrie...”, trimite template-uri, și aduce pozele pe care le trimite clientul.
  2. ? **De ce injectabil (httpx.AsyncClient):** testele pasează un MockTransport → zero apeluri reale în CI.
  3. ⚙️ **Metodele:** | Metodă | Ce face | |---|---| | `send_text` (:51) | POST `/phone/messages` → wamid; clamp la 4096 | | `send_template` (:75) | template aprobat (Meta randează server-side) | | `mark_typing` (:117) | read + „typing...” într-un call (dispare la ~25s) | | `fetch_media` (:137) | 2 hop-uri: metadata → bytes (cu cap `max_bytes`) |
  4. ✂️ **Cazuri:** | Situație | Rezultat | |---|---| | text > 4096 | clamp cu elipsă (mai bine trunchiat decât respins de Meta) | | răspuns fără message id | `MetaSendError` (dispatcher retry) | | media > `max_bytes` | ridică ÎNAINTE de download (nu bufferiza MB) | | eroare HTTP | se propagă → dispatcher backoff |
  5. 🧠 **De ce send\_template NU trimite textul randat:** Meta randează template-ul server-side din `name` + `params`. Trimiți doar valorile poziționale (`{{1}}`, `{{2}}`). Poarta NX-71 a validat deja consent + approved.
  6. ✂️ **Ce-ar fi dacă erorile n-ar propaga:** un mesaj eșuat ar fi marcat „sent” tăcut → client nu primește nimic. Propagarea → dispatcher retry → livrare eventuală sau `dead` vizibil.
-

## media.py — registry de MediaFetcher (download inbound)

📌 [media.py:34](#) ( `get_media_registry` ). 🤖 biroul de recepție colete: doar WhatsApp poate primi poze azi. ⚙️ singleton per proces (ca `get_llm`); creează `httpx.AsyncClient` DOAR dacă e token Meta. Fără token → registry gol → Gates degradează fail-soft (nicio poză rutată, dar nici excepție). 🧠 cuplajul de transport la margine, zero cod de canal în pipeline.

## web/sender.py — WebSender (SSE prin Redis Pub/Sub)

0. 📌 **Unde:** [web/sender.py:34](#). Implementează `ChannelSender`.

1. 🤖 **Analogie:** un difuzor. „Trimite” = anunță pe canalul vizitatorului ( `web:out:{tenant}:{visitor}` ); handler-ul SSE (abonat) retransmite la browser. Plus un „caiet de rezervă” (backlog) pentru reconectare.

2. ? **De ce Pub/Sub + backlog:** Pub/Sub nu persistă (dacă browserul e deconectat, pierde mesajul). Backlog-ul (LIST cu TTL, ultimele N) e plasa pentru reconectare ( `Last-Event-ID` ).

3. ⚙️ **Cum:**

```
async def _publish(account_id, to, evt):
 await self._redis.publish(out_channel(account_id, to), payload) # PUBLISH ÎNTÂI
 await self._push_backlog(account_id, to, payload) # apoi backlog (atomic MULTI)
def send_rich(payload):
 rendered = render_web(reply_from_outbox(payload), language) # ACELAȘI render ca sync
 return self._publish(..., {"type": "rich", **rendered}) # paritate sync↔async
```

4. 🚨 **Cazuri:** | Situație | Rezultat | |---|---| | `send_text` | `type:"text"` pe SSE | | `send_rich` | `type:"rich"` cu `content+products+chips` (paritate cu `/web/chat`) | | `publish` pică | dispatcher marchează `failed` (retry) — nu se pierde tăcut | | browser reconectat | citește backlog-ul (Last-Event-ID) |

5. 🧠 **De ce PUBLISH ÎNTÂI, backlog DUPĂ:** dacă `publish` pică, dispatcher-ul face retry (nu `sent`) → mesajul nu se pierde. Backlog-ul (reconectare) vine doar după un `publish` reușit (paritate cu `send_text`).

6. 💣 **Ce-ar fi fără backlog:** un client cu net instabil (reconectare) ar pierde mesaje trimise cât era offline. Backlog-ul + Last-Event-ID = livrare la reconectare.

## telegram/ — client + poller (canal de TEST)

📌 [telegram/poller.py:70](#) ( `poll_once` ) + [telegram/client.py](#) ( `TelegramClient` ).

1. 🤖 **Analogie:** un curier de test care, în loc să aștepte scrisori (webhook), merge el la poștă la fiecare 30s să întrebe „aveți ceva pentru mine?” (long polling).

2. ? **De ce polling (nu webhook):** rulează pe VPS **fără HTTPS/tunel** — perfect pentru iterare rapidă pe comportamentul botului. Canal de TEST, aditiv (nu înlocuiește WhatsApp).

3. ⚙️ **Cum (poller):**

```

offset = await redis.get(offset_key) # dedupe: offset în Redis
updates = await client.get_updates(offset, timeout=30) # long poll
for update in updates:
 if "callback_query": answer_callback_query + enqueue(CallbackEvent) # navigare carusel
 else: enqueue(InboundEvent) # pe ACELAȘI stream ca WhatsApp
await redis.set(offset_key, max_update_id + 1) # avansează peste TOATE (chiar ignore)

```

4.  **Cazuri:** mesaj text → envelope neutru → stream (consumer-ul rezolvă `resolve_channel('telegram', bot_id)`); apăsare buton → `answer_callback_query` (oprește spinner-ul) + `CallbackEvent`; media fără text → ignorat (TEST). `TelegramClient` are `send_carousel_card` + `edit_message_media` (RICH pe Telegram).

5.  **De ce offset în Redis:** garantează că nu re-procesezi update-uri confirmate. `inbound_dedupe` (DB) rămâne plasa durabilă. Offset-ul avansează peste TOATE update-urile (chiar ignore), altfel le-ai re-cere la infinit.

6.  **Ce-ar fi dacă un update crapă procesarea:** bucla `run_poller` prinde excepția → log + sleep 3s + retry (nu oprește pollerul, P6).

## web/render.py — randorul web (JSON pentru widget)

 `web/render.py` (`render_web`, `reply_from_outbox`, `flatten_framing`).  decoratorul care aranjează recomandarea pe raftul web (carduri + chips + buton), după contractul FE ([FRONTEND-CONTRACT-IZI.md](#)).  transformă `Reply` / `RichReply` în JSON-ul pe care-l randează widget-ul (content + products + suggestions + offer + comparison). `reply_from_outbox` reconstruiește `Reply` din payload-ul de outbox (calea async).  widget-ul FE trăiește într-un repo SEPARAT — backend-ul emite doar JSON (vezi [\[\[web-render-contract-fe-separate\]\]](#)).

## Recap valurile 7-8

```

INTRARE: signature (HMAC brut) → meta (parser neutru) → body_limit (anti-OOM) → orders (atribuire)
 → redis_bus (XADD + dedupe + lock)
CANALE: base (Capability matrix) – WhatsApp (meta_client) / Telegram (poller+client, TEST) /
 Web (WebSender SSE + render JSON) – cuplaj DOAR la margini (NX-60)

```

**Firul roșu:** autenticitate prin HMAC pe corpul brut; margini subțiri (fără DB, <50ms); envelope neutru → pipeline agnostic de canal; degradare grațioasă la `send_text` (floor garantat); PII de canal doar în chei efemere.

## VALUL 9 — Web gateway (al treilea canal: widget-ul de pe site)

Cum funcționează chat-ul de pe pagina magazinului. Anonim by design (fără login), cu sesiune semnată HMAC, două moduri (sincron + async SSE), și un login passthrough opțional (JWT) pentru clienți autentificați pe eshop.

---

## session.py — sesiune anonimă semnată

---

0. 🔑 **Unde:** `session.py`. `issue_visitor`, `verify_web_session`, `SessionSecretCache`.

1. 🗨️ **Analogie:** o brățară de festival cu un cod semnat. Nu-ți cer buletinul (anonim), dar brățara are un sigiliu pe care doar organizatorul îl poate face → nu poți falsifica brățara altcuiva.

2. ❓ **De ce:** web-ul e anonim (fără cont, fără PII). Dar trebuie să legi mesajele unui vizitator între ele. Soluția: un `visitor_id` generat + semnat HMAC cu secretul tenantului. Clientul nu poate falsifica alt vizitator.

3. ⚙️ **Cum:**

```
def issue_visitor(token, session_secret): # la bootstrap
 visitor_id = f"web_{uuid4()}"
 return visitor_id, hmac(secret, f"{token}:{visitor_id}") # semnătura
def verify_sig(token, visitor_id, sig, secret):
 return hmac.compare_digest(compute_sig(...), sig) # timp constant
```

`SessionSecretCache` = cache LRU+TTL pe `public_token → {business_id, session_secret}` (evită DB la fiecare mesaj), cu **negative cache** (miss-urile se cache-uiesc → un flux de tokenuri invalide nu bombardează DB-ul).

4. 🚩 **Cazuri:** | Situație | Rezultat | |---|---| | bootstrap | emite visitor\_id + sig | | mesaj cu sig validă | sesiune verificată | | sig falsificată | None → 403 (nu distinge de token necunoscut) | | token invalid (spam) | negative cache → nu lovește DB |

5. 🧠 **De ce nu distinge „token necunoscut” de „sig invalidă” (ambele 403):** nu dai un ORACOL atacatorului. Dacă ai răspunde diferit, ar putea enumera tokenuri valide.

5b. 🧠 **De ce cache cu TTL scurt (60s):** revocarea/seed-ul unui canal se propagă repede, dar nu lovești DB la fiecare heartbeat SSE.

6. 💣 **Ce-ar fi fără negative cache:** un atacator ar trimite mii de tokenuri invalide → fiecare ar lovi DB (control plane) → DoS. Cache-uirea miss-urilor = plasă anti-flood.

---

## identity.py — login passthrough (JWT HS256)

---

0. 🔑 **Unde:** `identity.py:28` (`verify_identity_token`). Verificare cu `STDLIB`.

1. 🗨️ **Analogie:** dacă ești deja logat pe site, site-ul îți dă un „bilet VIP” semnat (JWT) pe care botul îl verifică → poate să-ți vadă comenzile reale, fără să te logezi din nou în chat.

2. ❓ **De ce:** un client autentificat pe eshop ar trebui să-și poată verifica comenzile în chat. Site-ul semnează identitatea lui (`customer_ref`) cu `identity_secret` -ul per-tenant → botul o verifică la margine.

3. ⚙️ **Cum (verificare JWT sigură cu stdlib):**

```

if header["alg"] != "HS256": return None, "bad_alg" # pin DUR (anti alg=none)
expected = hmac(secret, f"{header}.{payload}", sha256)
if not compare_digest(expected, signature): return "bad_signature" # timp constant
if time.time() > exp + leeway: return "expired" # exp OBLIGATORIU (anti replay)
return sub, None # sub = customer_ref

```

4. **Cazuri:** | Situație | Rezultat | |---|---| | JWT valid, neexpirat | (customer\_ref, None) → contact verificat | | alg=none (atac clasic) | (None, "bad\_alg") → anonim | | semnătură greșită | (None, "bad\_signature") → anonim | | fără exp | (None, "expired") → anonim (anti replay infinit) | | token invalid | **NU blochează chat-ul** → rămâne anonim |

5. **De ce pin DUR pe HS256 + exp obligatoriu:** două atacuri clasice JWT. alg=none = token nesemnat acceptat; confuzia de algoritm = folosește cheia publică RS256 ca secret HMAC. Pin-ul le blochează. exp obligatoriu = un token furat nu e valabil pe veci.

6. **Ce-ar fi dacă un token invalid ar bloca chat-ul:** un JWT expirat ar rupe conversația. În loc, eșecul → anonim (P6) + motivul în observabilitate (web\_identity\_rejected).

## web/app.py — endpointurile web (bootstrap, messages, chat, stream)

0. **Unde:** web/app.py. 4 endpointuri. Montat DOAR dacă web\_enabled.

1. **Analogie:** recepția widget-ului: îți dă brățara (bootstrap), primește mesajele (messages/chat), și-ți transmite răspunsurile (stream).

2. **De ce 2 moduri (chat sincron + messages async):** unele widget-uri vor răspunsul în același request (sincron, mai simplu); altele vor streaming (async + SSE, mai fluid).

3. **Cele 4 endpointuri:** | Endpoint | Ce face | |---|---| | GET /bootstrap (:136) | emite sesiune + verifică Origin server-side | | POST /messages (:156) | envelope pe stream (async) → reply prin SSE | | POST /chat (:189) | pipeline IN-PROCESS (deliver=False) → răspuns HTTP | | GET /stream (:279) | SSE: abonat la web:out:{visitor} + backlog replay |

4. **Cazuri de securitate:** | Gard | Ce face | |---|---| | Origin server-side (bootstrap) | Origin ne-allowlistat → 403 (CORS-ul de browser nu oprește un bot) | | rate limit (IP + visitor) | 2 contoare: IP prinde rotirea de visitor, visitor prinde spam-ul unui client | | fail-closed pe /chat | Redis jos → 429 (calea cheltuie LLM → nu lăsa să treacă) | | fail-open pe /messages | Redis jos → trece (doar pune pe stream, spend-ul se evaluează în worker) | | gard de buget (chat) | business SAU vizitator peste plafon → 429 |

5. **De ce /chat e fail-CLOSED și /messages fail-OPEN:** /chat cheltuie LLM real per request → un atacator cu token public NU trebuie să ardă bugetul când guard-ul e jos. /messages doar pune un envelope pe stream (ieftin) → indisponibilitatea guard-ului nu blochează ingestia.

5b. **De ce Origin verificat SERVER-side:** CORS-ul de browser blochează doar CITIREA răspunsului de JS cross-origin, NU procesarea pe server (un bot ignoră CORS). Verificarea server-side e apărarea reală.

6. **Ce-ar fi dacă /chat ar folosi outbox:** ar aștepta dispatcher-ul → n-ar putea întoarce răspunsul în același HTTP. De aceea deliver=False (răspunsul HTTP e transportul).

7. **Debug:** grep web\_identity\_verified / web\_identity\_rejected. SSE cu X-Accel-Buffering: no (nginx nu bufferizează streamul).

# VALUL 10 — Domain & config (generalizarea pe orice vertical)

Cum e proiectul GENERIC pe orice magazin (beauty, HVAC, auto), nu hardcodat. DomainPack = config per-vertical; plus utilitarele de text (normalizare, detectare limbă, canonicalizare).

`domain/pack.py` + `loader.py` — **DomainPack (config per-vertical)** 

0. 📌 **Unde:** `pack.py` (DomainPack) + `loader.py` (load\_domain\_pack). Atașat pe `BusinessConfig`.

1. 🧑 **Analogie:** un „kit de personalizare” per tip de magazin. Aceași aplicație, dar cu setările potrivite: la beauty știe „ten gras” → „oily”, la HVAC „zgomotos” → „low\_noise”. Un magazin nou = un kit nou (JSON), nu cod nou.

2. ❓ **De ce (principiul 9):** proiectul e multi-vertical. Ce e specific unui vertical (mapare nevoi, fațete de comparație, praguri de badge) trăiește în DB+seed, NU în cod. Un vertical nou = config, nu deploy.

3. ⚙️ **Cum:**

```
@dataclass(frozen=True)
class DomainPack:
 concern_map: dict # "ten gras" → "oily"
 comparison_facets: tuple[FacetSpec] # rânduri de tabel (finish/material/spf)
 searchable_facets: tuple # ce se poate filtra ("cu niacinamidă")
 badge_rules: dict # praguri Top Favorit / Super Preț
 rank_weights: dict # ponderi de ranking
 risk_terms, greetings, injection_patterns, profile_whitelist, currency...
 # loader: JSON defaults (src/domain/defaults/<vertical>.json) + override businesses.settings
```

4. 📄 **Cazuri:** | Vertical | `concern_map` | `comparison_facets` | |---|---|---| | beauty | „ten gras” → oily, „acnee” → acne | finish, ingrediente, tip de ten | | HVAC | „zgomotos” → low\_noise | BTU, clasă energetică | | auto | (fitment) | compatibilitate, material | | DomainPack lipsă (OFF) | — | cad pe default-urile din cod (byte-identic) |

5. 🧠 **De ce fail-safe (kill-switch OFF → default-uri din cod):** dacă `domain_pack_enabled=false` sau pack-ul e incomplet, consumatorii (taxonomy, fusion, badges) cad pe constantele lor de cod → byte-identic cu comportamentul vechi. Un pack incomplet NU crapă (P6).

6. 💣 **Ce-ar fi dacă maparea ar fi hardcodată în cod:** un vertical nou ar cere deploy + risc de regresie pe ceilalți. DomainPack = extensibilitate fără cod (adaugi un JSON, seed-uiеști, gata).

`normalize.py` — **normalizarea de text partajată**

📌 `normalize.py:17` (`normalize`). 🧑 un „nivelator” care face „Ten Grăs”/„TEN GRAS”/„ten gras” să colapseze la aceeași cheie. ⚙️ `lower + NFKD strip diacritice + trim`. 🧠 baza pentru lookup-uri deterministe (DomainPack,

taxonomy). **NB:** `gates._norm` și `greeting._norm` au variante MAI STRICTE, intenționat diferite — nu se unifică (le-ar schimba comportamentul).

---

## `lang/detect.py` — detectarea limbii (RO/HU/EN)

---

0. 📌 **Unde:** `lang/detect.py:127` ( `detect_language` ). **Pur, zero LLM.**

1. 🗣️ **Analogie:** un „ureche” care ghicește limba după cuvintele-cheie și literele specifice (ă/â pentru RO, ő/ű pentru maghiară).

2. ❓ **De ce determinist (nu LLM):** limba trebuie detectată IEFTIN și RAPID, înaintea straturilor locale-keyed (cache/FAQ/triaj). Un apel LLM ar fi risipă.

3. ⚙️ **Cum:**

```
score[lang] = len(tokens & stopwords[lang]) # câte stopwords ale limbii apar
if chars & diacritics[lang]: score += 2 # bonus pentru diacritice specifice
return best_lang if best_score >= 1 and strict > second # margine → altfel None
```

4. 📄 **Cazuri:** | Mesaj | Rezultat | |---|---| | „vreau un ser pentru ten gras” | ro (stopwords: vreau/un/pentru + „ă” bonus) | | „szeretnék egy krémet” | hu (stopwords + ő/ű) | | „I want a serum” | en | | „ok” (1 cuvânt ambiguu) | None (păstrează limba curentă) | | tie între 2 limbi | None (incertitudine) |

5. 🧠 **De ce precision-first (None la incertitudine):** mai bine păstrezi limba precedentă decât să sari pe o detectare nesigură și să răspunzi în limba greșită (bug, nu doar suboptimal).

6. ⚡ **Ce-ar fi dacă ai returna mereu „cea mai bună”:** un „ok” ar putea fi clasat aleator ca EN → un client RO ar primi brusc răspuns în engleză. Marginea (strict peste a doua) previne asta.

---

## `cache/canonical.py` — canonicalizare + clasificare de volatilitate

---

0. 📌 **Unde:** `canonical.py`. `canonicalize`, `classify_volatility`. **Pur.**

1. 🗣️ **Analogie:** un „standardizator” care face „Cât costă livrarea?” și „cat costa livrarea” să devină aceeași cheie (pentru cache) + un „sortator” care decide dacă o întrebare e cache-abilă sau nu.

2. ❓ **De ce:** cache-ul are nevoie de (1) o cheie stabilă (paraphrase-urile → un entry) și (2) o decizie „e sigur de cache-uit?”.

3. ⚙️ **Cum:**

```
def canonicalize(text): # cheia de cache
 norm = lower + strip diacritice + punctuație→spațiu + colaps spații
 return norm, sha256(norm) # L1 exact pe hash
def classify_volatility(text): # e cacheabil?
 if realtime_words: return "realtime" # comandă/personal → bypass
 if _CONTEXTUAL_RE: return "contextual" # „mai ieftin” → bypass
 if budget/dynamic_words: return "dynamic" # produs/preț → cache cu price-check
 return "static" # FAQ/generic → cacheabil
```

4.  **Cazuri (cele 4 tiere):** | Query | Volatilitate | Cache? | |---|---|---| | „cât e livrarea?” | static | DA (TTL zile) | | „ser cu vitamina C” | dynamic | DA (TTL minute + price-check) | | „unde e comanda mea?” | realtime | NU (specific userului) | | „ceva mai ieftin” | contextual | NU (relativ la setul afișat — cache poisoning) |

5.  **De ce contextual ÎNAINTE de dynamic :** „caut ceva mai ieftin” conține „caut” (dynamic) DAR e o rafinare relativă → trebuie clasat contextual (bypass), nu dynamic . Ordinea verificării contează.

5b.  **De ce \_CONTEXTUAL\_RE e duplicat din agent.\_CHEAPER\_RE :** cache-ul e strat INFERIOR, nu importă din stagiul agent (ar fi dependență inversă). Duplicat intenționat, cu comentariu.

6.  **Ce-ar fi dacă „mai ieftin” ar fi cacheabil:** răspunsul clientului A (baseline 200) ar fi servit clientului B (baseline 50) → cache poisoning. contextual bypass previne.

## Recap valurile 9-10

```
WEB: session (HMAC anonim + negative cache) → identity (JWT HS256, pin dur + exp) →
 app (bootstrap/messages/chat/stream, fail-closed pe /chat, Origin server-side)
DOMAIN: DomainPack (config per-vertical, fail-safe → default-uri cod) → normalize (nivelator) →
 detect (limbă determinist, precision-first) → canonical (cheie cache + volatilitate)
```

**Firul roșu:** web-ul e sigur by design (anonim + HMAC + fail-closed pe calea scumpă); proiectul e GENERIC pe orice vertical prin DomainPack; utilitarele de text sunt deterministe (zero LLM) și precision-first.

## VALUL 11 — Proactiv & joburi (ce face botul FĂRĂ ca clientul să scrie)

Până acum totul a fost REACTIV (clientul scrie → botul răspunde). Aici e PROACTIVUL (botul scrie primul) + mentenanța de fundal (rollup, embed, lifecycle, curățenie) + GDPR.

`proactive/scheduler.py` — **motorul proactiv**

0.  **Unde:** `proactive/scheduler.py:181` ( `run_scheduler` ). Proces separat. Diagrama 10.

1. 🗨️ **Analogie:** un asistent care, din proprie inițiativă, îți scrie „ți-a rămas ceva în coș” sau „a revenit pe stoc produsul pe care-l voiai” — dar DOAR dacă ai fost de acord să primești astfel de mesaje.
2. ❓ **De ce:** recuperezi vânzări pierdute (coș abandonat) + reangajezi clienți (stoc revenit, AWB). Dar cu reguli STRICTE (consent + fereastra 24h Meta) — cele mai reglementate decizii din sistem.
3. ⚙️ **Cum (ca dispatcher-ul):**

```
business_ids = await business_ids_with_due_jobs(conn) # control plane
for business_id:
 async with tenant_conn(business_id):
 jobs = await claim_due_jobs(FOR UPDATE SKIP LOCKED)
 for job:
 async with conn.transaction(): # savepoint per job
 route = get_proactive_route(...) # conversație + canal + destinatar
 spec = build_message_spec(...) # textul per kind
 decision = decide_proactive(...) # POARTA (consent + 24h + template)
 if decision.allowed: enqueue_outbox(f"proactive:{job_id}", ...); mark_job("sent")
 else: mark_job("skipped_no_optin"/"skipped_no_window")
```

4. 📧 **Cazuri:** | Situație | Rezultat | |---|---| | consent + în fereastra 24h | mesaj liber → outbox | | consent + în afara ferestrei + template aprobat | template → outbox | | consent + în afara ferestrei + fără template | skipped\_no\_window | | fără consent | skipped\_no\_optin (nici în fereastră) | | job crapă | savepoint curat → failed (nu rupe lotul) |
5. 🧠 **De ce tot prin outbox (nu trimite direct):** principiul 5 — un singur punct de ieșire. Motorul produce DECIZIA + textul, îl pune în outbox, dispatcher-ul livrează. Zero logică de trimitere duplicată.
6. 💣 **Ce-ar fi fără savepoint per job:** un job stricat ar face rollback la tot lotul → celelalte joburi valide s-ar reprocessa. Savepoint-ul izolează eșecul.

---

## proactive/templates.py — POARTA (consent + 24h + template) 🚦

---

0. 📌 **Unde:** templates.py:83 ( decide\_proactive ). 100% cod determinist, ZERO LLM.
1. 🗨️ **Analogie:** un portar juridic foarte strict. Înainte să lase orice mesaj proactiv să plece, verifică 3 lucruri, în ordine: ai voie (consent)? ești în fereastra permisă (24h)? dacă nu, ai un template aprobat?
2. ❓ **De ce atât de strict:** Meta INTERZICE mesaje libere în afara ferestrei de 24h (doar template-uri aprobate). Iar GDPR/consent interzice marketing fără opt-in. Încălcarea = ban de la Meta + amenzi. Astea sunt cele mai reglementate decizii din tot sistemul.
3. ⚙️ **Cum (3 porți în ordine):**

```
if not _has_optin(consent, kind): return blocked("no_optin") # 1. CONSENT
in_window = await is_in_24h_window(...) # 2. FEREASTRA 24h (funcție SQL)
if in_window: return free(free_text) # în fereastră → mesaj liber
tmpl = await get_approved_template(business, channel, name, locale) # 3. TEMPLATE
if tmpl is None: return blocked("no_window_no_template")
return template(render_template(...), params) # afară → template aprobat
```

4.  **Cazuri consent ( `_has_optin` ):** | Consent | Kind | Rezultat | |---|---|---| | `{marketing: true}` | `abandoned_cart` (marketing) | opt-in | | `{proactive: true}` | `awb_update` (tranzacțional) | opt-in | | `{abandoned_cart: false}` | `abandoned_cart` | opt-out EXPLICIT (bate default-ul) | | `{}` (fără consent) | orice | NU (default = fără opt-in) |

5.  **De ce fereastra 24h e o funcție SQL ( `is_in_24h_window` ), nu recalculată în Python:** e derivat din `last_inbound_at` — sursa de adevăr e DB, nu un flag stocat (poate diverge). Un singur loc de calcul.

5b.  **De ce template NU trimite textul randat (doar name+params):** Meta randează template-ul server-side din `name` + valorile poziționale. `rendered_text` e doar floor de degradare pe canale fără TEMPLATE. Filtru pe `locale` (P11: lipsă în limbă ≠ fallback pe altă limbă).

6.  **Ce-ar fi dacă poarta ar returna `allowed=True` la eroare DB:** ai trimite un mesaj neautorizat (fără să verifici template-ul) → posibil ban Meta. De aceea eroarea se PROPAGĂ (jobul → `failed`, retry), nu „allowed tăcut”.

---

## `proactive/initiators.py` — sweeper-ele care CREEAZĂ joburi

---

0.  **Unde:** `initiators.py:122` ( `run_initiators` ). Rulate de mini-scheduler.

1.  **Analogie:** paznicii care fac ronduri și notează „masa 5 a plecat cu coșul plin, dar n-a plătit” → creează un bilet (job) pentru asistentul proactiv.

2.  **De ce (gap CRITICAL):** motorul + poarta erau gata, dar NIMENI nu insera joburi → zero proactiv în prod. Sweeper-ele scanează surse persistente și creează joburile.

3.  **Cele 2 sweeper-e + 2 seam-uri:** | Sursă | Idempotență | Stare | |---|---|---| | `sweep_abandoned_cart` | `dedupe_key = abandoned_cart:<link_id>` (un reminder/coș) |  LIVE | | `sweep_back_in_stock` | `notified_at` (re-subscribe re-armează) |  LIVE | | `schedule_awb_update` | `awb_update:<order_id>` |  definit, **niciodată apelat** (TODO) | | `schedule_follow_up` | opțional |  definit, **niciodată apelat** (TODO) |

4.  **Cazuri:** coș abandonat > 1h (dar < 7 zile) → job `abandoned_cart` ; abonament la produs care a revenit pe stoc → job `back_in_stock` + marchează abonamentul notificat.

5.  **De ce `schedule_awb_update` / `follow_up` sunt seam-uri neapelate:** AWB e un EVENIMENT (la expediere), nu un sweep. Webhook-ul de comenzi ar trebui să le cheme la `shipments` — dar `shipments` n-are writer azi. Un sweep pe `shipments` gol ar fi cod mort. Seam-urile sunt gata de apelat când sursa apare.

6.  **Debug:** `grep proactive_enqueued / proactive_skipped / proactive_failed`.

---

## `proactive/builders.py` — textul per kind

---

 `builders.py` ( `build_message_spec` ).  redactorul care scrie mesajul potrivit fiecărui tip de job (coș abandonat vs stoc revenit vs AWB), cu `free_text` (în fereastră) + `template_name` + `variables` .  poate întoarce `cancel=True` (ex. coșul a fost deja cumpărat între timp → nu mai trimite).

---

## `jobs/scheduler.py` — mini-scheduler de mentenanță

---

0.  **Unde:** `jobs/scheduler.py:186` ( `_loop` ). Proces separat.

1. 🧑🏻 **Analogie:** îngrijitorul de noapte al clădirii: la ore fixe face curat, actualizează registrele, recalculează statisticile. Nu rescrie logica — cheamă funcțiile existente.
2. ❓ **De ce mini-scheduler intern (nu pg\_cron/celery):** rollup/embed rulează cod Python (nu SQL pur). Cron de sistem e fragil (în afara compose). `asyncio.sleep` e suficient pentru 3-4 joburi periodice.
3. ⚙️ **Joburile:** | Job | Interval | Ce face | |---|---|---| | `rollup_usage` | nocturn (00:10 UTC) | `analytics_events` → `usage_daily` (facturare) | | `embed_products` | 1h | `ai_summary` → `product_embeddings` (doar cu cheie OpenAI) | | `cleanup_dedupe` | 6h | purjă `inbound_dedupe` > 48h | | `proactive_initiators` | 15min | sweep coș abandonat + stoc revenit | | `lifecycle` | nocturn (02:10 UTC) | reclasifică `contacts.lifecycle` (new/engaged/customer/churn) |
4. 🚨 **Cazuri:** heartbeat ( `/tmp/scheduler_alive` ) atins la fiecare buclă → compose verifică vechimea. Un job lent NU fură slotul celui alt (fiecare cu `next_run` propriu). Job picat → log + sărit (P6).
5. 🧠 **De ce cheamă funcțiile existente (nu rescrie):** DRY. Fiecare job are un `__main__` propriu (rulabil manual); scheduler-ul e doar orchestrarea la intervale. `_safe_run` prinde orice excepție → un job picat nu oprește bucla.
6. 🐛 **Debug:** `grep "job .* ok" / "job .* a eșuat"` . `HEARTBEAT` vechi = scheduler blocat.

## jobs/\* — joburile individuale (grupate)

| Fișier                         | Ce face                                                                                                  |
|--------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>rollup_usage.py</code>   | agregă <code>analytics_events</code> de ieri → <code>usage_daily</code> (sursa de FACTURARE)             |
| <code>embed_products.py</code> | generează embeddings pentru produsele cu <code>content_hash</code> schimbat (re-embed doar la schimbare) |
| <code>lifecycle.py</code>      | UPDATE determinist <code>contacts.lifecycle</code> din comenzi + recentă (Val3: era mereu „new”)         |
| <code>cleanup_dedupe.py</code> | purjă <code>inbound_dedupe</code> > 48h (admin_conn)                                                     |
| <code>seed_faqs.py</code>      | seed FAQ RO (script, nu job de buclă)                                                                    |

## gdpr/erase.py — dreptul de ștergere + export (GDPR)

0. 🔑 **Unde:** `gdpr/erase.py:115` ( `erase_contact` ). Pe `admin_conn` (security definer + cross-tabel).
1. 🧑🏻 **Analogie:** biroul juridic care, la cererea unui client („ștergeți-mi datele”), execută ștergerea urmărit și documentat (cine, când, ce — audit).
2. ❓ **De ce:** GDPR — dreptul la ștergere (erase), portabilitate (export), acces (access). Fiecare urmărit în `gdpr_requests` + `audit_log` .
3. ⚙️ **Cum:**

```
req_id = create_request(kind="erase") # cerere → processing
if not contact_in_business(business, contact): failed # izolare: e al acestui tenant?
async with conn.transaction():
 await conn.execute("select gdpr_erase_contact($1)") # funcția SQL security definer
 write_audit("gdpr_erase", ...) # cine/când/ce
mark_done(req_id)
```

`gdpr_erase_contact` (SQL): anonimizează `contacts` (`display_name=NULL`, `profile={}`, `erased_at=now`), **șterge** `channel_identities` (telefonul dispare), NULL-ează `messages.body` (păstrează structura pentru analytics).

4.  **Cazuri:** | Operație | Ce face | |---|---| | erase | anonimizează contact + șterge PII + audit (idempotent) | | export | dump integral (contact + identities + conversații + mesaje + comenzi) | | access | sumar (fără dump-ul integral de mesaje, doar volumul) | | contact al altui tenant | `failed` (izolare) |

5.  **De ce pe `admin_conn` (nu `bot_runtime`):** erase-ul e security definer + atinge PII cross-tabel (`channel_identities`) — operații care transcend scope-ul de tenant runtime. Dar FIECARE query are totuși `business_id` în WHERE (P7). Loguri DOAR cu id-uri (P12).

6.  **Ce-ar fi dacă erase n-ar șterge `channel_identities`:** telefonul (PII) ar rămâne → încălcarea dreptului la ștergere. Funcția SQL îl șterge; mesajele își păstrează structura (`body=NULL`) pentru analytics.

---

## Recap Valul 11

---

```
PROACTIV: scheduler (motor, claim+outbox) → templates (POARTA: consent + 24h + template, zero LLM)
 → initiators (sweep coș abandonat + stoc revenit) → builders (text per kind)
JOBURI: scheduler (mini-cron intern) → rollup (facturare) / embed / lifecycle / cleanup
GDPR: erase (anonimizare + șterge PII) / export / access – urmărit în gdpr_requests + audit_log
```

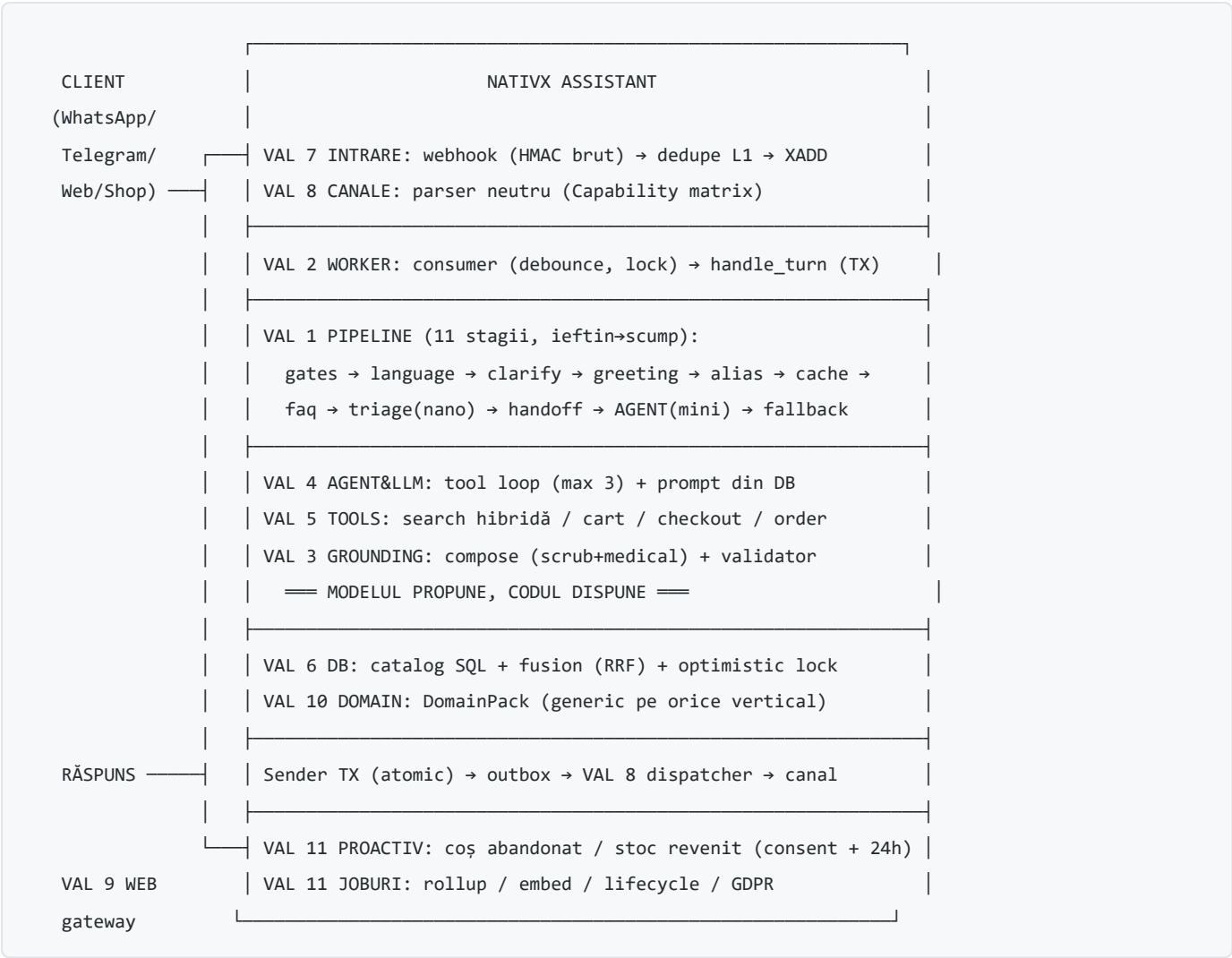
**Firul roșu:** proactivul e cel mai reglementat (consent + 24h, 100% determinist); mentenanța refolosește funcțiile existente (DRY); GDPR-ul e urmărit + izolat pe tenant. Tot prin outbox (P5), tot best-effort (P6).

---

## PROIECT COMPLET — schema mentală a întregului sistem

---

Ai acum TOT proiectul, fișier cu fișier. Iată harta finală, de la un capăt la altul:



## Cele 12 principii — verificate în tot codul

1. **Pipeline liniar** — 11 stagii, early-exit, fără sărituri (VAL 1)
2. **LLM doar în 2 puncte** — triaj (nano) + agent (mini), restul cod (VAL 4)
3. **Un owner per câmp** — TurnContext (VAL 1)
4. **Buget de context în cod** — nu în prompturi (context.py)
5. **Un singur punct de ieșire** — outbox → dispatcher (VAL 2, 8)
6. **Niciodată tăcere** — degradare peste tot (2 găuri: NX-140)
7. **business\_id pe tot** + RLS — izolare (VAL 6)
8. **State = ref-uri** — nu obiecte (VAL 1)
9. **Promptul din DB** — generat, nu hardcodat (VAL 4, 10)
10. **Observabilitate din runner** — stagiile nu știu (VAL 1)
11. **Limba e parte din cheie** — locale în FAQ/cache/template (VAL 1, 10)
12. **PII într-un loc** — channel\_identities (VAL 6, 11)

## Cele 3 idei care leagă TOT proiectul

---

1. **Modelul propune, codul dispune.** LLM-ul dă cuvinte + ID-uri; codul pune faptele (preț/link/badge) din DB. Apărat de validator (proză) + compose (rich). Zero halucinație STRUCTURAL.
  2. **Decuplare + self-healing peste tot.** Coadă (Redis) + outbox (DB) decuplează; dedupe/optimistic lock/visibility timeout/price-check se auto-vindecă. Niciodată tăcere (cu 2 excepții documentate: NX-140).
  3. **Generic prin construcție.** Multi-tenant ( `business_id` + RLS), multi-canal (Capability matrix), multi-vertical (DomainPack), multi-limbă (locale în cheie). Un client/canal/vertical nou = config, nu cod.
- 

**Ai terminat tot proiectul.** Poți acum: explica fiecare fișier, urmări orice mesaj prin cod, depana în producție, adăuga feature-uri fără să strici, și onboarda alt dev. 🏠

# Prompt de predare — „Deep Dive pe înțeles” (motorul masterclass-ului)

Ce e asta: șablonul (prompt-ul) pe care îl aplic la FIECARE componentă (fișier / funcție / caz) când aprofundez codul. Garantează că explicația e mereu: pe înțeles, cu exemplu concret la fiecare caz, cu „de ce așa” ÎNAINTE de „cum”. Îl poți refolosi în orice sesiune sau da altui dev.

**Cum se folosește:** „Aplică PROMPT-ul de predare pe `<fișier:funcție>`” → primești explicația în formatul de mai jos. Se aplică la o funcție, un bloc de decizie, sau un fișier întreg (pe rând, funcție cu funcție).

## Reguli de ton (obligatorii)

1. **Limbaj simplu întâi, jargon după** — orice termen tehnic (idempotent, RLS, embedding, RRF) e explicat în paranteză prima dată când apare, cu o vorbă de zi cu zi.
2. **Analogie din viața reală** la fiecare componentă — o metaforă de 1-2 propoziții (magazin, restaurant, poștă).
3. **DE CE înainte de CUM** — întâi problema pe care o rezolvă, apoi codul.
4. **Exemplu CONCRET la FIECARE caz** — pentru orice ramură/if/decizie: *input real* → *ce se întâmplă* → *output real*. Exemplele folosesc magazinul demo (seruri, vitamina C, ten gras, sub 100 lei) ca să fie palpabile.
5. **Cod real, nu pseudocod** — citate din fișier cu `fișier:linie`. Fiecare linie ne-evidentă, explicată.
6. **Fără să sar peste raționament** — dacă ceva pare arbitrar, explic trade-off-ul (ce alternativă a fost respinsă).
7. **Leagă de diagramă** — spun ce nod din `ARCHITECTURE-WORKFLOWS.md` corespunde.
8. **Junior-friendly** — presupun că nu știi codebase-ul. Nimic „evident”.

## Structura fixă a fiecărui Deep Dive (9 secțiuni)

Când aprofundez o componentă, produc EXACT aceste 9 secțiuni, în ordine:

### 0. 📌 Unde suntem

- `fișier:linie` + numele funcției.
- Ce stagi / subsistem. Ce nod din care diagramă.
- Ce primește (input) și ce lasă în urmă (ce câmp din `TurnContext` scrie / ce efect are).

### 1. 🗣️ Analogia

- O metaforă de zi cu zi (1-2 propoziții) care surprinde esența. Reperul mental la care te întorci.

## 2. ? DE CE există (problema)

- Ce s-ar strica / ce ar fi greșit fără componenta asta. Problema concretă pe care o rezolvă.
- Dacă a fost adăugată ca fix la un bug real, spun care (NX-XXX / PR).

## 3. ⚙️ CUM funcționează (codul, bucată cu bucată)

- Codul real, împărțit în felii mici. Fiecare felie: ce face, în cuvinte simple.
- Variabilele cheie: ce conțin, de unde vin.

## 4. 📄 FIECARE caz, cu exemplu concret

- Pentru fiecare ramură/decizie (if/elif/else, fiecare condiție dintr-un guard):
- **Condiția:** ce se verifică, în română.
- **Exemplu de intrare:** un mesaj/stare concretă.
- **Ce se întâmplă:** pas cu pas.
- **Ieșirea:** ce primește clientul / ce se schimbă în state/DB.
- Tabel când sunt multe cazuri.

## 5. 🧠 DE CE așa (și nu altfel)

- Trade-off-urile. Ce design alternativ ar fi părut mai simplu și de ce a fost respins.
- Legătura cu principiile (1-12) din CLAUDE.md.

## 6. 💣 CE-AR FI DACĂ (îl strici)

- Dacă ștergi componenta → ce se rupe (scenariu concret).
- Dacă o modifici greșit (schimbi o condiție, muți o linie) → ce bug apare, cu exemplu.

## 7. 🐛 Bug-uri comune + cum depanezi

- Simptome tipice. Ce `grep` -uiești în log. Unde pui breakpoint. Ce variabile inspecțezi.

## 8. ✅ Test rapid (fără răspuns)

- 1-3 întrebări (grilă / „ce se întâmplă dacă” / „găsește bug-ul”) pe care le rezolvi tu, apoi te corectez.

---

## Progresia (ordinea de aprofundare)

---

Aplicăm prompt-ul pe componente în ordinea în care execuția le atinge, de la simplu la complex:

1. `agent.py` — cele 3 faze (PRE-loop intenții deterministe → tool loop → POST-loop compunere + validator), funcție cu funcție.
2. `catalog_tools.py` — căutarea hibridă (moștenire sesiune → scara de relaxare → lexical+semantic → fuse → diversify → dedupe).
3. `compose.py` — grounding-ul (membership → scrub câmp-cu-câmp → medical → badge → pick → chips).
4. (opțional, la cerere) orice alt fișier: `dispatcher.py`, `commerce_tools.py`, `web/app.py`, etc.

După fiecare funcție aprofundată: un test rapid. La finalul fiecărui fișier: un recap + o schemă mentală.

---

## Exemplu de aplicare (format de referință)

---

Vezi mai jos o aplicare completă a prompt-ului pe `_handle_link_intent` (`agent.py:894`) — așa arată fiecare Deep Dive. Dacă formatul e bun, îl rulez identic peste tot restul.